

Chapter 1

Introduction to Software Engineering



Lesson Objectives

- Differentiate between **Generic and Bespoke** software product
- Describe what is software engineering and its **importance**
- Discuss the **key challenges** of software engineering and the **professional responsibilities** for software engineers

Definition

Software consists of computer programs that control the workings of the computer hardware and program documentation used to explain the programs to the user.

IEEE defines software as a collection of programs, instructions, and data that, when executed, produce a desired result. It also includes the necessary documentation that describes the operation and use of the programs.

Benefits of software

Software offers numerous benefits that contribute to increased efficiency, productivity, innovation, and overall improvement in various areas. Here are some key benefits of software:

- **Automation and Efficiency:** Software automates repetitive and time-consuming tasks, leading to increased efficiency and productivity. It streamlines processes, reduces manual effort, and allows organizations to accomplish more in less time.
- **Accuracy and Reliability:** Software eliminates human errors and inconsistencies that can occur in manual processes. Once programmed correctly, software consistently executes tasks with precision and reliability, reducing the risk of mistakes and improving data accuracy.
- **Scalability and Flexibility:** Software can be scaled up or down to meet changing demands. It provides the flexibility to adapt to evolving requirements, whether it's accommodating a growing user base, expanding functionality, or integrating with other systems.
- **Improved Decision Making:** Software enables data collection, analysis, and visualization, providing valuable insights for informed decision-making. It helps organizations leverage data to identify trends, patterns, and opportunities, leading to better strategic choices and outcomes.

Benefits of software

- **Innovation and Competitive Advantage:** Software is a catalyst for innovation and a source of competitive advantage. By developing new software applications, organizations can introduce innovative products or services, optimize operations, and differentiate themselves in the market.
- **Improved Customer Experience:** Software enables organizations to deliver enhanced customer experiences. Customer relationship management (CRM) systems, personalized marketing tools, and customer service software enable better customer engagement, support, and satisfaction.
- **Cost Savings:** Software can help reduce costs in various ways. It automates tasks that would otherwise require manual labor, reduces paperwork, optimizes resource allocation, and eliminates the need for physical storage. It also enables remote work, reducing office space and infrastructure expenses.
- **Accessibility and Convenience:** Software provides easy access to information and services anytime, anywhere. Cloud-based software, mobile applications, and web-based platforms ensure convenient access across devices, enabling users to work, interact, and access information on the go.
- **Continuous Improvement and Updates:** Software can be regularly updated and improved to address evolving needs, fix bugs, and enhance functionalities. This allows organizations to stay current, adapt to technological advancements, and deliver better experiences to users.

Functions of Software



Functions of Software

The functions of software are to: -

- **Manage** the computer resources of the organisation
- **Provide tools** for human beings to take advantage of these resources
- **Act as an intermediary** between organisations and stored information.

Characteristics of Software



Developed or
Engineered

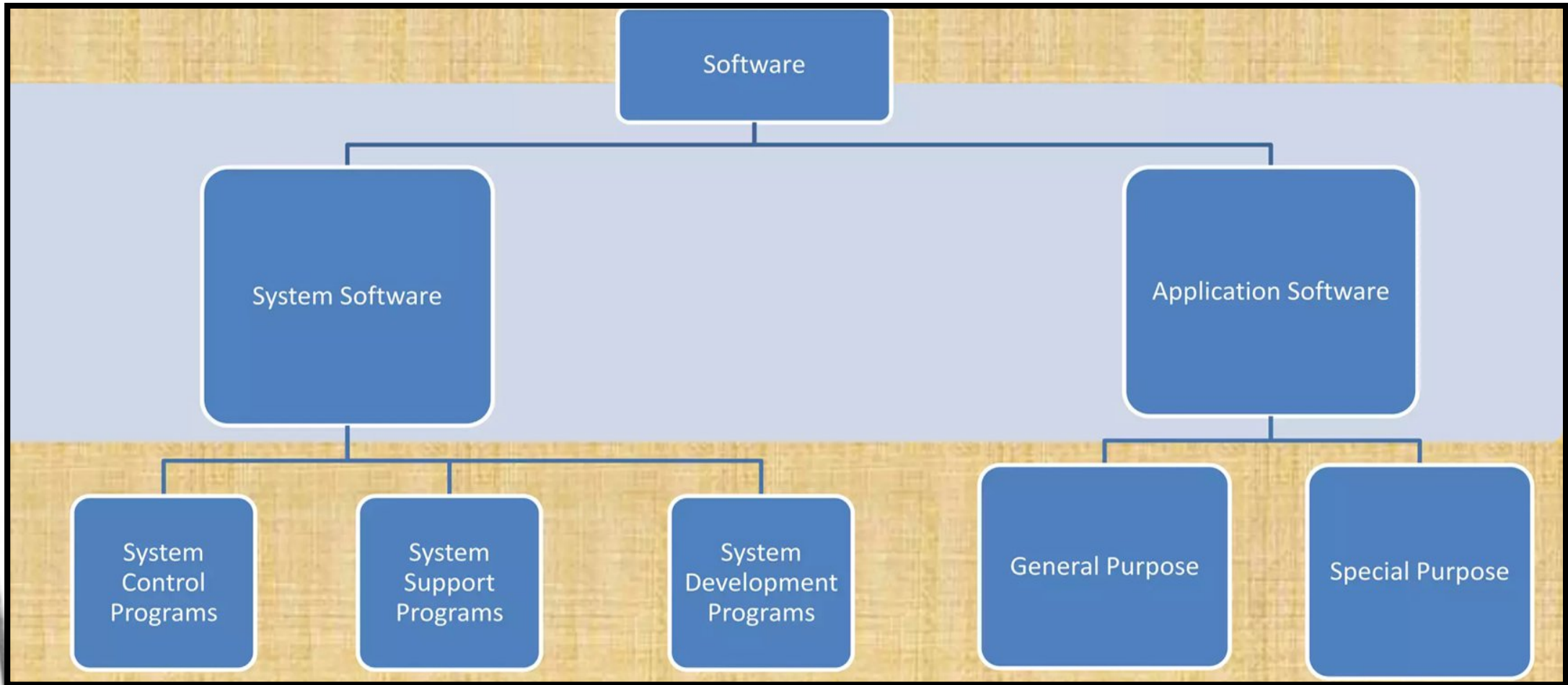
Doesn't "wear out"

Continues to be
custom built

Characteristics of Software

- Software is **developed or engineered**, it is not manufactured in the classical sense
- Software **doesn't “wear out”**
- Most software continues to be **custom built**

Types of Software



System Software

- The system software is collection of programs designed to operate, control and extend the processing capabilities of the computer itself.
- These are generally prepared by computer manufacturers.
- These software perform a variety of functions like file editing, storage management, resource accounting, I/O management, etc.

Types of System Software

1. System Control Programs :

They control the execution of programs, manage the storage and processing resources of the computer and perform other management and monitoring functions. e.g., OS

2. System Support Programs :

They provide routine service functions to other computer programs and computer users. e.g., Utility Programs

3. System Development Programs :

They assist in the creation of publication programs.

e.g., Language translators like interpreters, compilers and assemblers

Types of Software



Examples of System
Software

Operating System

Translators

Compilers

Loaders

Utility Programs

What is Application Software?

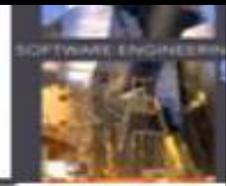
- ▶ **Application software** is computing software designed to carry out a specific task other than relating to the operation of the computer itself, typically to be used by end-users
- ▶ **App** is a very common term for an application, especially for simple applications that can be downloaded cheaply or even for free. Many apps are also available for mobile devices and even some TVs.



Types of Application software

- Software engineers are concerned with developing software products, i.e. software, which can be sold to a customer.
- Two types of software product:
 - **Generic** products
 - **Bespoke** (or customized) products

Application software



✧ Generic products

- Stand-alone systems that are marketed and sold to any customer who wishes to buy them.
- Examples – PC software such as graphics programs, project management tools; CAD software; software for specific markets such as appointments systems for dentists.

✧ Customized products

- Software that is commissioned by a specific customer to meet their own needs.
- Examples – embedded control systems, air traffic control software, traffic monitoring systems.

3 types of generic software

1. **Desktop App** - is applications that run in a personal computer and laptops.
2. **Web App** - is an application that is accessed over a network such as the Internet or an intranet.
3. **Mobile App** is a software application designed to run on smartphones, tablet computers and other mobile devices.

I. Desktop Applications

There are countless desktop applications out there, and they fall into many different categories. Some are more full-featured (like Microsoft Word), while others may only do one or two things (like gadgets). Below are just a few types of applications that you might use:

The Types of Desktop Application:

1. Word Processors
2. Web Browsers
3. Games
4. Media Players
5. Gadgets



II. Web App

- is an application that is accessed over a network such as the Internet or an intranet.

The Types of Web Application:

1. **Online Email Services**
2. **Google Docs**
3. **Facebook**



III. Mobile App

- ▶ is a software application designed to run on smartphones, tablet computers and other mobile devices.
- ▶ Desktop and laptop computers aren't the only devices that can run apps. You can also download apps for mobile devices like smartphones and tablet computers, which opens up a lot of new possibilities. Here are a few examples of mobile apps:

The Types of Mobile Application:

1. **Snapchat**
2. **Messenger**
3. **Instagram**



Customised software

- A customised software is a tailored software solution built specifically for a business while considering their business requirements and goals.
- Customized software development is the procedure in which developers design, create, deploy, and maintain a tailored software solution according to the required functions and features of an organization.

Examples of customised software

- Banking services-manages the customer assets, tracks down the value among different currencies, and even fraud detection software's
- TAXI AND PRIVATE HIRE This gives the users a map, drivers who are currently above, their ratings, and the payment options for the customer to pay for their ride.
- DELIVERY This software lets riders get to restaurants and delivery addresses and allows the customers where their order is at that time.

Attributes of Good Software

User view

Functionality

Efficiency

Availability

Reliability

Secure

Flexiblility

Usability

Attributes of Good Software

Developer's View

Manageable

Reusable

Buildable

Maintainable

Attributes of Good Software-user view

- Availability - the probability that a system will work as **required** when required during the period of a mission
- Functional – the system will perform the **functions** that it is required it
- Efficient - the system performs those functions efficiently in **terms of time and resources**
- Reliable - not prone to hardware or software **failure**, will deliver the functionality when the users want it

Attributes of Good Software-user view

- Secure - **protected** against errors, attacks and loss of valuable data
- Flexible - capable of being **adapted to new uses**, to run in **different countries** or to be moved to a different **platform**
- Usable - provides users with a **satisfying experience** (not a source of dissatisfaction)

Attributes of Good Software –Developer view

- Manageable - easy to **estimate work** involved and to check of progress
- Maintainable - design makes it possible for the maintenance programmer to **understand** the designer's intention
- Reusable - **elements** of the system can be reused in other systems
- Buildable - **Design is not too complex** for the developers to be able to implement it

Software engineering



- ✧ The economies of ALL developed nations are dependent on software.
- ✧ More and more systems are software controlled
- ✧ Software engineering is concerned with theories, methods and tools for professional software development.

Software costs



- ✧ Software costs often dominate computer system costs. The costs of software on a PC are often greater than the hardware cost.
- ✧ Software costs more to maintain than it does to develop. For systems with a long life, maintenance costs may be several times development costs.
- ✧ Software engineering is concerned with cost-effective software development.

What is Software Engineering?

- Software engineering is an **engineering discipline**, which is concerned with **all aspects of software production** from the early stages of system specification through to maintaining the system after it has gone into use.

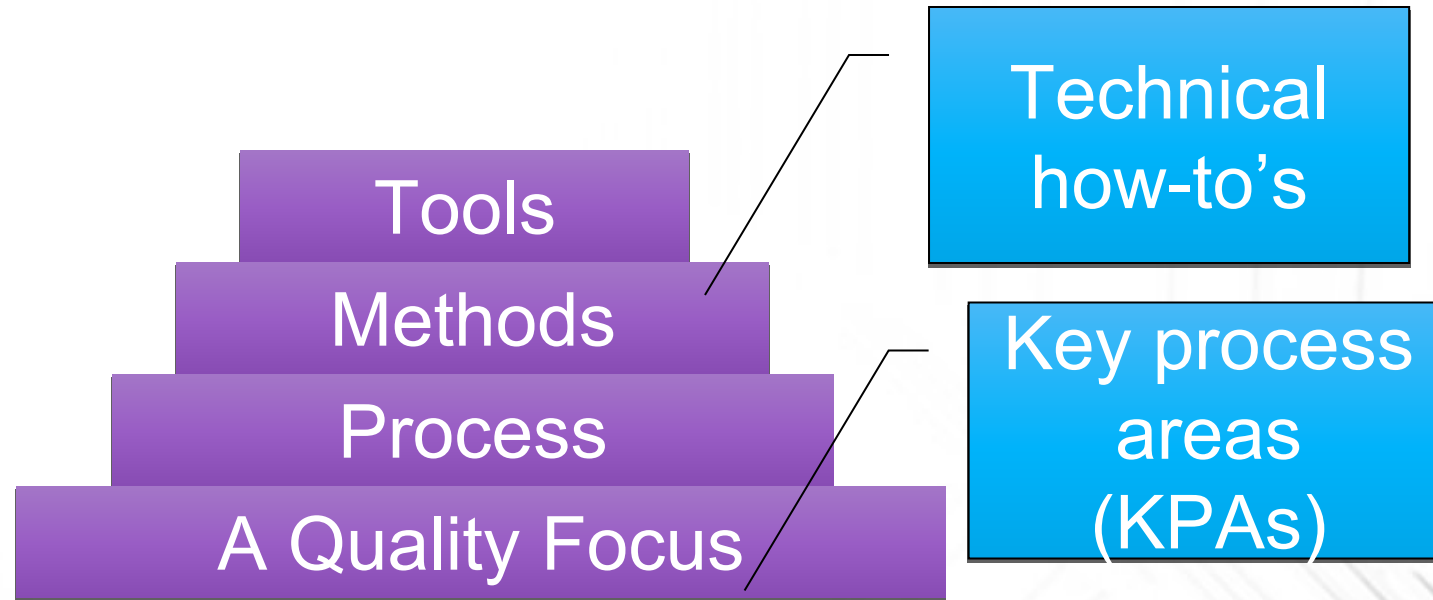
Software Engineers apply **theories**, **tools** & **methods** that are appropriate to discover solutions & work within company's constraints

What is Software Engineering?

- Engineering discipline:
 - Software Engineers apply theories, tools & methods that are appropriate to discover solutions & work within company's constraints.
- All aspects of software production:
 - Technical processes, software project management, development of tools, methods to support software production.

What is Software Engineering?

- Software engineering is a layered technology consisting of the following four important layers



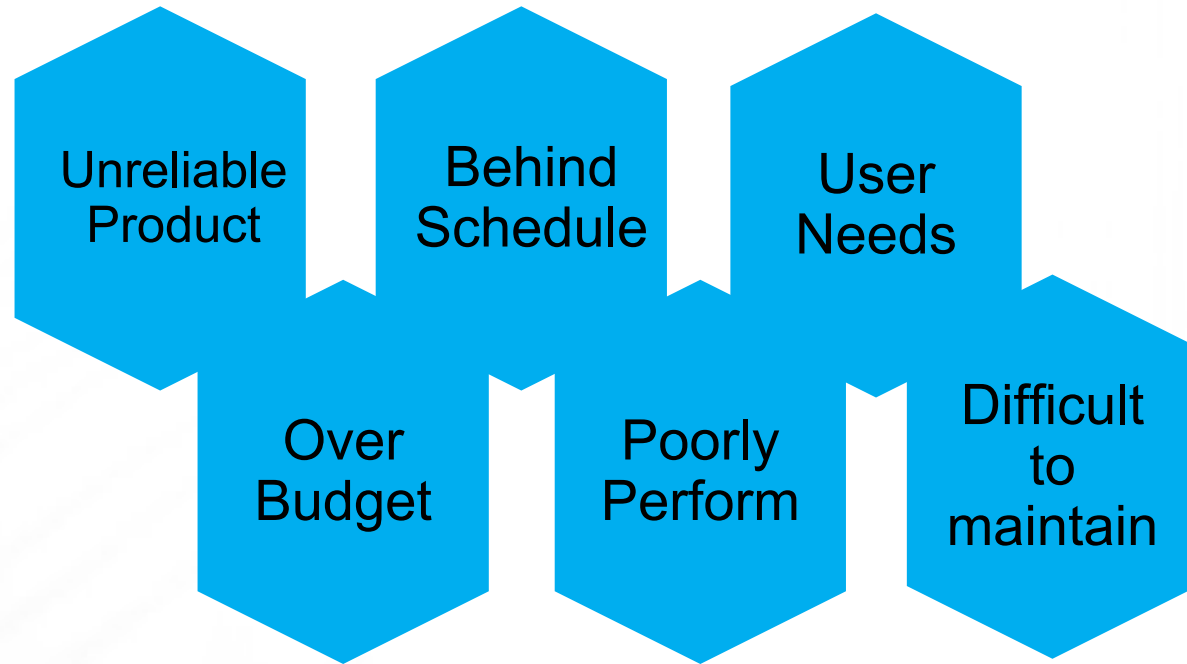
What is Software Engineering?

- Quality Focus
 - Any engineering approach must rest on an organizational commitment to quality.
 - Total quality management and similar philosophies foster a continuous process improvement culture, and this culture ultimately leads to development of increasingly more mature approaches to software engineering.
- Process
 - Process defines a framework for a set of key process areas (KPAs) that must be established for effective delivery of software engineering technology.

What is Software Engineering?

- Methods
 - Methods are organized ways of producing software. They include **suggestions** for the process to be followed, the **notations** to be used, **rules** governing the system descriptions, which are produced and **design guidelines**.
- Tools
 - Tools provide automated and semi-automated **support** for the process and the methods.

Importance of Software Engineering



***SOFTWARE
PROJECT***



Importance of Software Engineering

Hence, software engineering is an engineering discipline whose goal is **cost-effective** development of software system

***SOFTWARE
PROJECT***



Key Challenges of Software Engineering

Delivery
Challenge

Legacy Challenge

Heterogeneity/
diversity
Challenge



Key Challenges of Software Engineering

- The **legacy challenge**
 - the challenge of maintaining & updating *legacy software* in such a way that excessive costs are avoided and essential business services continue to be delivered
- The **heterogeneity / diversity challenge**
 - the challenge of developing techniques to build dependable software which is flexible enough to cope with different types of computer & support systems

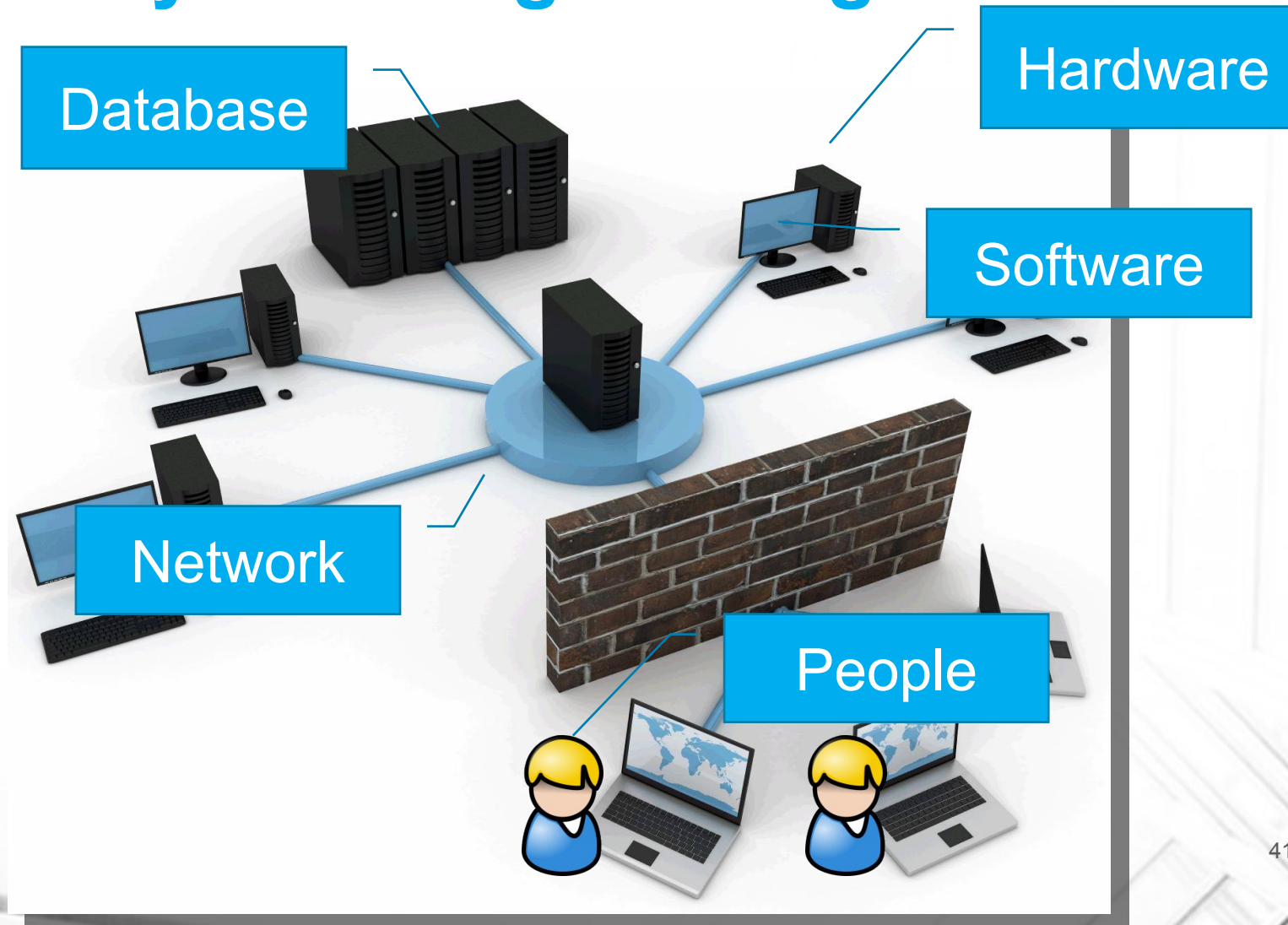
Key Challenges of Software Engineering

- The **delivery challenge**
 - the challenge of shortening delivery times for large & complex systems without compromising system quality.
 - Break the work into small, testable parts, automate checks, integrate often, limit work-in-progress.

What is System Engineering?

- Systems engineering is an interdisciplinary field of engineering that focuses on how **complex** engineering projects should be **designed** and **managed**.
- Issues such as logistics, the coordination of different teams, and automatic control of machinery become more difficult when dealing with large, complex projects.

What is System Engineering?



What is System Engineering?

- **Systems engineering** deals with work-processes and tools to handle such projects, and it overlaps with both technical and human-centered disciplines such as control engineering and project management.
- **System engineering** deals with all aspects of computer-based system development.

What is System Engineering?

- **Software engineering** is a part of **system engineering**.
- **System engineering** is to identify the roles of hardware, software, people, database and other system elements involved with that system which is going to be developed.
- **Software engineering** is to tell the practicalities of developing and delivering useful software.

Professional & Ethical Responsibility

Software engineering ethics



- ✧ Software engineering involves wider responsibilities than simply the application of technical skills.
- ✧ Software engineers must behave in an honest and ethically responsible way if they are to be respected as professionals.
- ✧ Ethical behaviour is more than simply upholding the law but involves following a set of principles that are morally correct.

Issues of professional responsibility



✧ Confidentiality

- Engineers should normally respect the confidentiality of their employers or clients irrespective of whether or not a formal confidentiality agreement has been signed.

✧ Competence

- Engineers should not misrepresent their level of competence. They should not knowingly accept work which is outwith their competence.

Issues of professional responsibility



✧ Intellectual property rights

- Engineers should be aware of local laws governing the use of intellectual property such as patents, copyright, etc. They should be careful to ensure that the intellectual property of employers and clients is protected.

✧ Computer misuse

- Software engineers should not use their technical skills to misuse other people's computers. Computer misuse ranges from relatively trivial (game playing on an employer's machine, say) to extremely serious (dissemination of viruses).

ACM/IEEE Code of Ethics

- The professional societies in the US have cooperated to produce a code of ethical practice.
- We have these posted along our inner hallways.
- Members of these organizations sign up to the code of practice when they join.
- Absolutely you need to belong to the ACM, IEEE CS and perhaps a couple of others in your area of specialization.
- The Code contains eight Principles related to the behavior of and decisions made by professional software engineers, including practitioners, educators, managers, supervisors and policy makers, as well as trainees and students of the profession.

Professional & Ethical Responsibility

- Professional responsibilities for software engineers include:



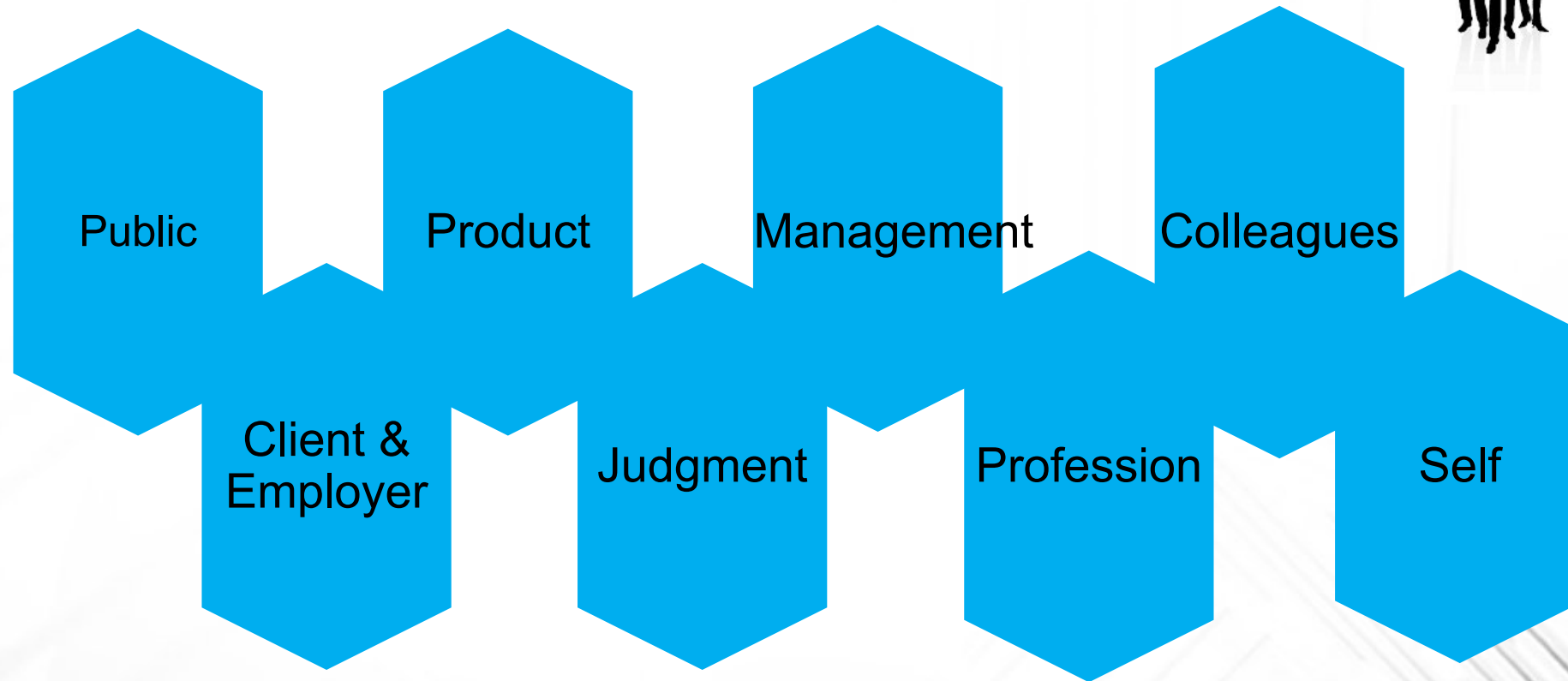
- ✓ Confidentiality
- ✓ Competence
- ✓ Intellectual property rights
- ✓ Computer misuse

ACM/IEEE Code of Ethics

- The Software Engineering Code of Ethics focuses on providing high-quality software. They are committed to analyzing, specifying, designing, developing, testing, and maintaining software that is beneficial and effective for the company or client.

Professional & Ethical Responsibility

ACM/IEEE Code of Ethics



Professional & Ethical Responsibility



ACM/IEEE Code of Ethics

1. **PUBLIC** - Software engineers shall act consistently with the public interest.
2. **CLIENT AND EMPLOYER** - Software engineers shall act in a manner that is in the best interests of their client and employer consistent with the public interest.
3. **PRODUCT** - Software engineers shall ensure that their products and related modifications meet the highest professional standards possible.
4. **JUDGMENT** - Software engineers shall maintain integrity and independence in their professional judgment.

Professional & Ethical Responsibility



ACM/IEEE Code of Ethics

5. **MANAGEMENT** - Software engineering managers and leaders shall subscribe to and promote an ethical approach to the management of software development and maintenance.
6. **PROFESSION** - Software engineers shall advance the integrity and reputation of the profession consistent with the public interest.

Professional & Ethical Responsibility



ACM/IEEE Code of Ethics

7. COLLEAGUES - Software engineers shall be fair to and supportive of their colleagues.
8. SELF - Software engineers shall participate in lifelong learning regarding the practice of their profession and shall promote an ethical approach to the practice of the profession.

Revision

- Generic vs. Bespoke software product
- Software Engineering (SE) & its importance
- Software Attributes
- Key challenges of SE
- ACM/IEEE code of ethics

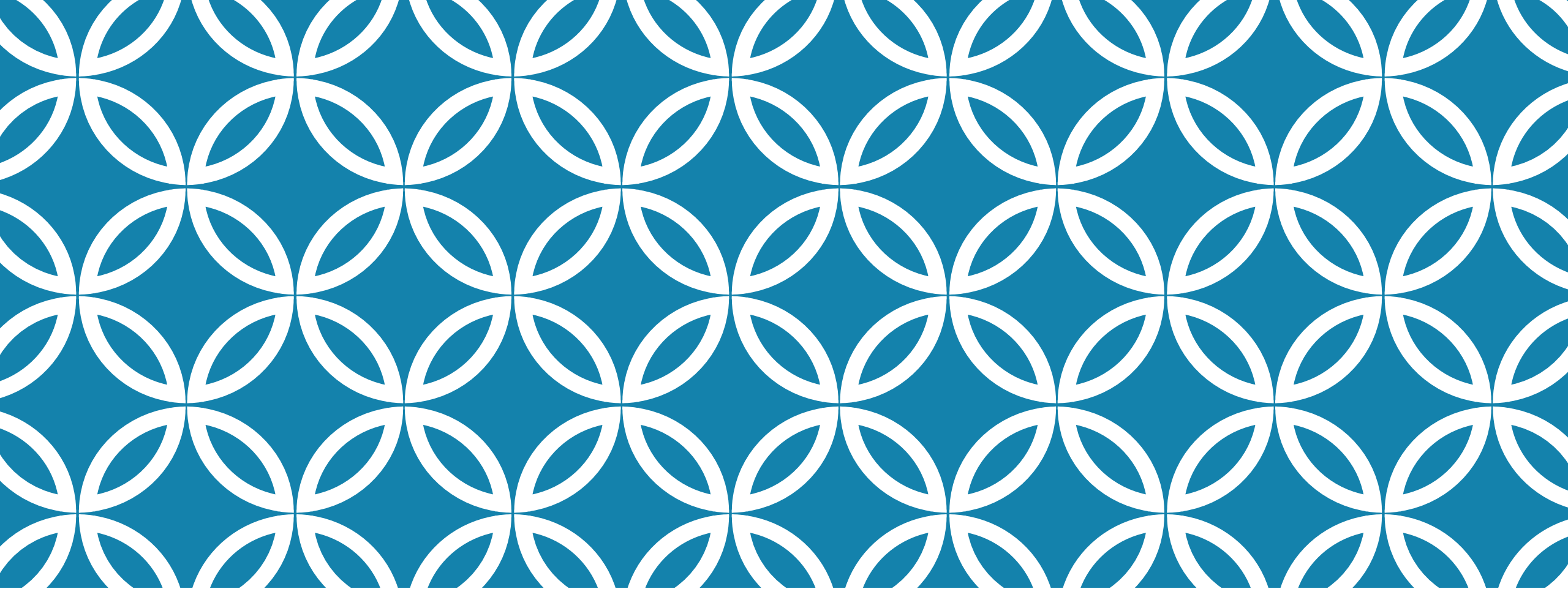
Q&A

How to develop software to meet all the requirements and fulfil the task?

Answer: The Software Development Life Cycle (SDLC) is a framework that can help in developing software to meet requirements and fulfill the task effectively. the goal of SDLC is to minimize project risks through planning so that software meets customer expectations during production and beyond. This methodology outlines a series of steps that divide the software development process into tasks you can assign, complete, and measure.



Thank you



CHAPTER 2

Software process
model

OVERVIEW

Definition of process and its characteristics

Example of software process model

The software process models characteristics

Agile and it's characteristics

WHAT IS A PROCESS?

- ⦿ A series of steps taken to produce an intended output
- ⦿ Steps involves
 - Activities
 - Constraints
 - Resources
- ⦿ Process involves
 - Tools
 - techniques

CHARACTERISTICS OF PROCESS

- ⦿ Prescribes all major process activities
- ⦿ Uses resources, subject to set of constraints (such as schedule, no. of people working)
- ⦿ Produces intermediate and final products
- ⦿ May be composed of sub-processes with hierarchy or links
- ⦿ Each process activity has entry and exit criteria
- ⦿ Activities are organized in sequence, so timing is clear
- ⦿ Each process guiding principles, including goals of each activity
- ⦿ Constraints may apply to an activity, resource or product

WHY ARE PROCESSES SO IMPORTANT

- ◉ Impose consistency and structure on a set of activities
- ◉ Guide us to understand, control, examine, and improve the activities
- ◉ Enable us to capture our experiences and pass them along

SOFTWARE PROCESS MODEL

May include activities which are part of the software process, software products and the roles of people involved in software engineering

The models specify the various stages of the process and the order in which they are carried out

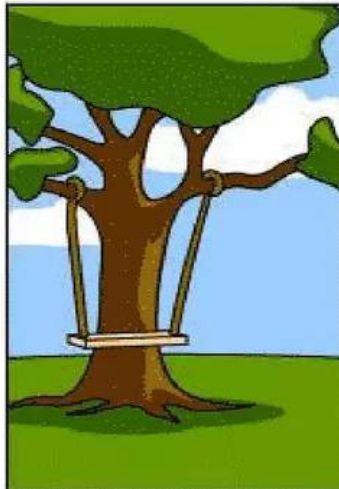
Advantages of using Software Process Model

- Increased development speed
- Increased product quality
- Improved tracking & control
- Improved client relations
- Decreased project risk
- Decreased project management overhead

PERSPECTIVES



How the customer explained it



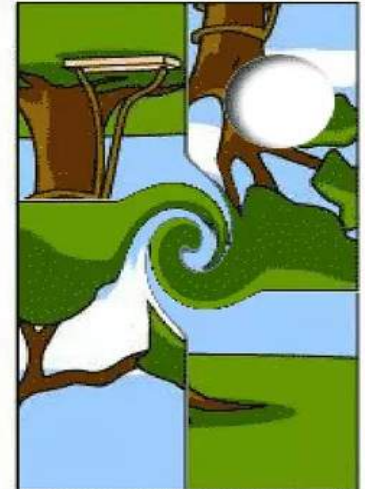
How the Project Leader understood it



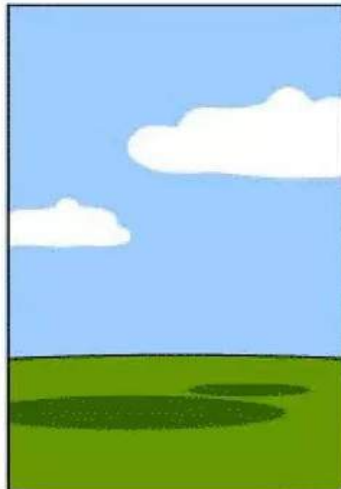
How the Analyst designed it



How each developer integrated with others



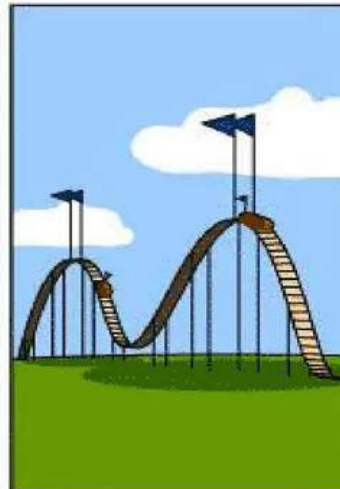
How QA got the 1st, 2nd, and 3rd build



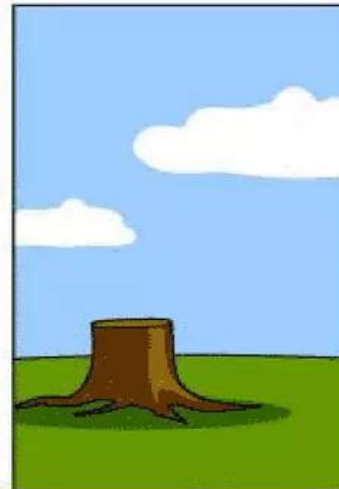
How the project was documented



How the Business Consultant described it



How the customer was billed



How it was supported



What the customer really needed

HOW TO SELECT THE RIGHT SOFTWARE PROCRSS MODEL

- Selecting the right SDLC is a process in itself that organization can implement internally or consult for. There are some steps to get the right selection

Learn the
about SDLC
Models



Assess the
needs of
Stakeholders



Define the
criteria

The selection process



Step 1

To select the right SPM, you should have enough experience and be familiar with the SPMs that will be chosen and understand them correctly. As described in the software process model.

STEP 2: Assess the needs of Stakeholders

We must study the business domain, stakeholder's concerns and requirements, business priorities, our technical capability and ability, and technology constraints to be able to choose the right SPM against their selection criteria.

The selection process

STEP 3: Define the criteria

❖ Some of the selection criteria or arguments that you may use to select an SPM are:

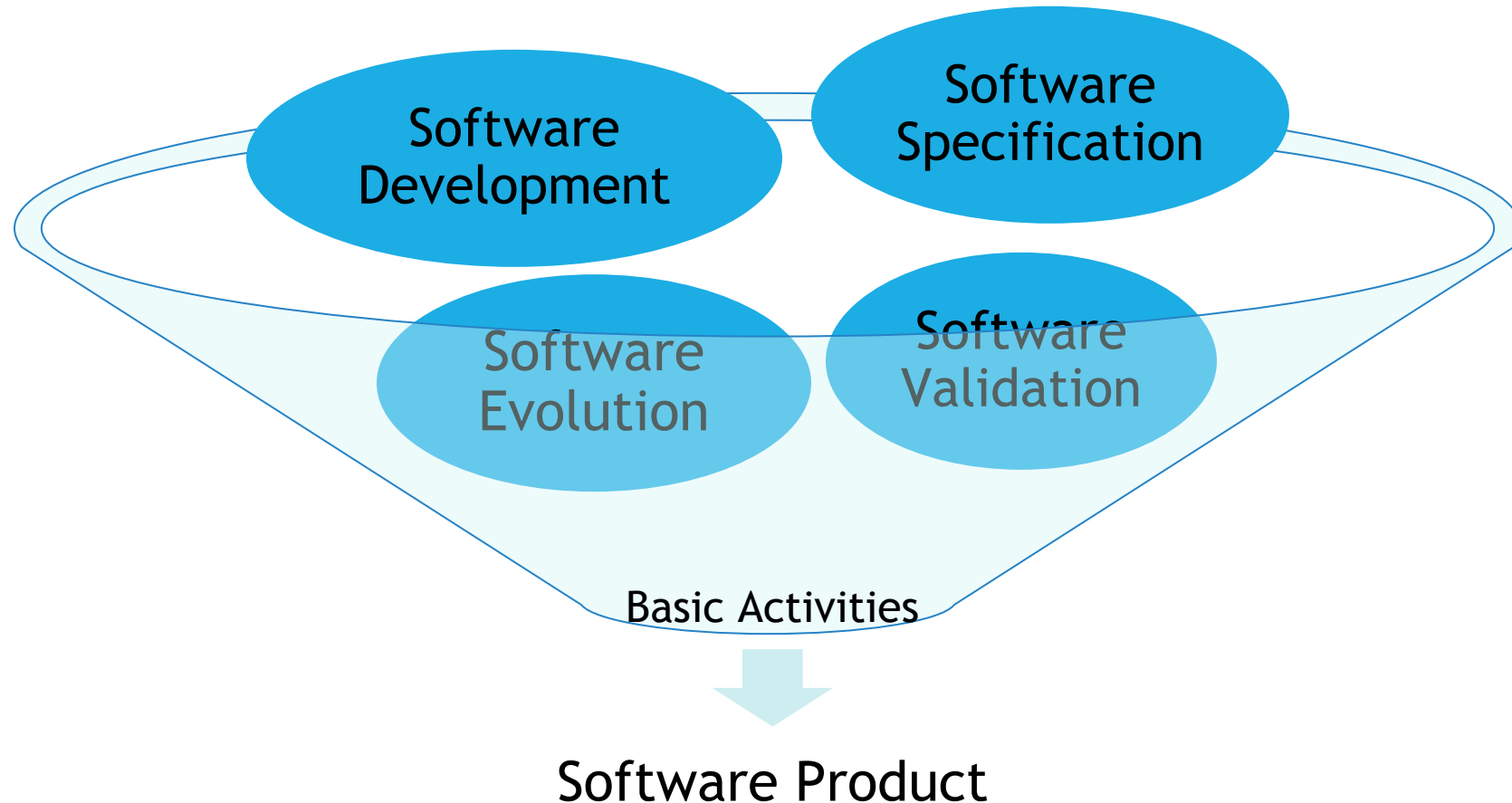
-  Is the SPM suitable for the size of our team and their skills?
-  Is the SPM suitable for the selected technology we use for implementing the solution?
-  Is the SPM suitable for client and stakeholders' concerns and priorities?
-  Is the SPM suitable for the geographical situation (distributed team)?
-  Is the SPM suitable for the size and complexity of our software?
-  Is the SPM suitable for the type of projects we do?
-  Is the SPM suitable for our software engineering capability?
-  Is the SPM suitable for project risk and quality insurance?

SOFTWARE PROCESS MODELS (SPMs): SDLC REVISITED

- ◎ When a process involves building a software, the process may be referred to as software life cycle
 - Requirements analysis and definition
 - System (architecture) design
 - Program (detailed/procedural) design
 - Writing programs (coding/implementation)
 - Testing: unit, integration, system (many more..)
 - System delivery (deployment)
 - Maintenance

SOFTWARE PROCESS

Activities to develop software



SOFTWARE PROCESS

Software process consists of activities, which are involved in developing software products.

Basic activities are:-

-  *Software Specification*
-  *Software Development*
-  *Software Validation*
-  *Software Evolution*

There are four basic key process activities:

1. Software Specifications –

In this process, detailed description of a software system to be developed with its functional and non-functional requirements.

2. Software Development –

In this process, designing, programming, documenting, testing, and bug fixing is done.

3. Software Validation –

In this process, evaluation software product is done to ensure that the software meets the business requirements as well as the end users needs.

4. Software Evolution –

It is a process of developing software initially, then timely updating it for various reasons.

INTRODUCTION

SDLC or the Software Development Life Cycle is a process that produces software with the highest quality and lowest cost in the shortest time possible.

SDLC provides a well-structured flow of phases that help an organization to quickly produce high-quality software which is well-tested and ready for production use

All SDLC models can be divided into several groups depending on the workflow organization, type of relationships between the teammates, and their communication with the customers.

Plan-driven and agile development

- Plan-driven development
 - A plan-driven approach to software engineering is based around separate development stages with the outputs to be produced at each of these stages planned in advance.
 - Not necessarily waterfall model – plan-driven, incremental development is possible
 - Iteration occurs within activities.
- Agile development
 - Specification, design, implementation and testing are interleaved and the outputs from the development process are decided through a process of negotiation during the software development process.

Difference between Agile and plan-driven

	Agile	Plan-driven
Approach	Iterative and incremental	Sequential and linear
Process	Adaptive and flexible	Rigid and inflexible
Requirements	Prioritizes customer feedback and changing requirements	Emphasizes detailed upfront planning and documentation
Communication	Collaborative and frequent	Formal and occasional
Documentation	Minimalist and informal	Detailed and formal
Risk management	Continuous identification and mitigation	Predefined risk management plan
Team structure	Self-organizing and cross-functional teams	Hierarchical and specialized teams

PLAN DRIVEN MODELS

- ❖ Waterfall Model
- ❖ V-Model
- ❖ Incremental Model
- ❖ Prototyping Model
- ❖ Spiral Model
- ❖ Rapid Application Development (RAD)

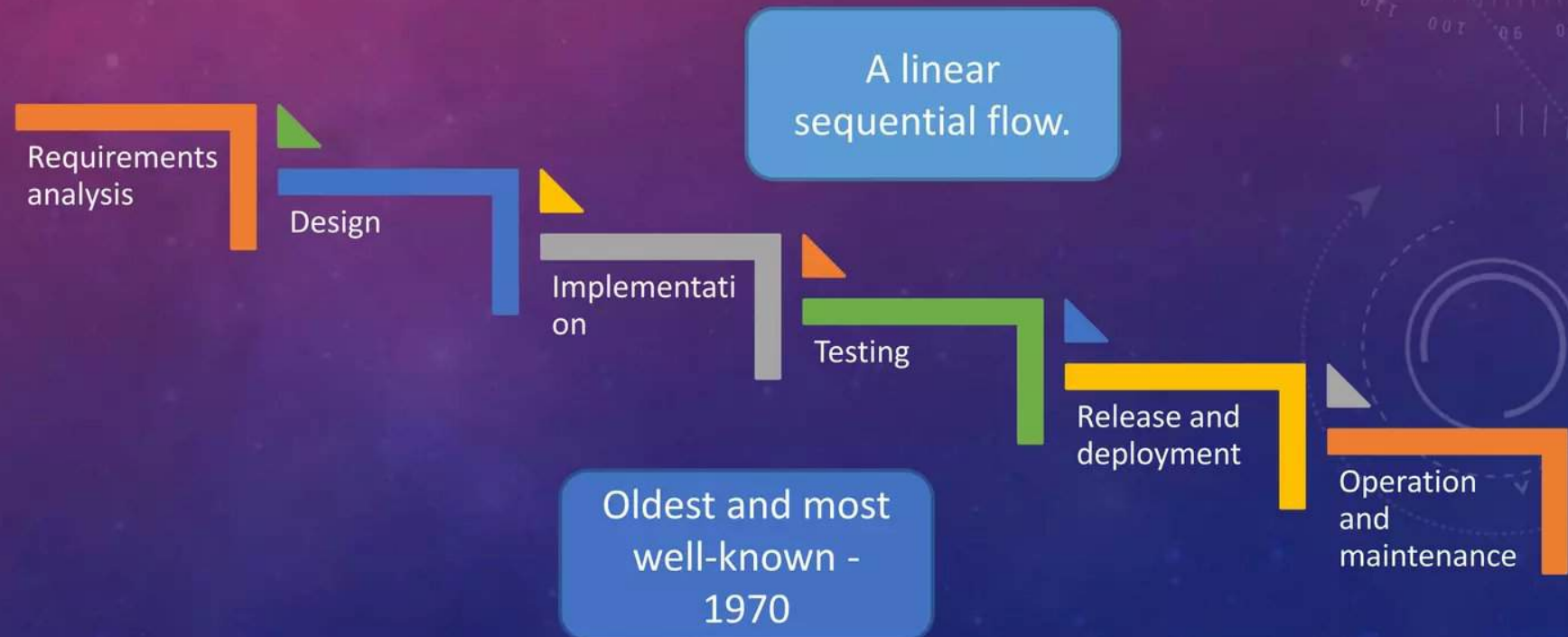
WATERFALL MODEL

WATERFALL

The usage

Projects which not focus on changing requirements, for example, responses for request for proposals (RFPs)

- The waterfall Model is a linear sequential flow.
- In which progress is seen as flowing steadily downwards (like a waterfall) through the phases of software implementation.
- This means that any phase in the development process begins only if the previous phase is complete.
- The waterfall approach does not define the process to go back to the previous phase to handle changes in requirement.
- The waterfall approach is the earliest approach that was used for software development



WATERFALL MODEL

Requirements. Potential requirements, deadlines and guidelines for the project are analyzed and placed into a formal requirements document, also called a functional specification. This stage of development defines and plans the project without mentioning specific processes.

Analysis. The system specifications are analyzed to generate product models and business logic to guide production. This is also when financial and technical resources are audited for feasibility.

Design. A design specification document is created to outline technical design requirements, such as the programming language, hardware, data sources, architecture and services.

Coding and implementation. The source code is developed using the models, logic and requirement specifications designated in the prior phases. Typically, the system is coded in smaller components, or units, before being put together.

Testing. This is when quality assurance, unit, system and beta tests identify issues that must be resolved. This may cause a forced repeat of the coding stage for debugging. If the system passes integration and testing, the waterfall continues forward.

Operation and deployment. The product or application is deemed fully functional and is deployed to a live environment.

Maintenance. Corrective, adaptive and perfective maintenance is carried out indefinitely to improve, update and enhance the product and its functionality. This could include releasing patch updates and new versions.

WHEN CAN WE USED WATERFALL MODEL

fixed requirements

ample resources

an established timeline

well-understood technology

unlikely to require significant changes.

ADVANTAGES OF WATERFALL MODEL

enables large or changing teams to move toward a common goal that's been defined in the requirements stage;

forces structured, disciplined organization;

simplifies understanding, following and arranging tasks;

enables early system design and specification changes to be easily done

clearly defines milestones and deadlines.

DISADVANTAGES OF THE WATERFALL MODEL

Design isn't adaptive; when a flaw is found, the entire process often needs to start over.

Waterfall model delays testing until the end of the development lifecycle.

Waterfall isn't ideal for complex, high-risk ongoing projects.

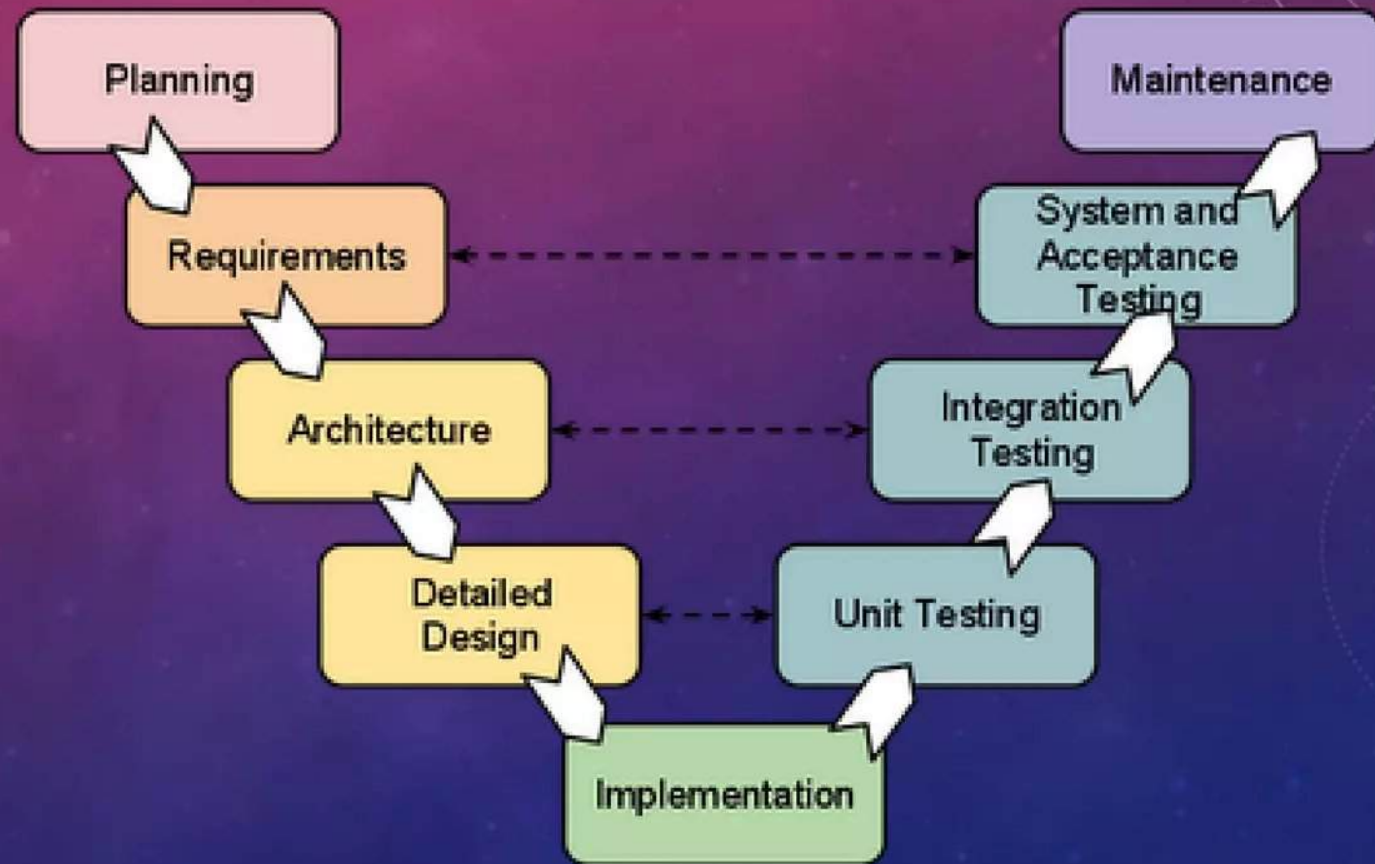
No working product is available until the later stages of the project lifecycle.

V-SHAPED

The usage

- Software requirements clearly defined and known
- Software development technologies and tools is well-known

- It is an extension for waterfall model, Instead of moving down in a linear way, the process steps are bent upwards after the coding phase, to form the typical V shape.
- The major difference between v-shaped model and waterfall model is the early test planning in v-shaped model.



V-MODEL DESIGN

- 1. Requirements Gathering and Analysis:** The first phase of the V-Model is the requirements gathering and analysis phase, where the customer's requirements for the software are gathered and analyzed to determine the scope of the project.
- 2. Design:** In the design phase, the software architecture and design are developed, including the high-level design and detailed design.
- 3. Implementation:** In the implementation phase, the software is built based on the design.
- 4. Testing:** In the testing phase, the software is tested to ensure that it meets the customer's requirements and is of high quality.
- 5. Deployment:** In the deployment phase, the software is deployed and put into use.
- 6. Maintenance:** In the maintenance phase, the software is maintained to ensure that it continues to meet the customer's needs and expectations.

ADVANTAGES AND DISADVANTAGES

Advantages	Disadvantages
Verification and validation of the product in early stages of product development	Assumes that the requirements of a system can be frozen and known from beginning
Easy to use. Stages and activities are well defined	Does not easily handle dynamic changes in requirements
Higher chance of success over the waterfall model due to the development of test plans early on during the life cycle	Inflexible and adjusting scope is difficult and expensive
Each phase has specific deliverables	Costly and required more time
Works well for where requirements are easily understood	Needs very proper and detailed planning
	No continuous customer involvement

INCREMENTAL MODEL

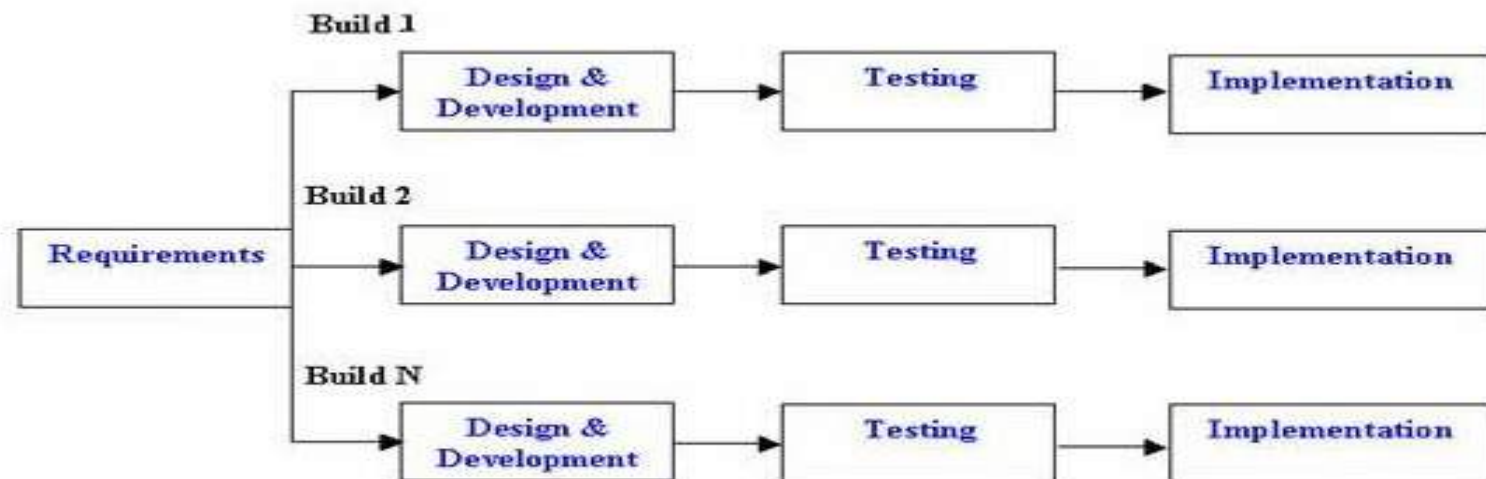
This model requirements are broken down into multiple standalone modules of software development cycle.

In incremental model the whole requirement is divided into various builds.

In **Incremental Model**, each module passes through the requirements, design, implementation and testing phases



Diagram of Incremental model:



Incremental Life Cycle Model

STAGES IN INCREMENTAL MODEL

- 1.Requirement analysis:** In Requirement Analysis At any time, the plan is made just for the next increment and not for any kind of long-term plan. Therefore, it is easier to modify the version as per the needs of the customer.
- 2.Design & Development:** At any time, the plan is made just for the next increment and not for any kind of long-term plan. Therefore, it is easier to modify the version as per the needs of the customer. The Development Team first undertakes to develop core features (these do not need services from other features) of the system. Once the core features are fully developed, then these are refined to increase levels of capabilities by adding new functions in Successive versions. Each incremental version is usually developed using an iterative waterfall model of development.
- 3.Deployment and Testing:** After Requirements gathering and specification, requirements are then split into several different versions starting with version 1, in each successive increment, the next version is constructed and then deployed at the customer site. in development and Testing the product is checked and tested for the actual process of the model.
- 4.Implementation:** In implementation After the last version (version n), it is now deployed at the client site.

ADVANTAGES OF INCREMENTAL MODEL:

Generates working software quickly and early during the software life cycle.

This model is more flexible - less costly to change scope and requirements.

It is easier to test and debug during a smaller iteration.

In this model customer can respond to each built.

Lowers initial delivery cost.

Easier to manage risk because risky pieces are identified and handled during it's iteration.

DISADVANTAGES OF INCREMENTAL MODEL:

Needs good planning and design- a clear and complete definition of the whole system before it can be broken down and built incrementally.

Difficult to Manage Changes- Changes in later increments can impact previous increments, leading to potential rework or redesign.

Increased Overhead-Since the system is being developed incrementally, there can be significant overhead in terms of coordination, scheduling, and planning.

Integration Issues-As increments are developed independently, integrating them later can be difficult and time-consuming

WHEN TO USE THE INCREMENTAL MODEL:

This model can be used when the requirements of the complete system are clearly defined and understood.

Major requirements must be defined; however, some details can evolve with time.

There is a need to get a product to the market early.

A new technology is being used

Resources with the needed skill set are not available

There are some high-risk features and goals.

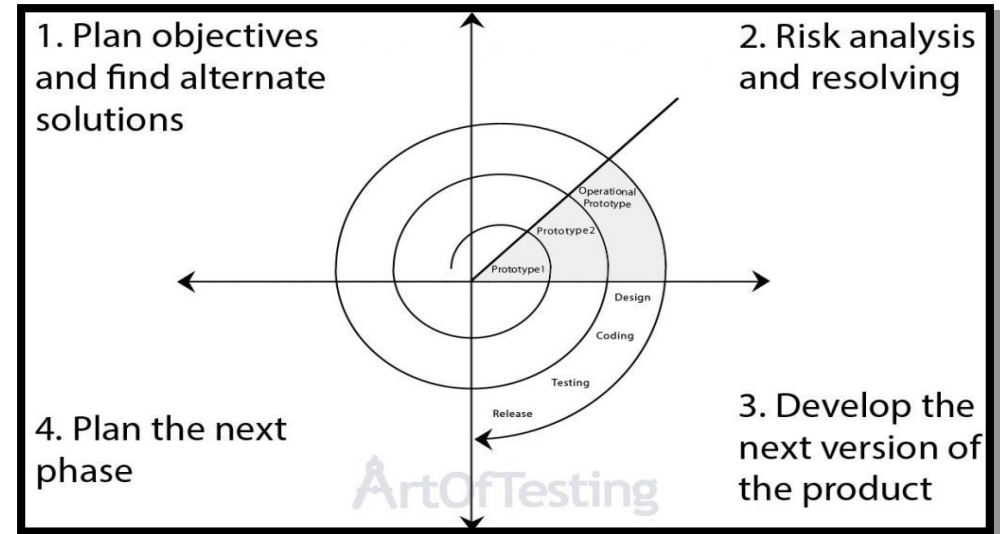
SPIRAL MODEL

The spiral model is also known as the cyclic model.

Using the spiral model, the software is developed in a series of incremental releases.

During later iterations, more and more complete versions of the engineered system are produced.

The spiral model has four phases: Planning, Design, Construct and Evaluation



PHASES IN THE SPIRAL MODEL

- 1.Objectives Defined:** In first phase of the spiral model we clarify what the project aims to achieve, including functional and non-functional requirements.
- 2.Risk Analysis:** In the risk analysis phase, the risks associated with the project are identified and evaluated.
- 3.Engineering:** In the engineering phase, the software is developed based on the requirements gathered in the previous iteration.
- 4.Evaluation:** In the evaluation phase, the software is evaluated to determine if it meets the customer's requirements and if it is of high quality.
- 5.Planning:** The next iteration of the spiral begins with a new planning phase, based on the results of the evaluation.

WHEN TO USE SPIRAL MODEL?

When deliverance is required to be frequent.

When the project is large

When requirements are unclear and complex

When changes may require at any time

Large and high budget projects

ADVANTAGES

- 1.Risk Handling:** The projects with many unknown risks that occur as the development proceeds, in that case, Spiral Model is the best development model to follow due to the risk analysis and risk handling at every phase.
- 2.Good for large projects:** It is recommended to use the Spiral Model in large and complex projects.
- 3.Flexibility in Requirements:** Change requests in the Requirements at a later phase can be incorporated accurately by using this model.
- 4.Customer Satisfaction:** Customers can see the development of the product at the early phase of the software development and thus, they habituated with the system by using it before completion of the total product.
- 5.Iterative and Incremental Approach:** The Spiral Model provides an iterative and incremental approach to software development, allowing for flexibility and adaptability in response to changing requirements or unexpected events.
- 7.Improved Communication:** The Spiral Model provides for regular evaluations and reviews, which can improve communication between the customer and the development team.
- 8.Improved Quality:** The Spiral Model allows for multiple iterations of the software development process, which can result in improved software quality and reliability.

DISADVANTAGES

- 1.Complex:** The Spiral Model is much more complex than other SDLC models.
- 2.Expensive:** Spiral Model is not suitable for small projects as it is expensive.
- 3.Too much dependability on Risk Analysis:** The successful completion of the project is very much dependent on Risk Analysis. Without very highly experienced experts, it is going to be a failure to develop a project using this model.
- 4.Difficulty in time management:** As the number of phases is unknown at the start of the project, time estimation is very difficult.
- 5.Time-Consuming:** The Spiral Model can be time-consuming, as it requires multiple evaluations and reviews.

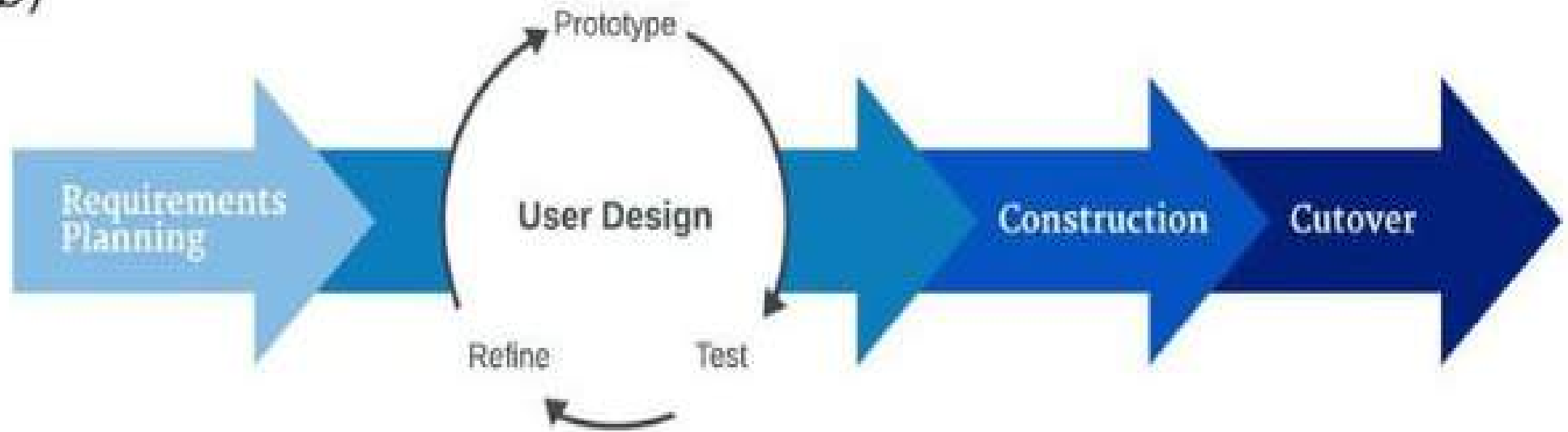
RAD MODEL

The **RAD (Rapid Application Development)** model is based on prototyping and iterative development with no specific planning involved.

The RAD model or Rapid Application Development model is a type of software development methodology that emphasizes quick and iterative release cycles, primarily focusing on delivering working software in shorter timelines.

Phases in the RAD model

Rapid Application Development
(RAD)



RAD PHASES

Phase 1 – Requirements Planning

In the requirements planning phase, the groundwork for the entire project is laid out. Developers, stakeholders, and users collaborate closely to define the project scope, identify key requirements, and establish objectives.

Phase 2 – User Design

The user design phase focuses on creating detailed models and prototypes based on the requirements gathered in the planning phase. In this stage, developers and users work together to design the system's interface, architecture, and functionality through iterative cycles of feedback and refinement. Prototypes are built and continuously adjusted to ensure they meet user needs and expectations, allowing users to interact with early versions of the system and provide real-time feedback.

RAD PHASES

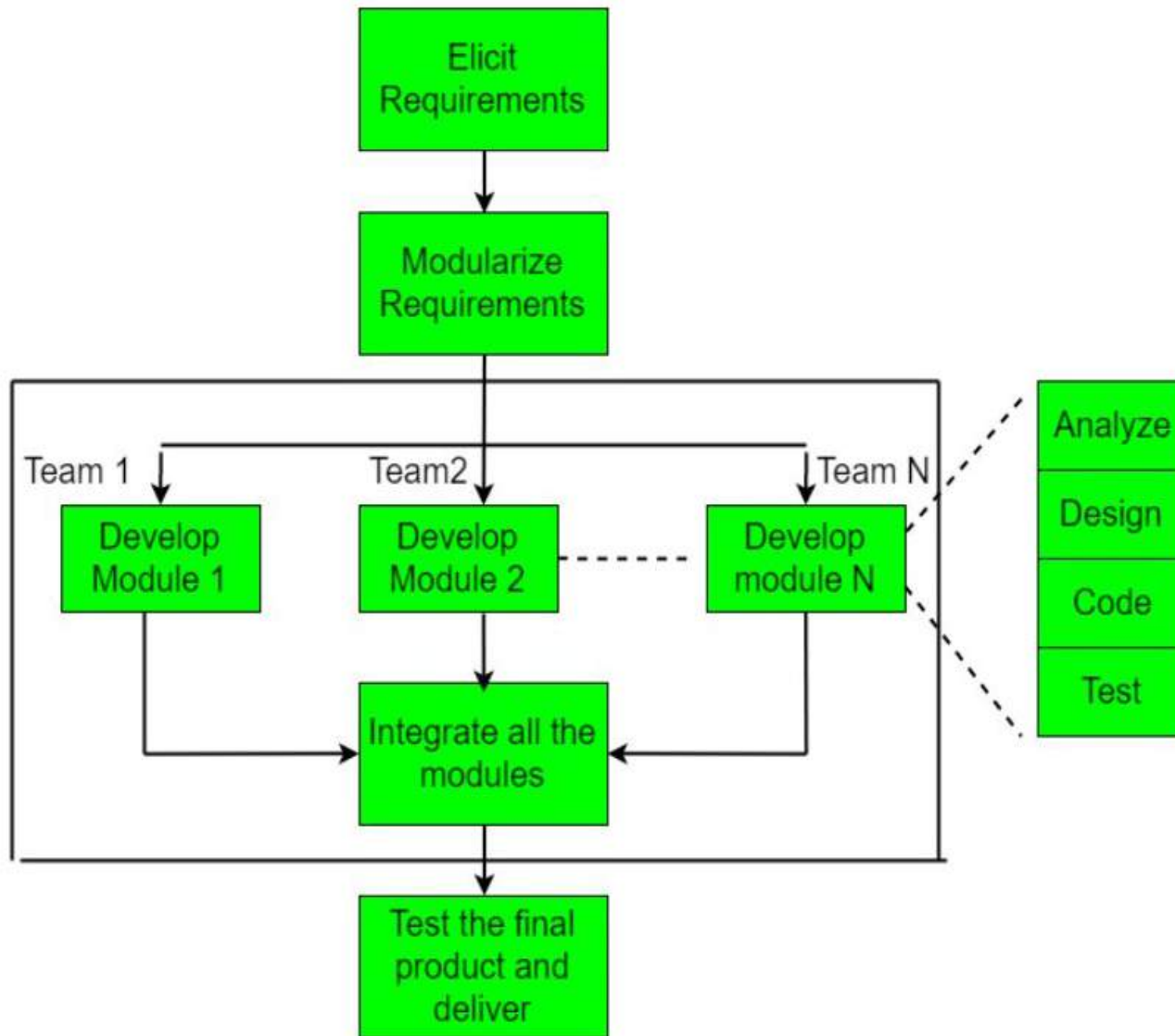
Phase 3 – Rapid Construction

In the rapid construction phase of RAD, the focus shifts to building the actual application. This phase leverages the detailed prototypes and design models created during the user design phase to develop the system's core functionality.

Phase 4 – Cutover/Implementation

The cutover or implementation phase is the final rapid application deployment stage in which the completed system is fully deployed into the production environment. This involves finalizing any last-minute changes, conducting thorough testing, and ensuring the system operates as intended in a live setting.

Multiple teams work on developing the software system using the RAD model parallelly.



The activities carried out in the RAD phases

ADVANTAGES OF RAD MODEL

- ❑ This model is flexible for change- In this model, changes are adoptable.
- ❑ Each phase in RAD brings highest priority functionality to the customer.
- ❑ It reduced development time.
- ❑ It increases the reusability of features.

DISADVANTAGES OF RAD MODEL

- ❖ It required highly skilled designers.
- ❖ All application is not compatible with RAD.
- ❖ For smaller projects, we cannot use the RAD model.
- ❖ On the high technical risk, it's not suitable.
- ❖ Required high user involvement.

WHEN TO USE RAD MODEL?

- ❖ When the system should need to create the project that modularizes in a short span time (2-3 months).
- ❖ When the requirements are well-known.
- ❖ When the technical risk is limited.
- ❖ When there's a necessity to make a system, which modularized in 2-3 months of period.
- ❖ It should be used only if the budget allows the use of automatic code generating tools.

PROTOTYPING MODEL

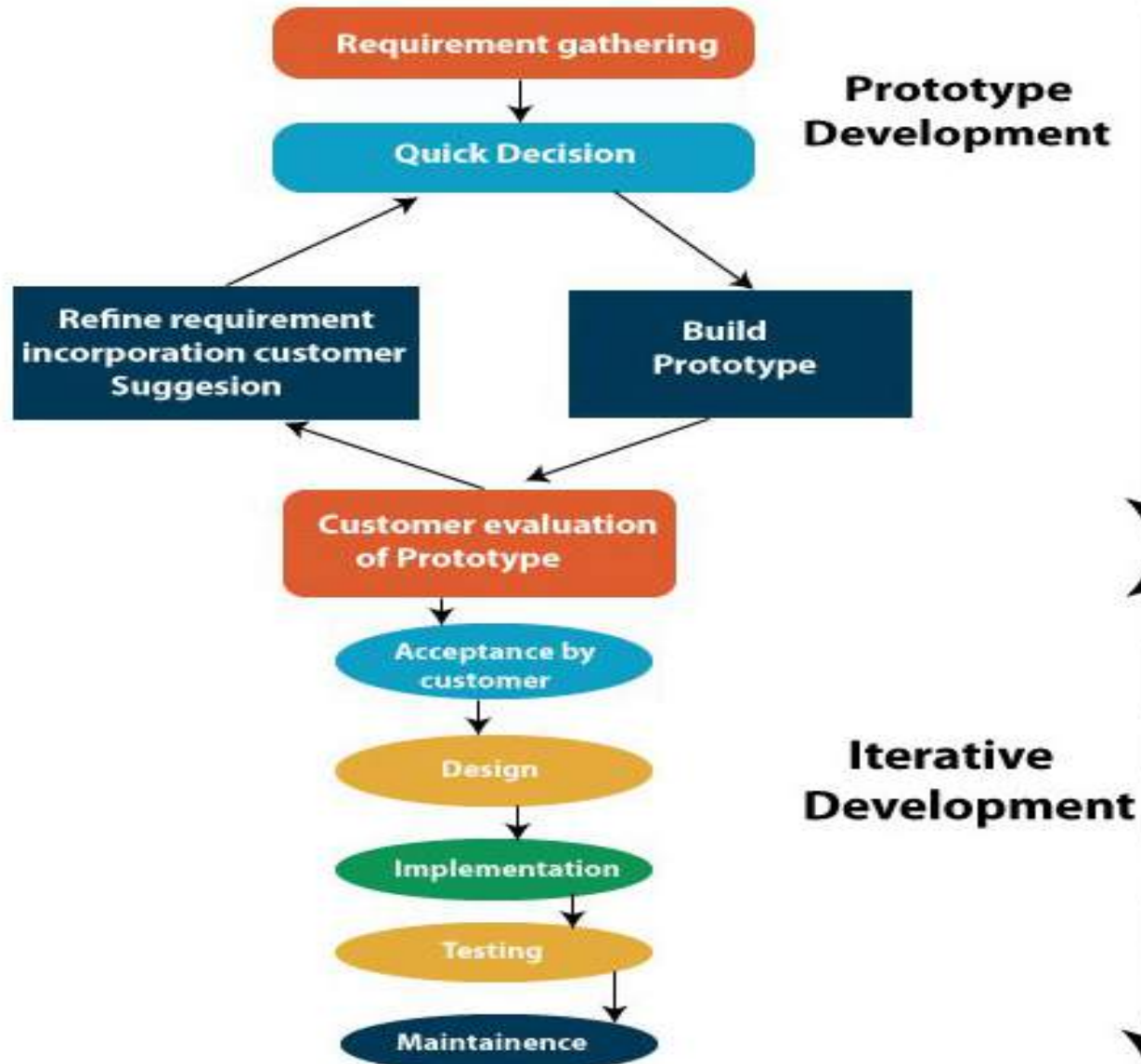
The prototype model requires that before carrying out the development of actual software, a working prototype of the system should be built.

A prototype is a toy implementation of the system.

This model works best in scenarios where not all of the project requirements are known in detail ahead of time.

It is an iterative, trial-and-error process that takes place between the developers and the users.

Fig: Prototype Model



Steps of Prototype Model:

1. Requirement Gathering and Analyst
2. Quick Decision
3. Build a Prototype
4. Assessment or User Evaluation
5. Prototype Refinement
6. Engineer Product

PROTOTYPE PHASES

Step 1: Requirement Gathering and Analysis: This is the initial step in designing a prototype model. In this phase, users are asked about what they expect or what they want from the system.

Step 2: Quick Design: This is the second step in the Prototyping Model. This model covers the basic design of the requirement through which a quick overview can be easily described.

Step 3: Build a Prototype: This step helps in building an actual prototype from the knowledge gained from prototype design.

Step 4: Initial User Evaluation: This step describes the preliminary testing where the investigation of the performance model occurs, as the customer will tell the strengths and weaknesses of the design, which was sent to the developer.

PROTOTYPE PHASES

Step 5: Refining Prototype: If any feedback is given by the user, then improving the client's response to feedback and suggestions, the final system is approved.

Step 6: Implement Product and Maintain: This is the final step in the phase of the Prototyping Model where the final system is tested and distributed to production, here the program is run regularly to prevent failures.

ADVANTAGE OF PROTOTYPE MODEL

- ❑ New requirements can be easily accommodated as there is scope for refinement.
- ❑ Missing functionalities can be easily figured out.
- ❑ Errors can be detected much earlier thereby saving a lot of effort and cost, besides enhancing the quality of the software.
- ❑ The developed prototype can be reused by the developer for more complicated projects in the future.
- ❑ Flexibility in design.
- ❑ The customers get to see the partial product early in the life cycle. This ensures a greater level of customer satisfaction and comfort.

DISADVANTAGES

- There may be too much variation in requirements each time the prototype is evaluated by the customer.
- Poor Documentation due to continuously changing customer requirements.
- It is very difficult for the developers to accommodate all the changes demanded by the customer.
- There is uncertainty in determining the number of iterations that would be required before the prototype is finally accepted by the customer.
- After seeing an early prototype, the customers sometimes demand the actual product to be delivered soon.
- Developers in a hurry to build prototypes may end up with sub-optimal solutions.
- The customer might lose interest in the product if he/she is not satisfied with the initial prototype.

WHEN TO USE PROTOTYPE MODEL

This model is used when the customers do not know the exact project requirements beforehand.

When the developers are new to the domain.

In this model, a prototype of the end product is first developed, tested and refined as per customer feedback repeatedly till a final acceptable prototype is achieved which forms the basis for developing the final product.

AGILE METHODOLOGY

AGILE METHODOLOGY

Agile is an approach to software development that seeks the continuous delivery of working software created in rapid iterations.

Advantages of Agile Model

- Customer satisfaction, they can check software release and revert feedback
- In agile process mostly meeting arranged before product release.
- More interaction maintained within developing and testing team in this agile process.
- Customers can change or add requirements at any stage.
- It concentrates on every process with expert team members.

Disadvantage Agile Model

- Documentation could get lengthy.
- It is not useful for small projects.
- It required expert persons to take important decisions in meeting.
- Cost more as compare to waterfall or interactive model.

PRINCIPLES OF AGILE DEVELOPMENT

1. Customer satisfaction by rapid delivery of useful software
2. Welcome changing requirements, even late in development
3. Working software is delivered frequently (weeks rather than months)
4. Working software is the principal measure of progress
5. Sustainable development, able to maintain a constant pace
6. Close, daily co-operation between business people and developers
7. Face-to-face conversation is the best form of communication (co-location)
8. Projects are built around motivated individuals, who should be trusted
9. Continuous attention to technical excellence and good design
10. Simplicity- The art of maximizing the amount of work not done - is essential
11. Self-organizing teams
12. Regular adaptation to changing circumstances

AGILE METHODOLOGY

- ❖ Extreme programming (XP)
- ❖ SCRUM
- ❖ FDD

EXTREME PROGRAMMING (XP)

- ❖ XP is the most specific Agile framework for software development practices. It aims to not only produce high-quality software but to make the entire process easier on the development team itself.
- ❖ XP values communication, feedback, simplicity, courage, and respect.
- ❖ XP is ideal for software teams that prioritize clean code, rigorous testing, and active customer collaboration throughout the development process.

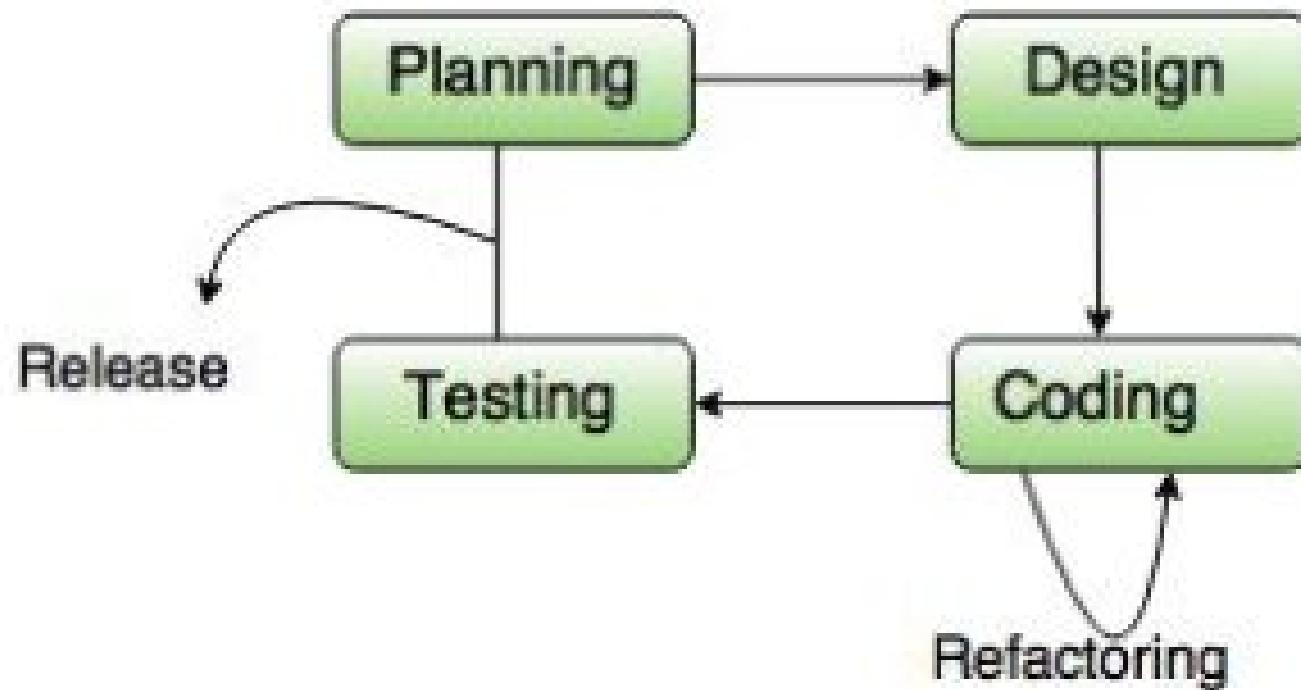


Fig. - The Extreme Programming Process

XP PHASES

1. Planning

Starting the process off right, the Extreme Programming development life cycle begins with the essential first step of Planning. In this stage, the team talks with the customer to understand the desired product, gathering user stories to form the basis of the project's goals. These stories are estimated and prioritized, then transformed into tasks. This step is crucial in setting the stage for a successful project, ensuring the customer's vision of the software is achieved.

2. Design

In this stage, developers take the user stories and design the code architecture to meet the customer's expectations. This includes deciding on the programming language, environment, libraries, and frameworks to be used in software development.

3. Coding

Here, the development team gets to work on the actual coding of the software. The code is written iteratively, meaning that the code is written in small chunks and tested regularly. This ensures that the code is high quality and that any bugs can be addressed quickly.

4. Testing

Testing is a vital part of the Extreme Programming methodology. Developers write automated tests to ensure that the code is working as expected. This helps identify any potential bugs before the software is released.

IT'S BEST APPLIED WHEN

There are constantly changing requirements

Teams have tight deadlines

Stakeholders want to reduce risk under deadlines

Teams can automate unit and functional tests

Extreme Programming (XP)

Advantages

- Simple codes, easy to improve
- The whole cycle of XP is visible
- Constant testing makes the process more agile
- Uplifting the talent of teams

Disadvantages

- More attention to code and less to design
- Not working in remote working teams
- If issues are not documented rightly, possibility for repeating the same issues

SCRUM

Scrum is a management framework that teams use to self-organise tasks and work towards a common goal.

It is a framework within which people can address complex adaptive problems while the productivity and creativity of delivering products are at the highest possible value.

Scrum is a management framework that teams use to self-organise and work towards a common goal.

SCRUM PHASES

Initiate - This phase includes the processes related to initiation of a project: Create Project Vision, Identify Scrum Master and Stakeholder(s), Form Scrum Team, Develop Epic(s), Create Prioritized Product Backlog, and Conduct Release Planning.

Plan and Estimate - This phase consists of processes related to planning and estimating tasks, which include Create User Stories, Estimate User Stories, Commit User Stories, Create Tasks, Estimate Tasks, and Update Sprint Backlog.

Implement - This phase is related to the execution of the tasks and activities to create a project's product. These activities include creating the various deliverables, conducting Daily Standup Meetings, and refining (i.e., reviewing, fine-tuning, and regularly updating) the Product Backlog at regular intervals.

Review and Retrospect - This phase is concerned with reviewing the deliverables and the work that has been done and determining ways to improve the practices and methods used to do project work.

Release - This phase emphasizes on delivering the Accepted Deliverables to the customer and identifying, documenting, and internalizing the lessons learned during the project.

THE MAIN ELEMENTS OF SCRUM

Roles	Artifacts	Meetings
• Product owner • Scrum master • Team members	• The product backlog • The sprint backlog • The product increment • Other artifacts	• Planning • Daily Scrum • Review • Retrospective

ROLES

Product Owner

The Product Owner focuses on ensuring the development team delivers the most value to the business. They understand and prioritize the changing needs of end users and customers

Scrum leader

Scrum leaders are the champions for Scrum within their teams. They are accountable for the Scrum Team's effectiveness. They coach teams, Product Owners, and the business to improve its Scrum processes and optimize delivery.

Scrum development team

The Scrum Team consists of testers, designers, UX specialists, Ops engineers, and developers. Team members have different skill sets and cross-train each other, so no one person becomes a bottleneck in delivering work.

ARTIFACTS

Product Backlog

The Product Backlog is a dynamic list of features, requirements, enhancements, and fixes that must be completed for project success.

Sprint Backlog

The Sprint Backlog is the list of items to be completed by the development team in the current Sprint cycle. Before each Sprint, the team chooses which items it will work on from the Product Backlog.

Product Increment

The Increment is a step towards a goal or vision. It is the usable end product from a Sprint. Teams can adopt different methods to define and demonstrate their Sprint Goals

MEETINGS

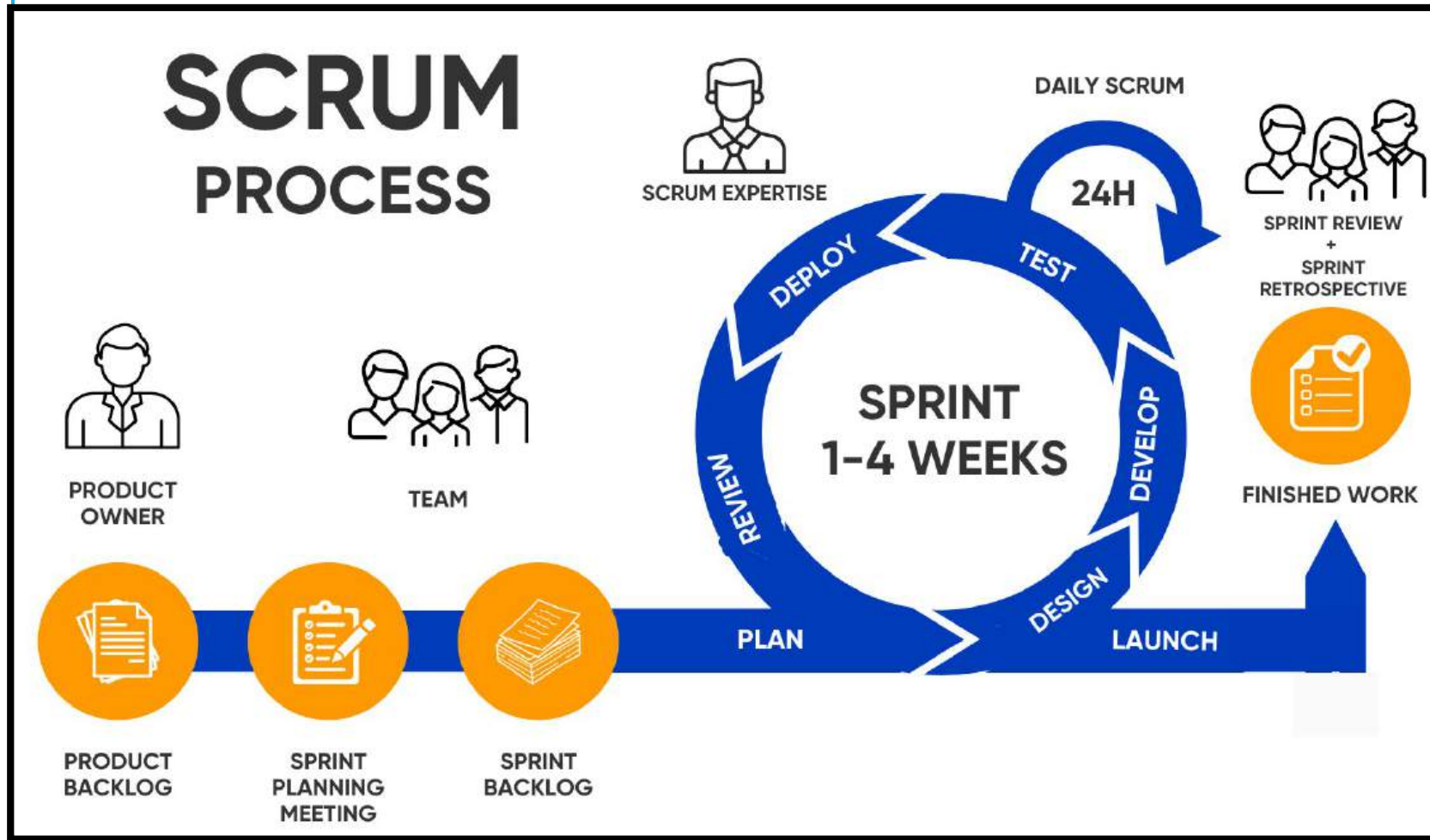
Sprint Planning: Sprint Planning refers to the sprint by laying out the work which needs to be performed for the sprint. This plan is created by the collaborative work of the scrum team. The meetings of scrum planning occur at the starting of the sprint. During the sprint planning the scrum team commits the tasks which needs to be completed in the upcoming sprint.

Daily Scrum: The main purpose or objective of daily scrum is to inspect the progress of the sprint goal and to adapt the sprint backlog to adjust the upcoming planned work.

Sprint Review: The main objective of the sprint review is to inspect the outcome of the sprint and determine the future adaptation. The teams shows the results of them to the stakeholders and get the required feedback from them.

Sprint Retrospective: The main objective of sprint retrospective is to help the team to build the habit of continuous process of improvement. During the sprint retrospective the members of the team first suggest the topic for the discussion by writing their thought on reflection cards.

Backlog Refinement: It refers to the act of breaking down and further defining the product backlog items into the smaller items. It is the ongoing activity to add the details such as order, size and description.



The entire workflow of agile software development occurs during various Scrum events.

SCRUM ADVANTAGES

increased productivity in complex and innovative projects

Efficient utilization of time and resources

Allows for faster completion of marketing projects

Allows to build high-quality products

High employee satisfaction

High client satisfaction

SCRUM DISADVANTAGES:

Projects often lack focus and clarity at certain stages

Requires skilled personnel/robust training

May sometimes require organizational transformation

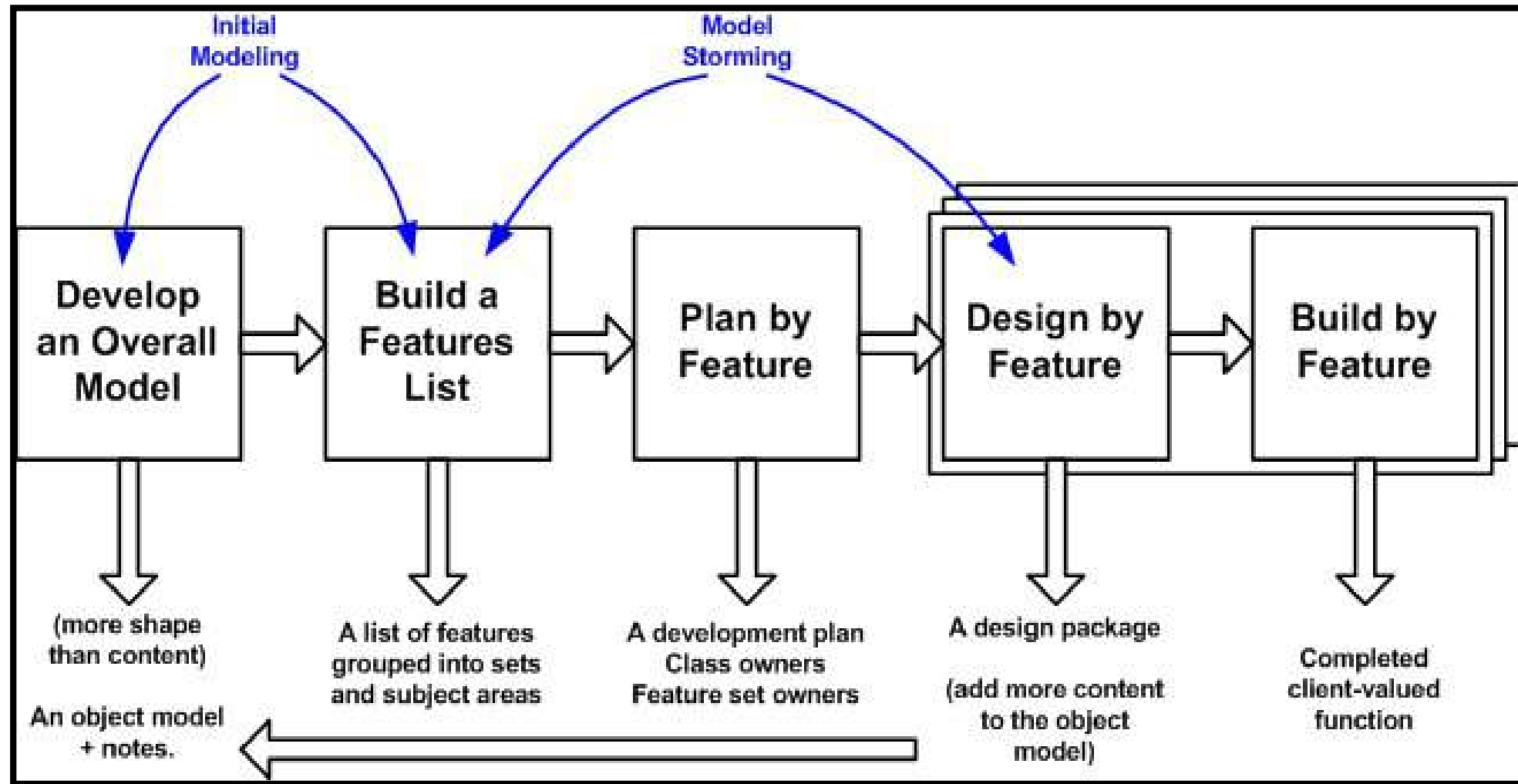
Too many steps needed to scale large projects

Daily meetings may be frustrating

FEATURE DRIVEN DEVELOPMENT

Feature Driven Development (FDD) is a client-centric Agile methodology tool that focuses on incremental development and status reporting across all levels.

FDD is a scalable model that delivers features on a much shorter timeframe than other Agile frameworks. For instance, instead of a typical 4-week iteration cycle in Scrum, FDD aims to deliver features every 2-10 days. This makes it easier for teams to track and address errors, adapt to client requests, and get new team members up-to-date quickly.



FDD PHASES

1: Develop the overall model

Here, an FDD team will determine the project scope. Multiple models will be proposed and merged to create one overall model.

2: Build the features list

Next, the team members will outline the customer-focused features to be developed. They will be small functions that can be completed in a short period of time. An example could be to create an automatic reminder for subscription renewal.

3: Plan by feature

The team will assess the individual features in the list and arrange them in the appropriate order. Then, the features will be assigned to team members.

4: Design by feature

At this stage, the team's chief programmer will choose which features to develop within a two-week period. A design package will be created for each feature, and team members will conduct a review before building commences.

5: Build by feature

Developers work to build the code for the aforementioned features. This code will be tested before the final version is created.

Feature-driven development (FDD)

Advantages

- ▲ Clear picture of project scope
- ▲ User-centric approach
- ▲ Works good with long projects and large teams
- ▲ Decreased need for meetings

Disadvantages

- ▲ Doesn't work good with small projects
- ▲ Dependence on lead developers
- ▲ No written documentation to the client



THANK YOU



Chapter 3 Software Project Management





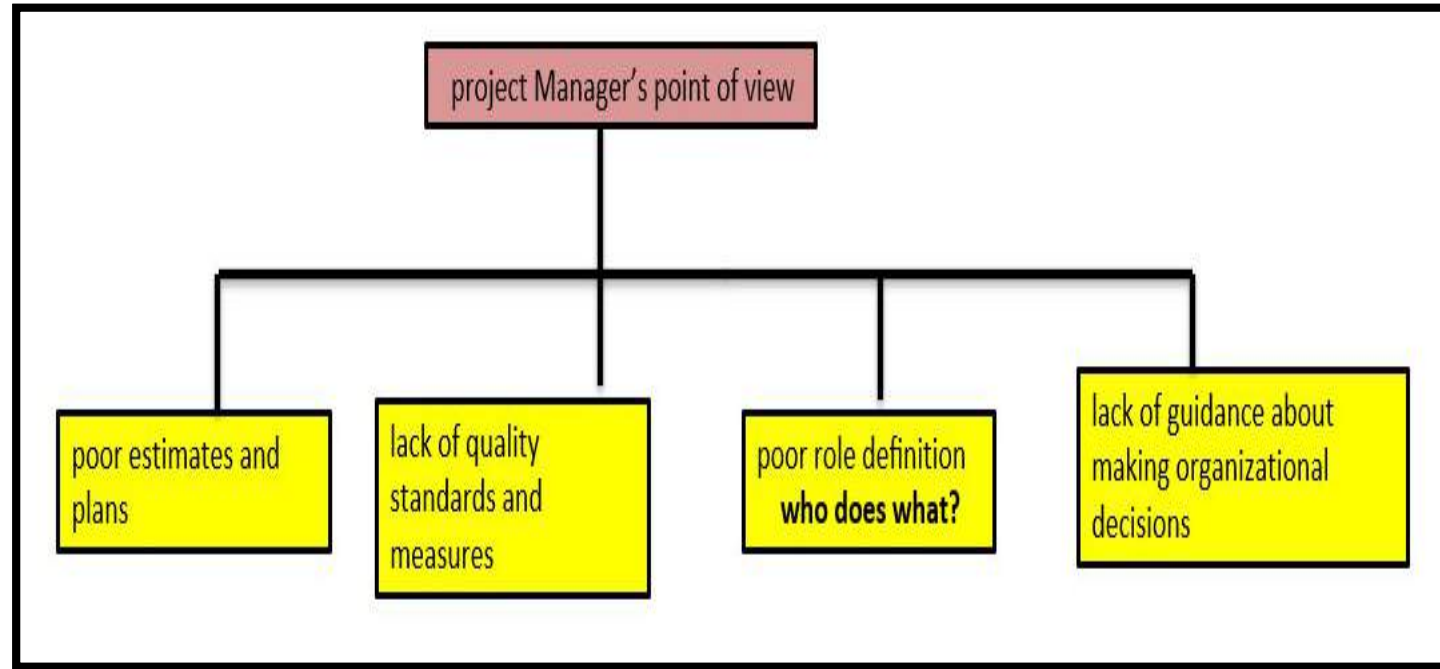
Lesson Objectives

- Explain the principal **tasks of software project management**
- Evaluate the **importance** of project management
- Discuss the **key issues of team working** – team composition, cohesiveness & communication

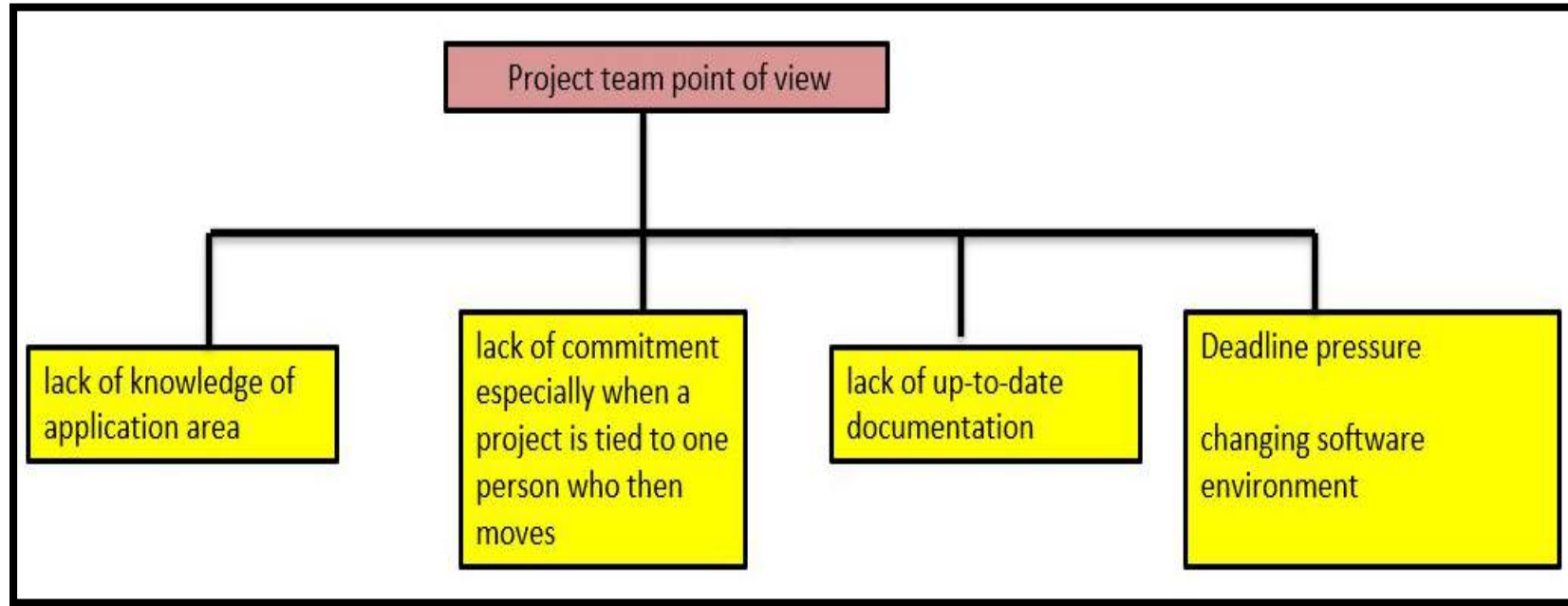
WHAT IS SOFTWARE PROJECT MANAGEMENT?

- ❖ Software project management is an art and discipline of planning and supervising software projects. It is a sub-discipline of software project management in which software projects planned, implemented, monitored and controlled.
- ❖ It is a procedure of managing, allocating and timing resources to develop computer software that fulfills requirements.
- ❖ In software Project Management, the client and the developers need to know the length, period and cost of the project.

Problems with software projects



Problems with software projects



WHY IS SOFTWARE PROJECT MANAGEMENT IMPORTANT?

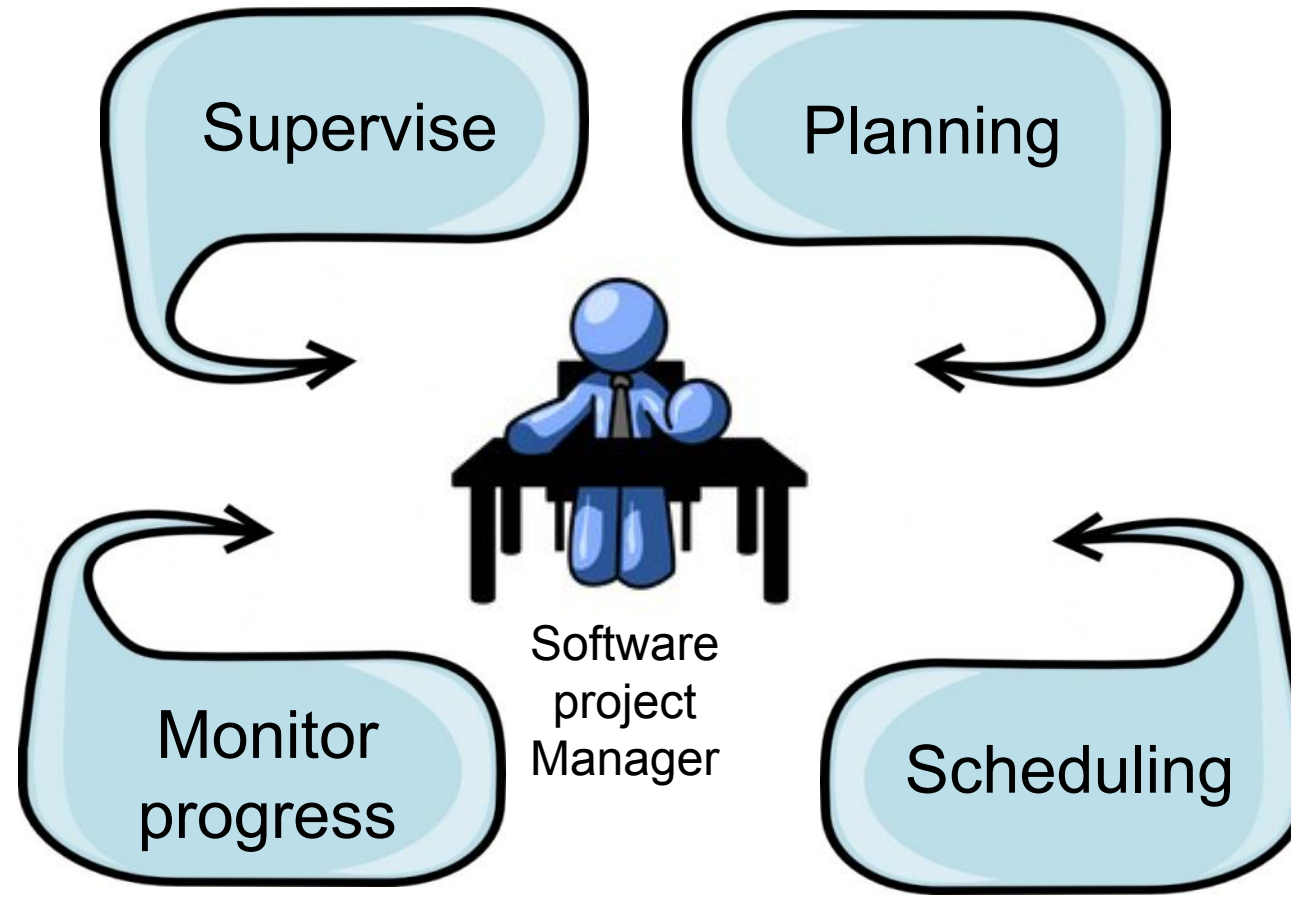
- Strategic Alignment
- Leadership
- Clear Focus & Objectives
- Realistic Project Planning
- Quality Control
- Risk Management
- Managing and Learning from Success and Failure



TASKS OF SOFTWARE PROJECT MANAGER



What are the main responsibilities of an software project manager?



What are the main responsibilities of an IT project manager?

- Software project managers are responsible for **planning** and **scheduling** project development.
- They **supervise** the work to ensure that it is carried out to the required **standards**.
- They **monitor progress** to check that the development is on time and within budget.





Project Management Activities

1. Proposal writing
2. Project planning and scheduling
3. Project costing
4. Project monitoring and reviews
5. Personnel selection and evaluation
6. Report writing and presentation





1. Proposal Writing

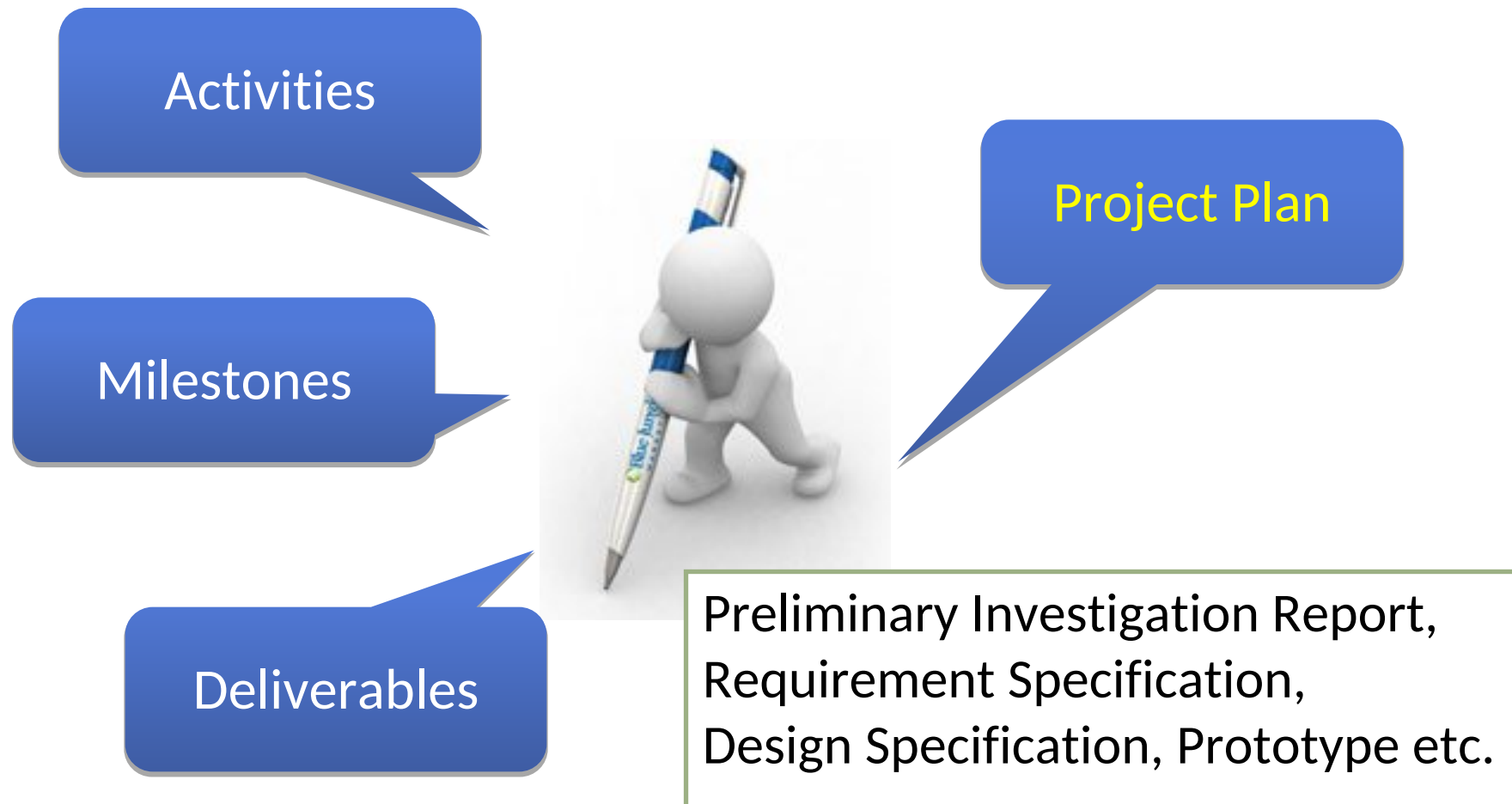
- **Definition:** Proposal writing involves creating a formal document that outlines a suggested plan or project.
- **Purpose:** Proposals are used to pitch ideas, secure funding, or gain approval for a project. They serve as a roadmap for the project's execution.
- **Key Elements:** Clear problem statement, proposed solution, budget breakdown, timeline and etc.



2. Project Planning and Scheduling

- **Definition:** Project planning involves defining project goals, identifying tasks, estimating resources needed, and creating a timeline for completion.
- **Purpose:** Project planning ensures that all stakeholders understand the project's scope, objectives, and deadlines. Scheduling involves allocating resources and setting milestones.
- **Tools:** Gantt charts, project management software, and critical path analysis are commonly used for project planning and scheduling.

2. Project Planning



2. Project Planning



Project Scheduling

Work Breakdown

Activity
Dependencies

Staff Allocations

2. Project Planning

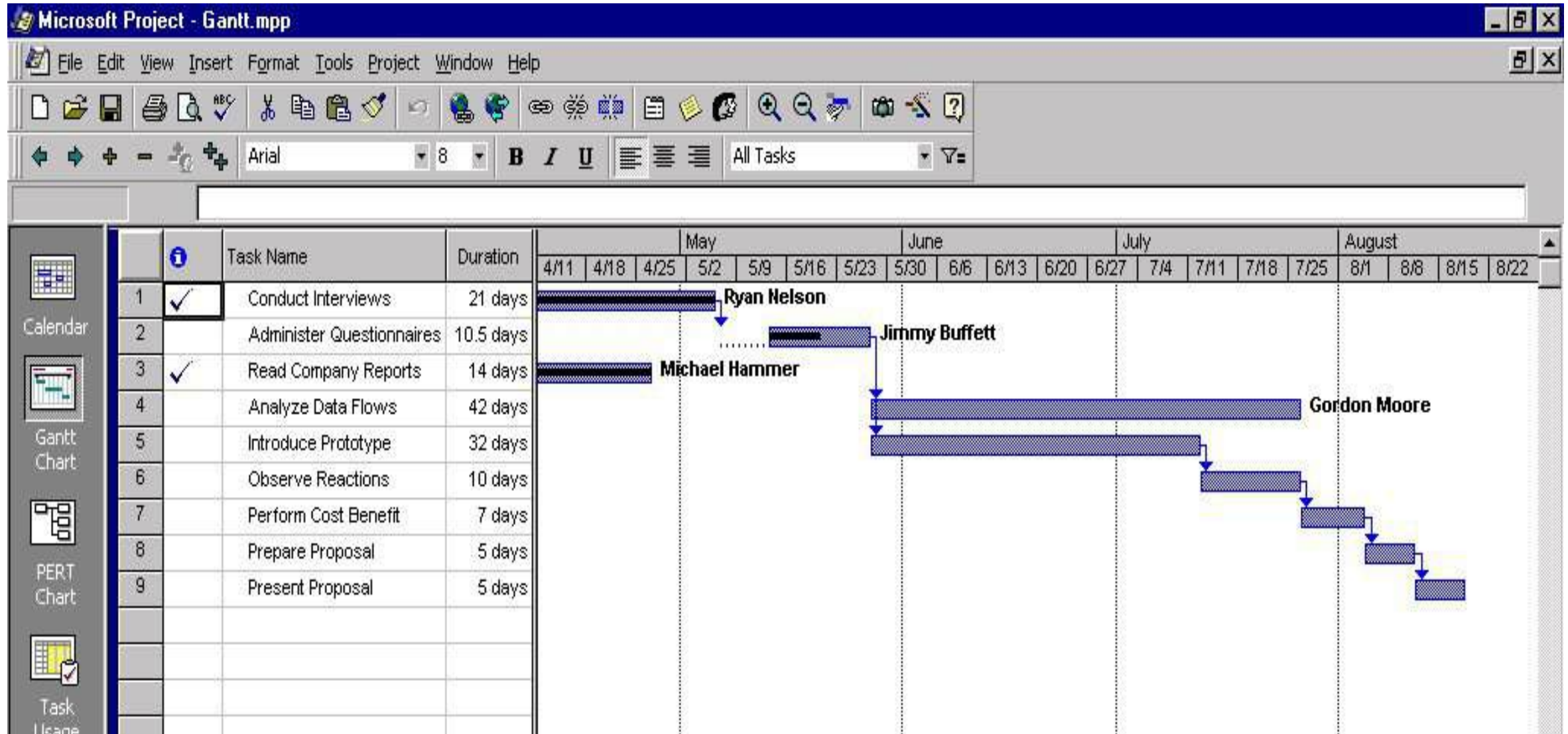


Gantt Chart

PERT Chart

Project Scheduling

2. Project Planning – Gantt Chart





3. PROJECT COSTING





3.Project Costing

- **Definition:** Project costing involves estimating and tracking the expenses associated with a project.
- **Purpose:** Costing helps in budget allocation, control, and ensure the project remains financially viable.
- **Components:** Direct costs (e.g., materials, staffs) and indirect costs (e.g., overhead, administrative expenses) are considered when calculating project costs.

3. Project Costing



Estimate Resources:

- ☐ hardware
- ☐ software
- ☐ people

Key Factors That Affect the Software Development Cost

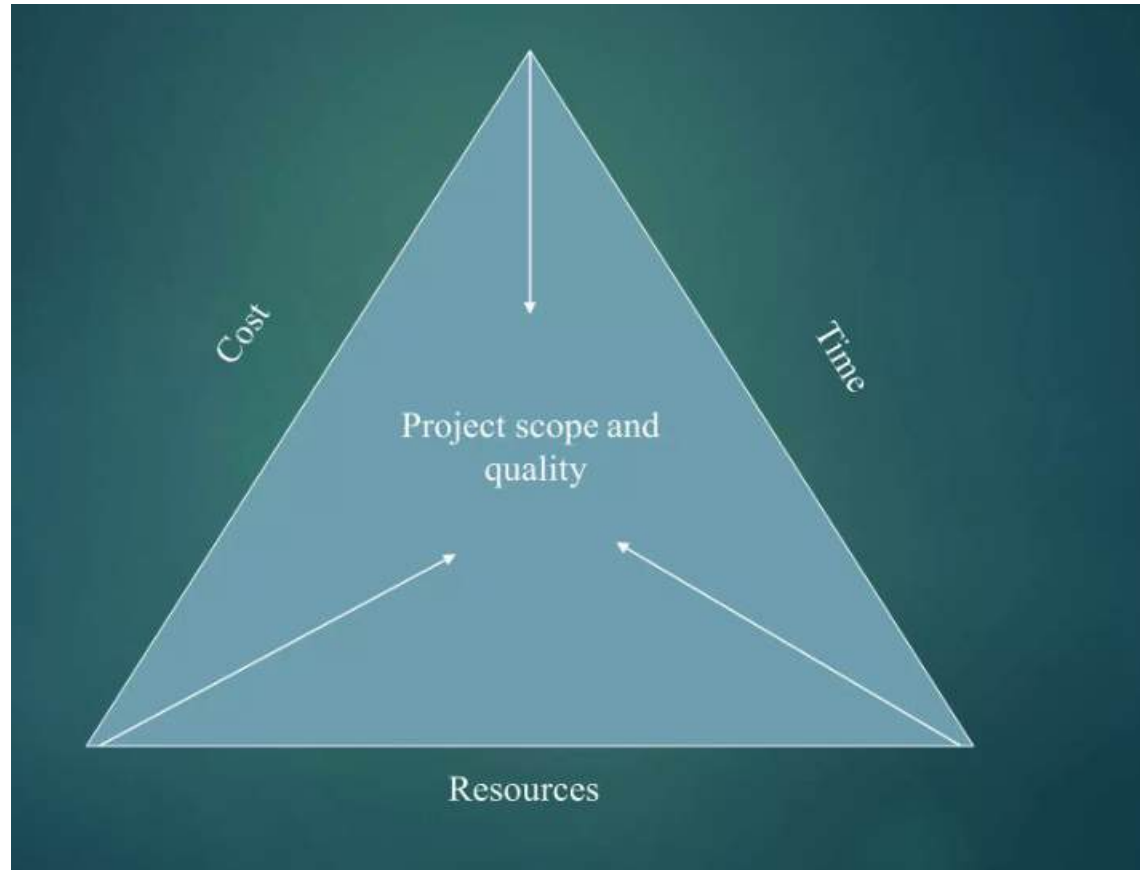




4. Project Monitoring and Reviews

- **Definition:** Project monitoring involves tracking project progress, identifying deviations from the plan, and taking corrective actions. Reviews involve evaluating project performance against predefined metrics.
- **Purpose:** Monitoring and reviews help in ensuring that the project stays on track, meeting objectives, and making adjustments as needed.
- **Tools:** Key performance indicators (KPIs), progress reports, and regular team meetings are utilized for project monitoring and reviews.

4. Project Monitoring



Triple Constraints:

- ❖ Time
- ❖ Cost
- ❖ Quality

Actual project progress

VS

Planned project progress



Scope

- This is the work that needs to be done. It documents the features and functions that will be included in this iteration of the product.
- The scope documentation should make it clear to everybody what will or won't be included in the final product.
- Project manager might consider using a work breakdown structure (WBS) chart to break down the scope into actionable tasks.



Time

- How much time it will take to complete the tasks in the scope. To determine time constraint, set a timeline for each task in each iteration.
 - Based on the timelines, set a deadline for the final release of the product.
 - Make sure that the timelines are realistic, especially if time is the least flexible constraint in the project.
- A PERT chart can help you to map out the time required for each task.



Cost

- Budget/Cost: The resources required to complete the project, including financial, labor, materials, etc. Review previous, similar projects to help you estimate how much your budget will cost. Look at the following costs to help you determine the overall budget:
 - Resources: How much will it cost to pay people working on the project? Base the costs on the estimated hours it takes to complete the tasks.
 - Materials: If you are in manufacturing, you have to consider the costs of raw materials and the supply chain.
 - Equipment: Will this project require new hardware or software? Consider the costs of upgrading vs. working with existing equipment.

4. Project Monitoring



Corrective actions

E.g. add more staff, reassign tasks, using OT, encourage staff etc

Formal vs informal reviews



4. Project Monitoring

- **Corrective actions** need to be adopted depending on the situation.
 - E.g. add more staff, reassign tasks, using OT, encourage staff etc.
- May include **informal monitoring** besides **formal project reviews**



5. Personnel Selection and Evaluation

- **Selection:** Involves identifying and recruiting individuals with the necessary skills and qualifications for project roles.
- **Evaluation:** Involves assessing the performance of team members based on predetermined criteria, providing feedback, and making decisions about promotions or role changes.
- **Importance:** Personnel selection and evaluation are crucial for ensuring that the project team is capable of achieving project goals effectively.

5. Personal Selection & Management

Application Domain

Programming
Language Exp.

Attitude

Communication Ability

Adaptability



Factors on staff selection

5. Personal Selection & Management

- Ideally **skilled** staff with appropriate **experience** will be available to work on the project
- Project manager should select the **right candidates** to join their project team.
- Factors on staff selection include: application domain and prog. language experience, attitude, communication ability and adaptability.

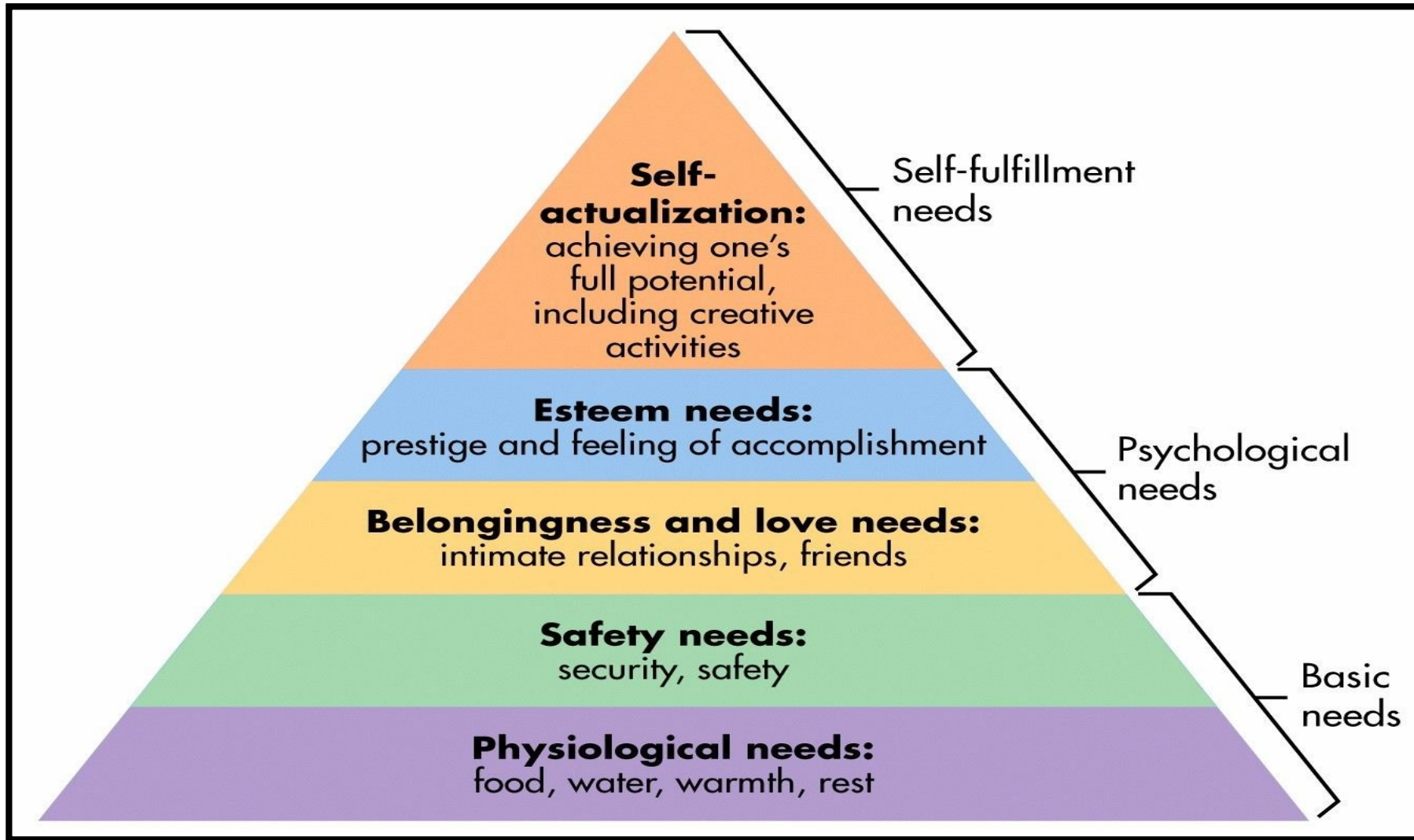




Factors on staff selection

- **Application Domain**-This refers to the area or field in which the project or job operates — for example, healthcare, finance, education, manufacturing, etc (web, mobile apps)
- **Programming Language Experience** -This is the staff's familiarity and skill level with the programming languages required for the project (e.g., Python, Java, C++, JavaScript).
- **Attitude**-Refers to the employee's mindset, motivation, and approach toward work, teamwork, and challenges.
- **Communication ability**-This is the capacity to convey ideas clearly, both verbally and in writing, and to listen and understand others effectively.
- **Adaptability**-The ability to adjust to new tools, methods, technologies, or working environments quickly.

5. Personal Selection & Management: Maslow's Motivation Model





Maslow's Hierarchy

- Maslow's Hierarchy of Needs is a motivational theory in psychology proposed by Abraham Maslow in 1943.
- The theory suggests that people are motivated to fulfil certain needs in a hierarchical order



Maslow's Hierarchy

Physiological Needs: Ensure that basic physiological needs of employees are met, such as providing a safe and comfortable working environment, access to necessary resources, and fair compensation. Address issues like workload management, breaks, and physical well-being.

Safety Needs: Establish a sense of job security and stability within the project team. Clearly communicate roles and responsibilities, provide opportunities for training and skill development, and ensure a safe and supportive work environment.



Maslow's Hierarchy

- Belongingness and Love Needs: Foster a sense of camaraderie and teamwork within the project team. Encourage collaboration, open communication, and mutual support among team members. Organize team-building activities and create opportunities for social interaction.
- Esteem Needs: Recognize and appreciate the contributions of team members. Provide regular feedback, opportunities for advancement and growth, and avenues for employees to showcase their skills and expertise within the project. Encourage a culture of respect and recognition.



Maslow's Hierarchy

- Self-Actualization Needs: Support employees in pursuing personal and professional development goals within the project. Offer opportunities for creativity, innovation, and autonomy in decision-making.

5. Personal Selection & Management



Factors that influence Group Working

5. Personal Selection & Management



Factors that influence Group Working

Self-oriented

Task-oriented

Interaction-oriented

i. Group Composition

Group composition refers to the characteristics, attributes, and diversity of individuals who form a group. The composition of a group can significantly impact its dynamics, performance, and outcomes.

Is there the right balance skills, experience and personalities in the team?

5. Personal Selection & Management

ii. Group Cohesiveness



Factors that influence Group Working

Group cohesiveness refers to the extent to which members of a group are connected, unified, and committed to achieving common goals. A cohesive group is characterized by strong interpersonal bonds, mutual trust, and a shared sense of identity

*Does the group think of itself as a **team** rather than as a collection of individuals who are working together?*

How to promote group cohesiveness?

5. Personal Selection & Management



Factors that influence Group Working

ii. Group Cohesiveness

How to promote group cohesiveness?

1. Establish group identity
2. Team building activities
3. Members are treated as responsible & trustworthy

5. Personal Selection & Management



Factors that influence Group Working

iii. Group Communication

Group communication plays a vital role in the functioning and success of teams and organizations. Effective communication within a group involves the exchange of information, ideas, feedback, and emotions among members.

*Do the members of the group
communicate effectively with
each other?*

5. Personal Selection & Management



Factors that influence Group Working

Did you assign the right job to the right people?

iv. Group Organization

Effective group organization involves defining goals, assigning roles, establishing communication channels, scheduling meetings, encouraging collaboration, setting deadlines, providing support, and evaluating progress to maximize efficiency and productivity.

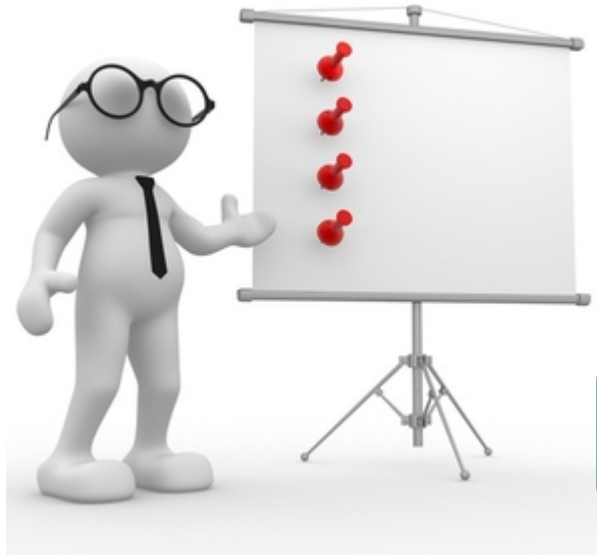
*Is the team organized in such a way that everyone feels **valued** and is **satisfied** with their role in the group?*



6. Report writing and presentation

- In project management, effective report writing and presentation skills are crucial for communicating project progress, findings, recommendations, and key insights to stakeholders

6. Report Writing and Presentation



Reporting on the project



Document

Presentation

Can good project management guarantee project success?

- Good management cannot guarantee project success. However, bad management usually results in project failure. The software delivered late, costs more than originally estimated and fails to meet its requirements.





“Coming together is a beginning, Keeping together is progress & Working together is success.”

Henry Ford

- Founder - Ford Motor Company



Exercise

Create a Gantt Chart based on the following information.

Activity	Duration (month)	Predecessor	Overlap
A	$\frac{1}{2}$	None	None
B	1	A	None
C	2	B	$\frac{1}{4}$
D	1	B	None
E	$\frac{1}{2}$	C, D	None

		July				August				Sept				Oct		
Task ID	Duration															
A	1/2															
B	1															
C	2															
D	1															
E	1/2															

Assume the project starts in July

Chapter 4

Software Requirements



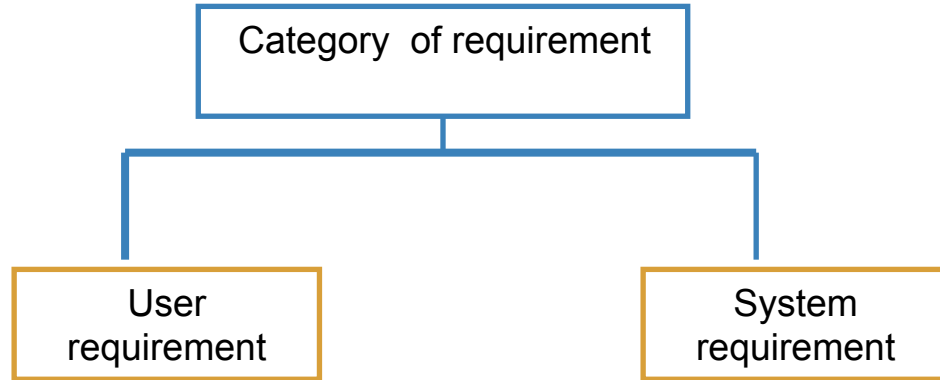


Lesson Objectives

- Differentiate between **User requirements** and **System requirements**
- Differentiate between **Functional** and **Non-functional Requirement**
- Explain why **natural language** specification is not desirable
- Understand how requirements may be organised in a **Software Requirements Specification (SRS)**



Requirements for a software system set out **what the system/software should do** and define **constraints** on its operation and implementation.





User Requirements

- User requirements are high-level abstract requirements. They are intended for use by people involved in **using** and **procuring** the system.
- User requirements are **statements** in **natural language**, with easy-to-understand **diagrams** and **tables**, of **what** services the system is expected to provide and the **constraints** under which it must operate.



User Requirements

High level abstract
requirement

For user & people
who procure system



Statements with
natural language with
diagrams & tables



System Requirements

- System requirements are also used by software designers as the starting point for the system design, so they are sometimes called “functional specifications”.
- They may be the basis of a contract between the system developer and customer. Therefore should be complete and consistent specification of the whole system.



System Requirements

- System requirements are **more precise** and **detail descriptions** of the system's functions, services and operational constraints.
- They may be written in **structured form of natural language** supported by **system models** and **tables**



System Requirements



Precious & Detail
Description

Extended Version of User
Requirement

Structured form of
natural language
supported by system
models & tables

During requirement analysis



System Requirements

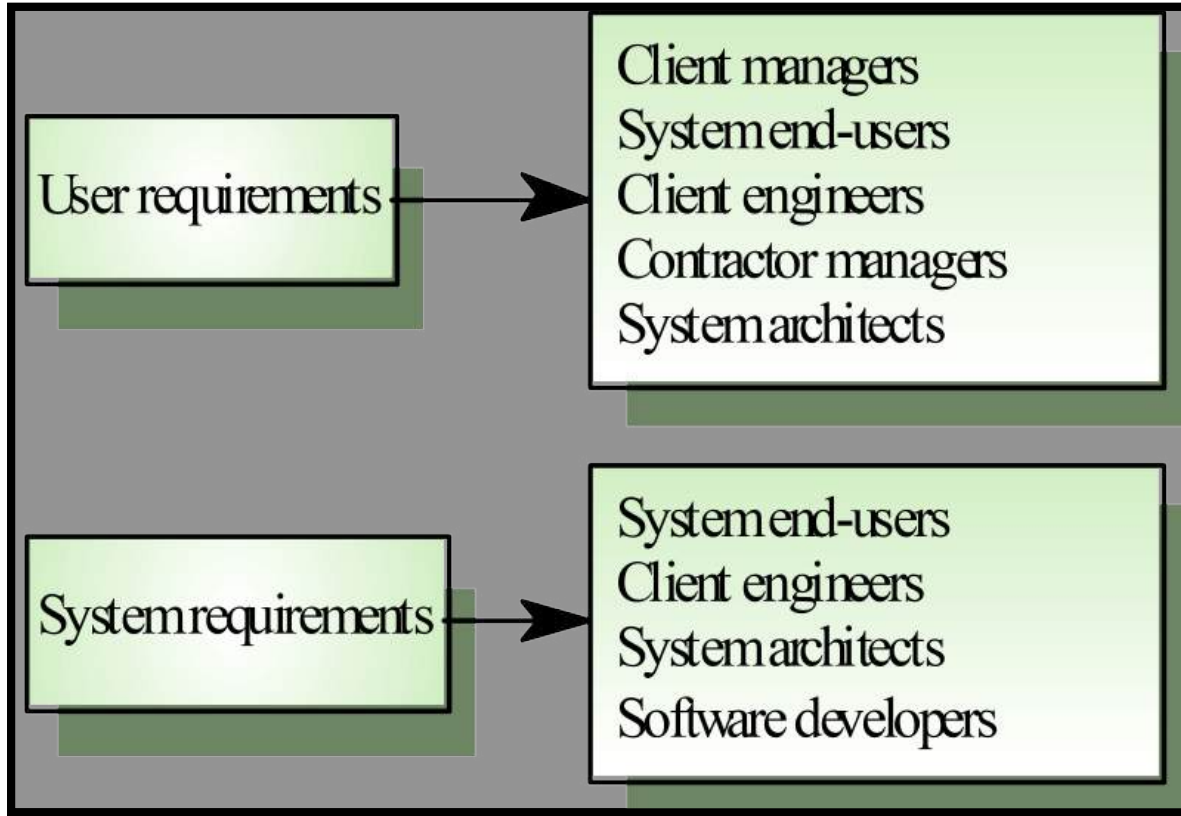


Starting point for
design

a.k.a. Functional Specifications

Contract between
developer & customer

Requirements readers





Issues in system requirement: Problems with Natural Language



Natural Language Problems

- 3 types of major problem with user requirements written in natural language: -
 - i. **Lack of clarity** : difficult to use language in a precise & unambiguous way without making the document wordy & difficult to read
 - i. **Requirements confusion** : functional, non-functional requirements, system goals, design info may not be clearly distinguished
 - i. **Requirements amalgamation** : several requirements expressed as a single requirement



Natural Language Problems



Lack of clarity

Ambiguity?

Precise?

Wordy

Difficult to read



Natural Language Problems

Requirement
confusion

Lack of clarity



Functional vs.
non-functional?



System goals?



Design info





Natural Language Problems

Requirement
confusion

Lack of clarity



Amalgamation



1. Lack of clarity

Example of scenario:

- ⦿ Requirement: "The system must generate an error message if the user inputs an invalid email address."
- **Alternative: "Display an error notification when an incorrect email is entered."**
- **Variation: "Provide feedback to users when they enter an invalid email address."**



2. Requirements confusion

- The team receives a requirements document that contains a mix of technical specifications, user interface design elements, and user stories without clear segregation. The language used in the document does not effectively partition the requirements based on functionality, user roles, or system components.



3. Requirements amalgamation

- **Scenario:**
- **Context:** A software development team is working on a project to create a new e-commerce platform. The team consists of developers, designers, and project managers who collaborate on various aspects of the platform's development.
- **Issue:** During a meeting, the developers and designers discuss the checkout process of the e-commerce platform. The developers refer to the customer's final step before purchase as the "checkout page," while the designers call it the "payment screen."



Alternatives to NL specification

- **Design description language** – Design Description Languages (DDLs) are formal languages used to describe the architecture, behavior, and structure of a system or software design.
- **Graphical notations** – Graphical notations are visual symbols, diagrams, or representations used to convey information in a graphical format. DFD, flowchart, use-case diagrams, sequence diagrams, activity diagrams etc.
- **Mathematical specifications** – Mathematical specifications are formal descriptions of the requirements and properties of a system, process, or algorithm using mathematical notation. mathematical concepts such as finite-state machines or sets. Formal specification like Z-specification



Exercise – Past Year Question

Online Purchase Concert Ticket

User Requirement

1. The system must provide a facility which allows the online users to check on concert details, seat availability and purchase tickets



Exercise

Online Purchase Concert Ticket

System Requirement

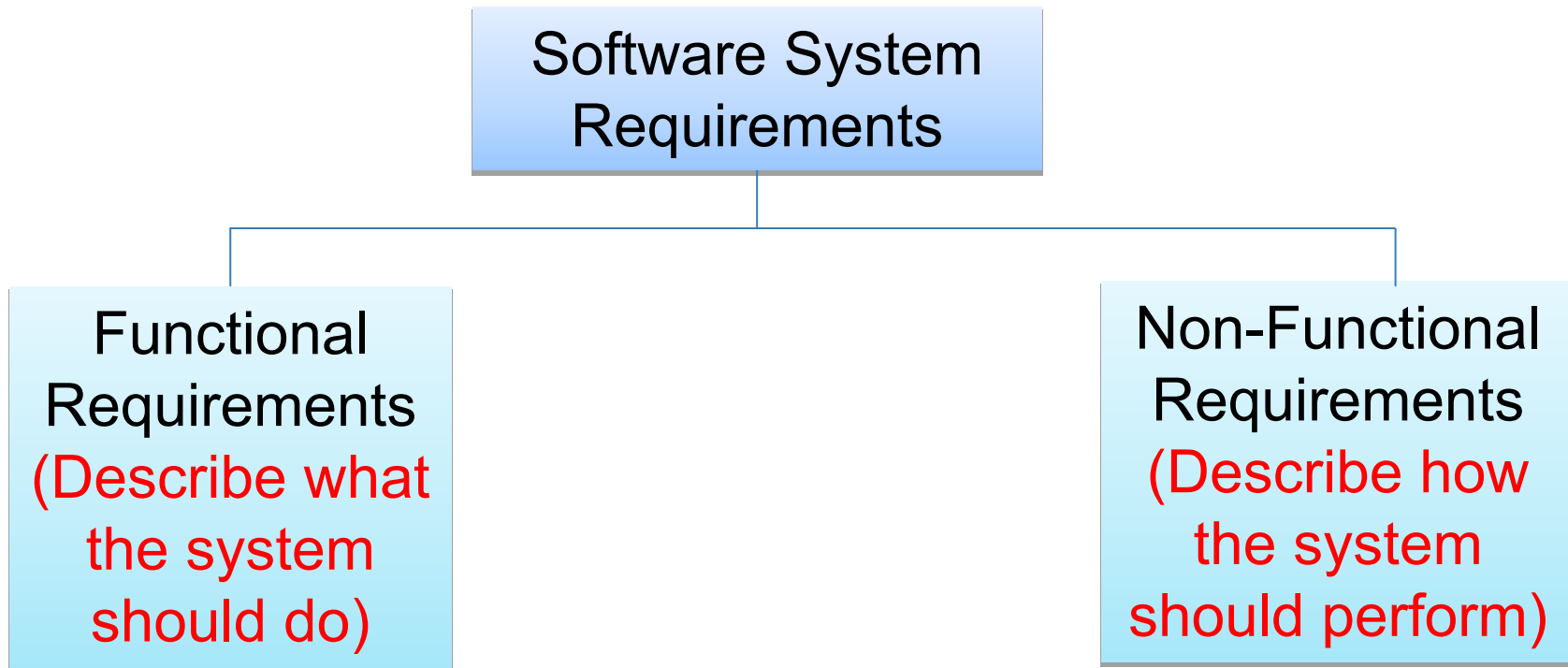
- 1.1 The system allows the online users to view concert's details
- 1.2 The users can check the seat availability for the concert they are interested in
- 1.3 The users can select the seating position available and make a purchase for the ticket(s)
- 1.4 The system will calculate the charges once the user confirmed the transaction
- 1.5 The system will also check on the payment details such credit card number and etc.
- 1.6 If invalid payment details, the system will show message
- 1.7 If valid, the system will proceed transaction and update the seat(s) to not available
- 1.8 The system will generate on-line receipt



Functional and Non-Functional Requirements



Functional & Non-Functional Requirements





Functional Requirements

- Functional requirements define the specific features and operations a system must perform to meet business and user needs.
- These are statement of **services** the system should provide, how the system should **react** to particular inputs and how the system should **behave** in particular situation. In some cases, the functional requirements may also explicitly state **what** the system should not do.



Functional Requirements

How System Behave
to situation

Services

What system
should not do

How System
React to input





Examples

- **User Authentication:** The system must allow users to log in using a username and password.
- **Search Functionality:** The software should enable users to search for products by name or category.
- **Report Generation:** The system should be able to generate sales reports for a specified date range.



Functional Requirements

- Examples of functional requirements of a university library system called LIBSYS:

1.1 The user shall be able to search either all of the initial set of databases or select a subset from it.

1.2 The system shall provide appropriate viewers for the user to read documents in the document store.



Functional Requirements

● HR System – Employee Self Service Sub-system:

1.0 Basic Personal Tasks

- 1.1 Employees can **perform** routine HR administration tasks such as changing name, address, home / work telephone numbers, emergency contacts, next of kin, marriage, births or adoption details
- 1.2 Employees can **upload** common information to existing databases e.g. telephone directories and organization charts
- 1.3 Employees can **view** employees own employment history i.e. promotions, dates, appraisal reviews, training records
- 1.4 Employees can **view** conditions of employment, HR policies, procedures, information packs, benefits details



Non-functional requirements

- Non-functional requirements describe how the software performs a task rather than what it should do. They define the quality attributes, performance criteria, and constraints.
- Non-functional requirements may be more critical than functional requirements. If these are not met, the system is useless
- **Examples:**
 - **Performance:** The system should process 1,000 transactions per second.
 - **Usability:** The software should be easy to use and have a user-friendly interface.
 - **Reliability:** The system must have 99.9% uptime.
 - **Security:** Data must be encrypted during transmission and storage.



Non-Functional Requirements



Product
Requirements

External
Requirements

Organizational
Requirements

Non-functional classifications

● Product requirements

Requirements which specify that the delivered product must behave in a particular way e.g. execution speed, reliability, etc.

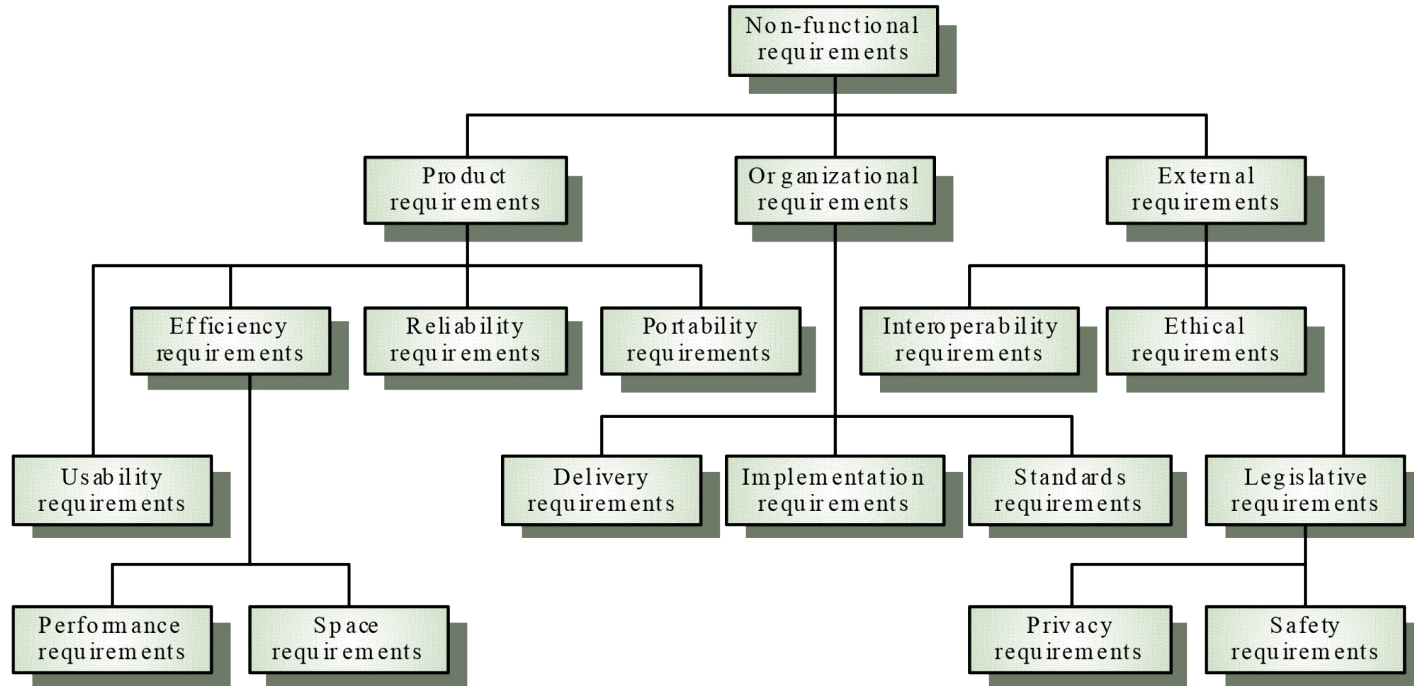
● Organisational requirements

Requirements which are a consequence of organisational policies and procedures e.g. process standards used, implementation requirements, etc.

● External requirements

Requirements which arise from factors which are external to the system and its development process e.g. interoperability requirements, legislative requirements, etc.

Non-functional requirement classifications





Non Functional Requirements

● Example

Product requirement

8.1 The **user interface** for LIBSYS shall be implemented as simple **HTML** without frames or Java applets.

Organizational requirement

9.3.2 The system development process and deliverable documents shall **conform** to the process and deliverables defined in **XYZ-ST95**.



Non Functional Requirements

● Example

External requirement

*10.6 The system **shall not disclose any personal information** about system users apart from their name and library reference number to library staff who use the system.*



Software Requirements Document

- The software requirements document (sometimes known as **SRS - software requirements specification**) is the official statement of what is required of the system developer.
- It should include both:
 - the user requirements for a system
 - a detailed specification of the system requirements



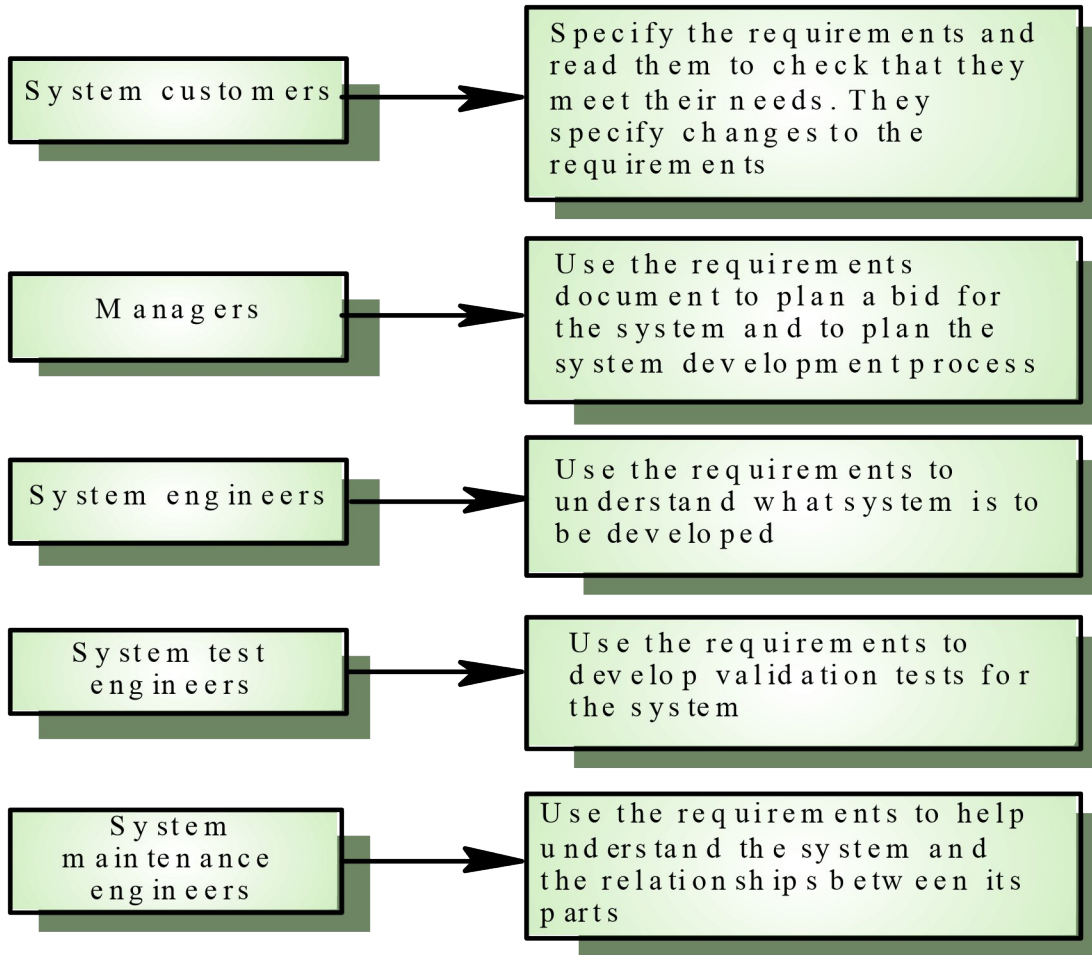
Software Requirements Document

- This document is not a design document. It should set out **what** the system should do **without specifying how** it should be done.
- In principle, the requirements set out in this document ought to be complete and consistent.
- All system functions should be specified and requirements should not conflict.



Notations for System Requirements Specification (SRS)

- **Structured natural language** – software requirements document is structured into sections such as Introduction, Scope, Functional Requirements, Non-Functional Requirements, and User Interface Design.
- Each section contains specific information organized in a clear and structured manner to effectively communicate the requirements



Software Requirements Document user's



Software Requirements Document

The requirements document has a diverse set of users:

i) System customers

Specify the requirements and read them to check that they meet their needs. They specify changes to the requirements

ii) (Project) Managers

Use the requirements document to plan a bid for the system and to plan the system development process



Software Requirements Document

The requirements document has a diverse set of users:

iii) System engineers

Use the requirements to understand what system is to be developed

iv) System test engineers

Use the requirements to develop validation tests for the system

v) System maintenance engineers

Use the requirements to help understand the system and the relationships between its parts



Software Requirements Document

The structure of SRS:

Preface

Introduction

- ✓ Purpose
- ✓ Scope
- ✓ Overview
- ✓ Business Context





Software Requirements Document

The structure of SRS:

Preface

Introduction

Glossary

✓ Definition of terms





Software Requirements Document

The structure of SRS:

Preface

Introduction

Glossary

User requirements definition

✓ General





Software Requirements Document

The structure of SRS:

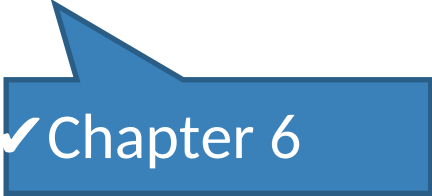
Preface

Introduction

Glossary

User requirements definition

System architecture



✓ Chapter 6



Software Requirements Document

The structure of SRS:

Preface

Introduction

Glossary

User requirements definition

System architecture

System requirements specification

✓ Functional
✓ Non-Functional





Software Requirements Document

The structure of SRS:

Preface

Introduction

Glossary

User requirements definition

System architecture

System requirements specification

System models

✓ Chapter 6





Software Requirements Document

The structure of SRS:

Preface

Introduction

Glossary

User requirements definition

System architecture

System requirements specification

System models

System evolution

✓ Version (Chp 10)





Software Requirements Document

The structure of SRS:

Preface

Introduction

Glossary

User requirements definition

System architecture

System requirements specification

System models

System evolution

Appendices

✓ References (supporting documents)





Software Requirements Document

The structure of SRS:

Preface

Introduction

Glossary

User requirements definition

System architecture

System requirements specification

System models

System evolution

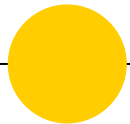
Appendices

Index

Index

Abstract, 6, 17, 27
add, 9, 11, 19, 20, 21
Add, 8, 9, 19
Article, 1, 5, 6, 7, 8, 9, 10, 11, 12, 13,
14, 15, 16, 17, 18, 19, 20, 21, 22, 23,
24, 25, 26, 27, 28
Article Manager, 5, 8, 9, 10, 11, 12, 13,
14, 15, 16, 19, 20, 21, 22, 23, 24, 25,
28
Author, 1, 4, 5, 6, 7, 8, 9, 13, 14, 16, 17,
19, 20, 22, 23, 25, 26, 27
Category, 5, 14, 17, 18, 20, 21, 23, 26,
27
Database, 2, 9, 11, 14, 15, 16, 17, 18, 19,
20, 21, 22, 24, 25, 26, 27
Editor, 1, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13,
14, 15, 16, 17, 19, 20, 21, 22, 23, 24,
25, 28
Field, 17, 19, 20
Form, 1, 6, 9, 10, 11, 12, 14, 19, 20, 21,
23, 24, 27
Grid, 9, 11, 12, 19, 20, 21
Historical Society, 1, 5, 9, 11, 16, 17, 19,
20, 26
Online Journal, 4, 5, 6, 7, 15, 16, 17, 18,
24, 27, 28
Reader, 4, 5, 6, 16, 17, 18
Review, 1, 7, 11, 12, 18, 21, 23, 26, 27
Reviewer, 1, 4, 5, 6, 7, 9, 11, 16, 17, 19,
20, 21, 22, 23, 26, 27
Security, 27, 28
Status, 11, 12, 13, 14, 17, 21, 22, 23, 27
update, 9, 11, 20, 21
Update, 8, 9, 10, 11, 12, 13, 14, 15, 17,
19, 20, 21, 22
User, 7, 16, 18
Web Publishing System, 1, 4, 5, 17

Thank you



Chapter 5

Requirements Engineering Process





Lesson Objectives

- Explain what is requirement engineering
- Describe the 5 activities in requirements engineering process
- Discuss the problems in requirement analysis phase of software engineering
- Explain and evaluate **Viewpoint-oriented analysis**



Introduction

What is Requirement engineering?

- ❖ Requirement engineering is a process that involves all of the activities required to create and maintain a system requirements document.
- ❖ Requirements engineering is the process of identifying, eliciting, analyzing, specifying, validating, and managing the needs and expectations of stakeholders for a software system.



Introduction

This process includes:

1. Feasibility Studies
2. Requirement Elicitation and Analysis
3. Requirement Specification (Discussed in Chapter 4)
4. Requirement Validation
5. Requirement Management



1. Feasibility Studies



Feasibility Studies

- A **feasibility study** is an analysis conducted to determine whether a proposed project or system is **practical, achievable, and worthwhile**, considering technical, economic, operational, and other relevant factors before development begins.



Overview of feasibility

Technical feasibility



Is the project technically possible?

Economic feasibility



Does it make financial sense ?

Legal feasibility



How can legal limitations affect the project?

Operational feasibility



How hard is it to maintain and manage the project?

Scheduling feasibility



Can we keep up with realistic deadlines?



1. Feasibility Studies

Technical Feasibility

An estimate is made of whether the identified user needs may be satisfied using current software and hardware technologies.

Technical feasibility inspects whether software can be built at all with available tools and experts



1. Feasibility Studies

Economical Feasibility

It will decide if the proposed system will be cost-effective and if it can be developed given existing budgetary constraints.

A Rough Order of Magnitude is the initial estimate that is often done before a project is started.



Legal feasibility

- makes sure that your product complies with all regulations and doesn't break any law.
- For example, medical software dealing with [protected health information](#) (PHI) must meet [HIPAA](#) rules. Besides that, you have to explore what legal risks there are and how they can impact your project.



Operational feasibility

- Operational feasibility explores how a new project will impact daily processes in your company.
- what procedures should be implemented, and what efforts should be taken to maintain it.



Schedule Feasibility

In Schedule Feasibility Study mainly timelines/deadlines is analyzed for proposed project which includes how many times teams will take to complete final project which has a great impact on the organization as purpose of project may fail if it can't be completed on time.



2. Requirements Analysis & Elicitation



2. Requirements Analysis & Elicitation

- This is the process of deriving the **system requirements**.
- Software engineers work with **stakeholders** to find out about the application domain, what services should provide, the required performance, hardware constraints, and etc.
 - Refer to any person or group who will be affected by the system, directly or indirectly.



2. Requirements Analysis & Elicitation



Deriving the system requirements

Stakeholders

Refer to any person or group who will be affected by the system, directly or indirectly.



2. Requirements Analysis & Elicitation



Stakeholders NOT CLEAR
what they want

Stakeholders use OWN
TERMS

Different stakeholders express
requirements in different ways

Difficult process



2. Requirements Analysis & Elicitation



Political factors

Changes due to economic &
business environment

Difficult process



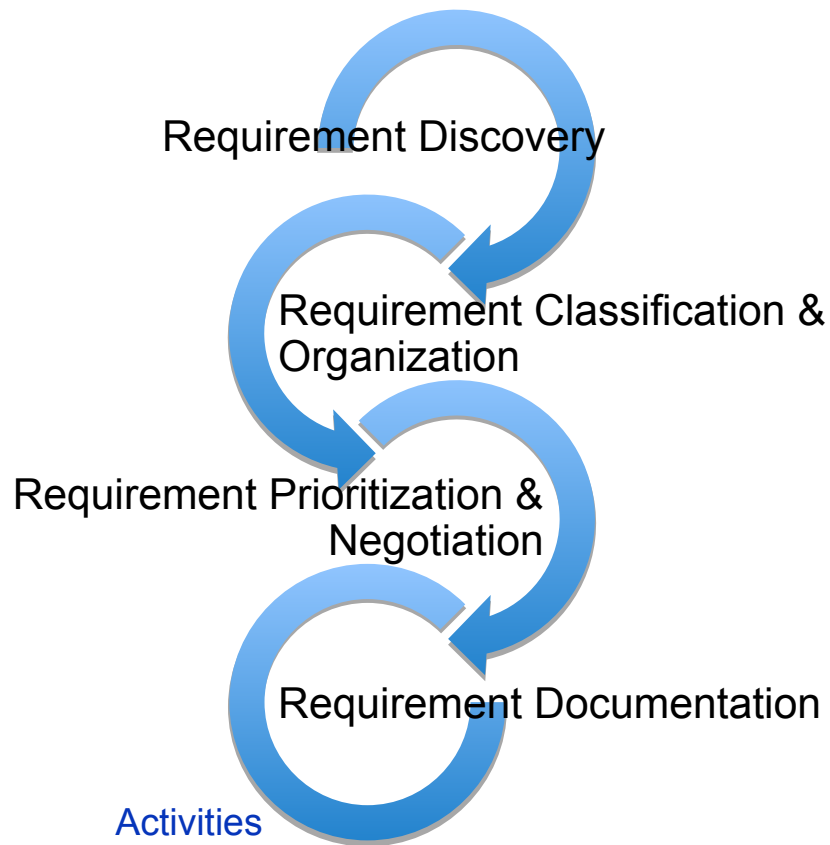
2. Requirements Analysis & Elicitation

It is a difficult process due to the following reasons:

- Users/stakeholders are **not clear** on what they want from the computer system
- Users/stakeholders normally **express requirements** in their **own terms**
- Different Users/stakeholders have different requirements and they may **express them in quite different ways**
- Analysis takes place in an organizational context and **political factors** may influence the requirements of the systems
- The economic and business environment in which the analysis takes place is dynamic. It inevitably **changes during the analysis process**.



2. Requirements Analysis & Elicitation



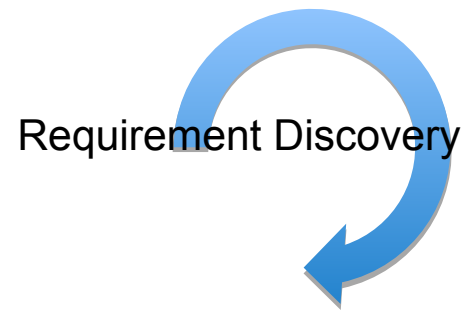


2. Requirements Analysis & Elicitation

Interact with stakeholders to discover requirements

Sources of info:

- ✓ Stakeholders
- ✓ Documentation
- ✓ Similar system specification





2. Requirements Analysis & Elicitation

Requirements Discovery

- This is the process of **interacting** with stakeholders in the system to **discover** their **requirements**
- Sources of information include:
 - Documentation
 - Stakeholders
 - Specification of similar systems
 - Etc.

Requirement Discovery





2. Requirements Analysis & Elicitation

Viewpoint-oriented Analysis

→ Interactor Viewpoints

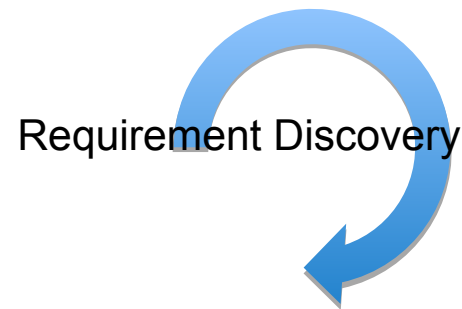
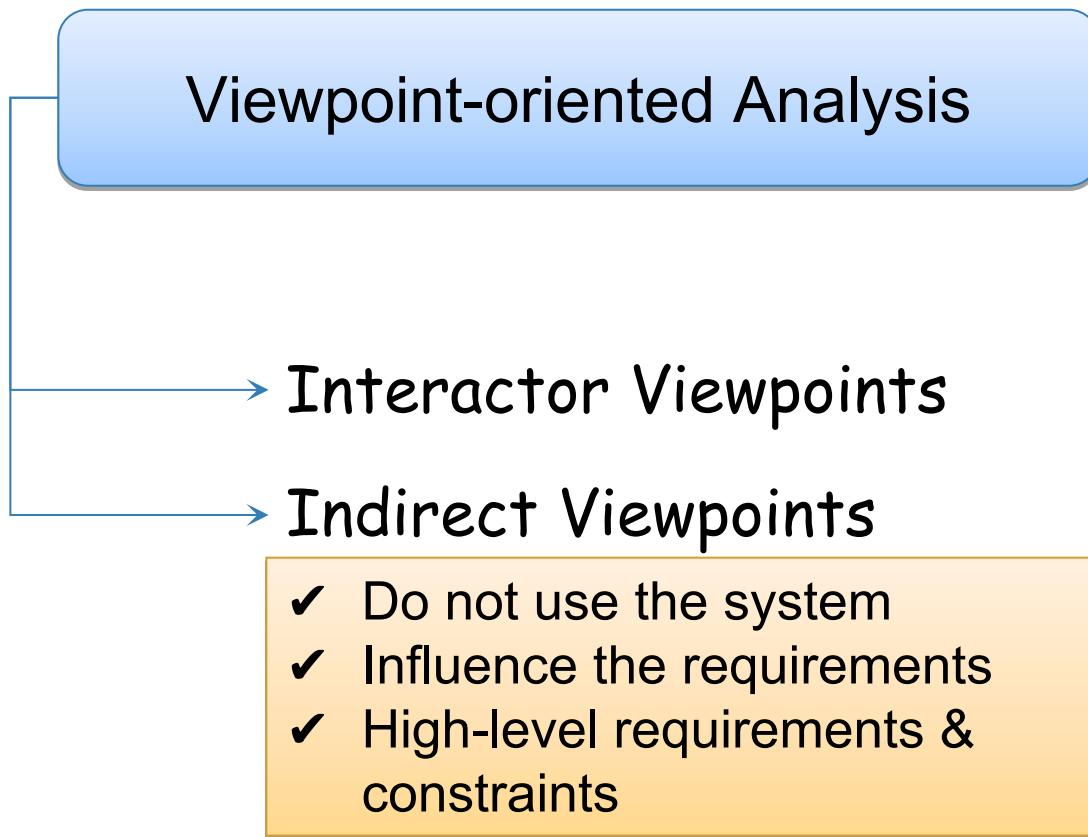
- ✓ Interact directly with system
- ✓ System features & interfaces

Requirement Discovery



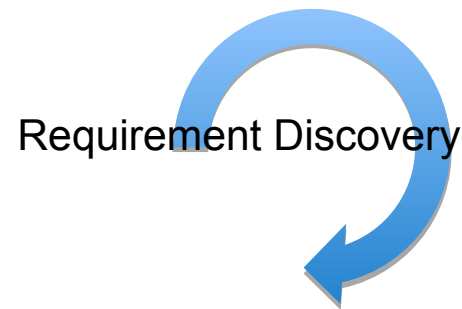
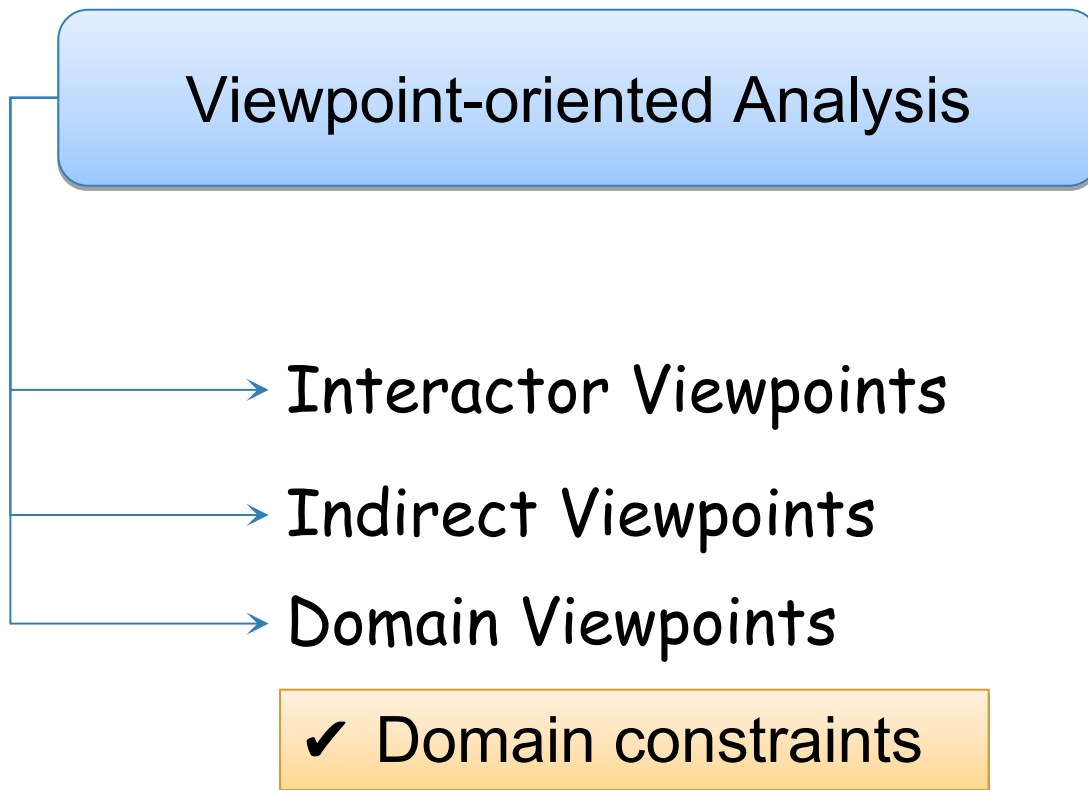


2. Requirements Analysis & Elicitation





2. Requirements Analysis & Elicitation





2. Requirements Analysis & Elicitation



- **Viewpoint-oriented analysis** is an approach that helps to ensure get **broad stakeholder coverage** when discovering requirements.
- Viewpoints can be used to **classifying** stakeholders and other sources of requirements.



2. Requirements Analysis & Elicitation

Viewpoint-oriented Analysis

Three generic types of viewpoints:

- **Interactor viewpoints** – people or other systems that interact directly with the system (**provide detailed system requirements like system features & interfaces**)
- **Indirect viewpoints** – stakeholders who do not use the system themselves but who influence the requirements in some way (**provide high-level organizational requirements & constraints**)
- **Domain viewpoints** – domain characteristics and constraints that influence the system requirements (**provide domain constraints**)

Requirement Discovery





advantages of viewpoints-oriented analysis

- **Structured Approach:** It provides a structured way to manage the complexity of requirements from diverse sources.
- **Conflict Identification:** It explicitly helps in finding and resolving conflicts between different stakeholder needs.
- **Completeness:** By systematically gathering requirements from various angles, it increases the likelihood of capturing all critical information.
- **Traceability:** It offers a mechanism to trace requirements back to their specific sources, improving accountability and validation.



Disadvantage of VP analysis:

- Time-consuming -VP analysis involves a detailed evaluation of the system's functions, cost, and performance. This includes collecting data,
- Tedious -The process often requires extensive data gathering, documentation, and calculations, which can be repetitive and labor-intensive.
- May be costly -VP analysis may require specialized tools, expert consultants, or additional testing, which increases the project's overall cost.



Exercise



ATM system

Viewpoint-oriented Analysis

→ Interactor Viewpoints

- ✓ bank's customers, bank's staff, bank's database

→ Indirect Viewpoints

- ✓ bank's manager, bank's security staff, bank's marketing staff

→ Domain Viewpoints

- ✓ UI standards
- ✓ Classification standards

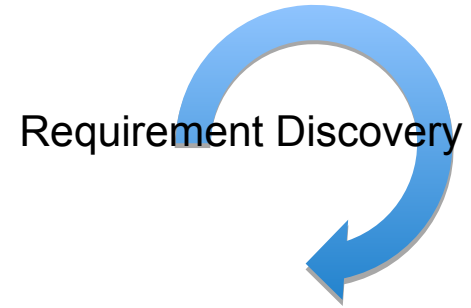


2. Requirements Analysis & Elicitation

Viewpoint-oriented Analysis

More specific types of viewpoints include:

- **Providers** and **receivers** of system services
- **Other systems** interface directly with the system
- **Regulations** and **standards** that apply to the system
- **Engineering** viewpoints
- **Marketing** and other viewpoints





Exercise

List the possible viewpoints for a college's library system.

Answer:

✓ ***Interactor VPs***

Students, Lecturer, Tutor, System Engineer, Librarian

✓ ***Indirect VPs***

Library manager, Finance Department, Suppliers

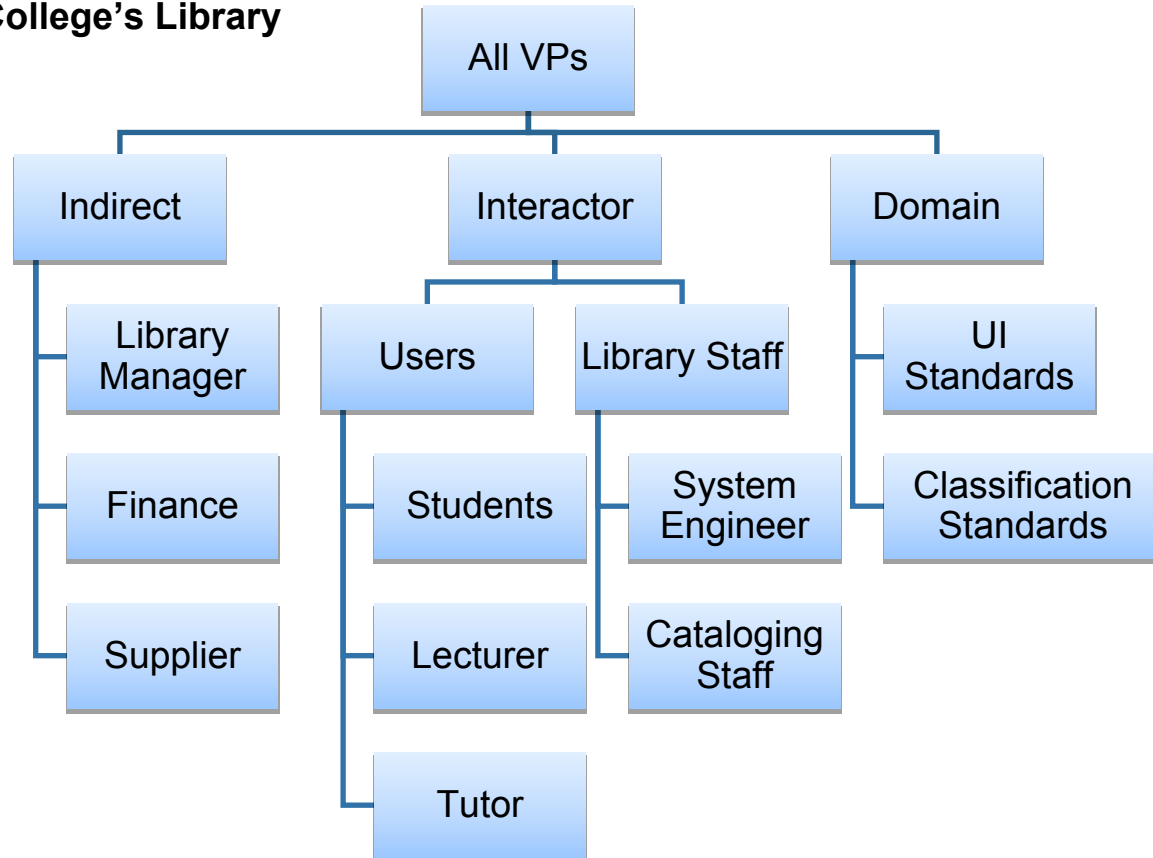
✓ ***Domain VPs***

UI Standards, Classification Standards



Exercise

Viewpoint Hierarchy for College's Library System





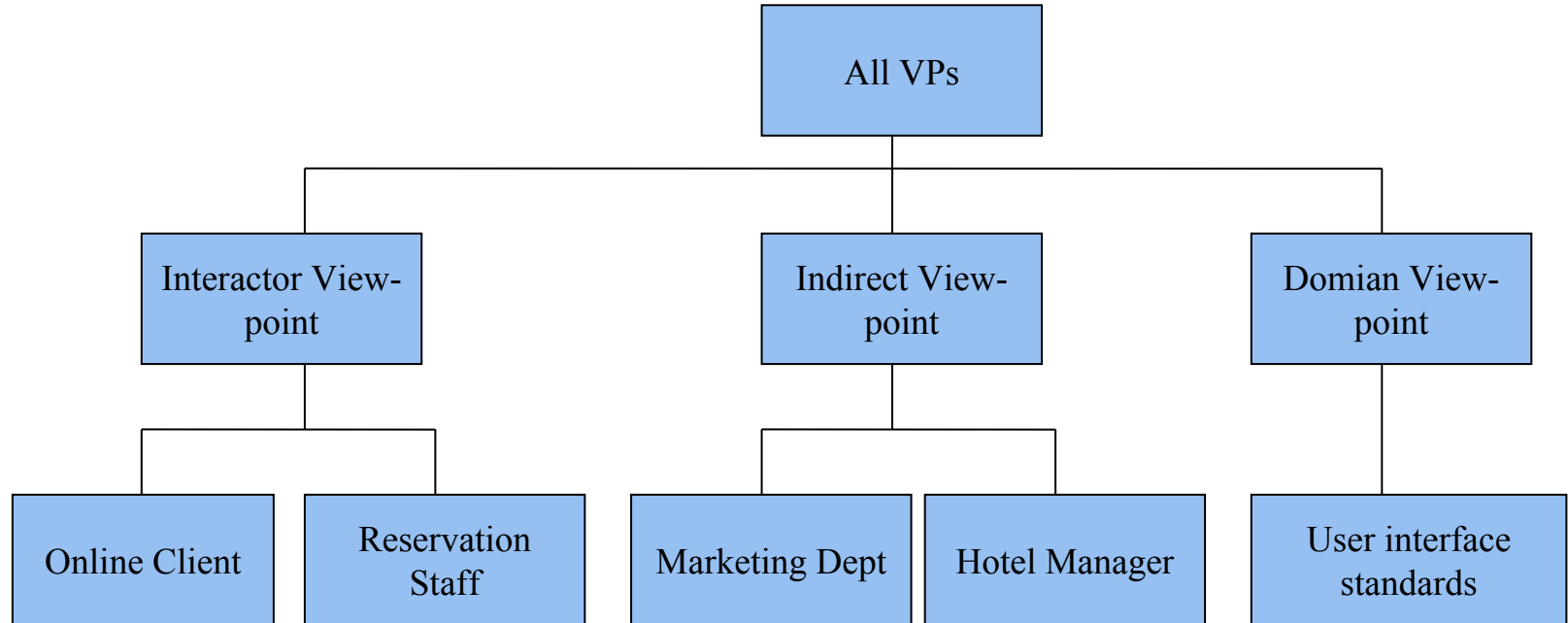
Exercise

Identify 5 viewpoints of an Online Hotel Room Reservation System and prepare a Viewpoint Hierarchy.



Exercise

Online Hotel Room Reservation System





Exercise

It is important to organize and structure the viewpoints into a hierarchy. WHY?

Answer:

- a) For large system, there are a huge number of viewpoints. It is practically impossible to elicit requirement from all of them. Hence, the hierarchy **helps to identify viewpoints** in the same branch are likely to **share common requirements**, and SE may not need to elicit requirement from each of them.



Exercise

It is important to organize and structure the viewpoints into a hierarchy. WHY?

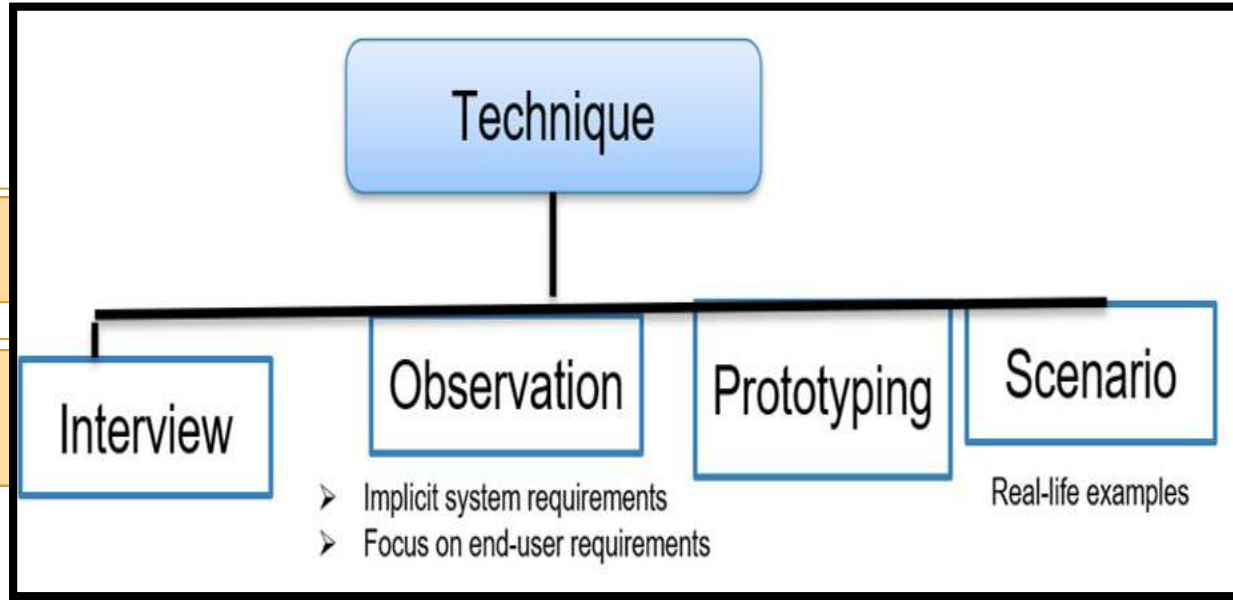
Answer:

- b) To help ensuring all VPs are covered in the analysis and elicitation stage.
- c) Help to decide whether a VP is a valid VP or not.



2. Requirements Analysis & Elicitation

Requirement Discovery





2. Requirements Analysis & Elicitation

Techniques of req. discovery include:

i) Interviewing

Interviewing is a valuable fact-gathering technique that involves asking questions to extract information from individuals who have relevant knowledge or experience

ii) Prototyping

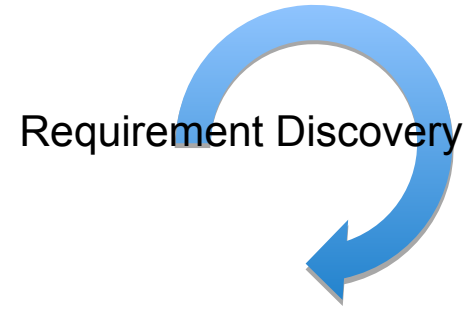
Prototyping is a crucial step in the design and development process, allowing for the creation of a preliminary version of a product or system to test and gather feedback





2. Requirements Analysis & Elicitation

Techniques of req. discovery include:



iii) Scenario

- Stakeholders usually find it easier to relate to real-life examples than abstract descriptions.
- Most general scenario are:
 - System and users expectation when the scenario starts
 - The normal flow of events
 - What can go wrong and how to handle
 - Info about other activities might go at the same time
 - System state when the scenario ends
- Scenarios may written in text plus diagrams. For e.g. Event scenario and Use-cases scenario



2. Requirements Analysis & Elicitation

Techniques of req. discovery include:

iv) **Ethnography** (Observation)

- an observational technique to understand social and organizational requirements
- Helps to discover implicit system requirements that reflect the actual rather than formal processes
- Focuses on end-users requirements only





2. Requirements Analysis & Elicitation

Requirement Classification & Organization

Organize requirements into clusters

Remove overlapping requirements





2. Requirements Analysis & Elicitation



Requirement Classification & Organization

- This activity takes the unstructured collection of requirements and **organizes** them into coherent clusters



2. Requirements Analysis & Elicitation

Requirement Classification & Organization

● Example: Hotel Management System

Customer
Registration

Check In

Room
Reservation

Check Out

Print Report

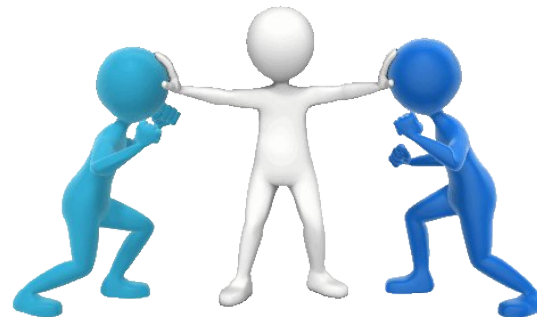


2. Requirements Analysis & Elicitation

Prioritize requirements

Resolve conflicts

Requirement Prioritization & Negotiation





2. Requirements Analysis & Elicitation

- Inevitable, where multiple stakeholders are involved, requirements will conflict. This activity is concerned with prioritising requirements, and finding and resolving these conflicts through negotiation

Requirement Prioritization & Negotiation





2. Requirements Analysis & Elicitation

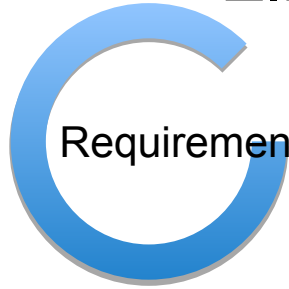
Requirement Documentation

- Complete
- Consistent
- In accordance with stakeholder's requirements



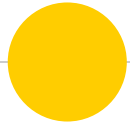


2. Requirements Analysis & Elicitation



Requirement Documentation

- The identified requirements are documented and checked to discover if they are **complete, consistent** and **in accordance** with what stakeholders really want from the system
- Formal and informal requirements documents may be produced



3. Requirements Specification

(Discussed in Chapter 4)



4. Requirements Validation



4. Requirements Validation

- Requirements validation is the process of checking that the defined requirements are for development, and defining the system that the customer really wants.
- “Are we building the right system?”



4. Requirements Validation



Several aspects of requirements that must be checked

Validity

- ✓ functions user wanted is correct and confirmed needed

Consistency

- ✓ No conflict

Electronic library system :

- A. Allow user to retrieve items in any format.
- B. No file conversion.



4. Requirements Validation



Several aspects of requirements that must be checked

Validity

Consistency

Completeness

Realism check

Verifiable



Measurable



Demo



4. Requirements Validation

- **Validity** - functions user wanted is correct and confirmed needed
- **Consistency** - no conflicting functions
- **Completeness** - include ALL functions needed
- **Realism check** – could be implemented (with existing technology, budget, schedule)
- **Verifiability** – requirements are measurable? to reduce potential dispute (demo that the delivered system meets each requirements)

5. Requirements Management



Requirements management is the process of validating and meeting the needs (or requirements) of customers and external and internal stakeholders. Requirements represent capabilities that will satisfy your [product strategy](#).



5. Requirements Management

1. Management Planning

A management plan is a comprehensive plan that provides the objectives of any given project, clearly defines roles and responsibilities, and more to make sure it's a success!

Your management plan is a resource that everyone in the firm can use for better guidance.

2. Change Management

methods and manners in which a company describes and implements change within both its internal and external processes. This includes preparing and supporting employees, establishing the necessary steps for change, and monitoring pre- and post-change activities to ensure successful implementation.

Effective communication is one of the most important success factors for effective change management.



5. Requirements Management

This process includes :

- **Management planning** - where policies and procedures for requirements management are designed and specified
- **Change management** - where proposed requirements changes are analyzed and their impacts are assessed.



Revision

Requirement Engineering:

1. Feasibility Studies
2. Requirement Elicitation and Analysis
3. Requirement Specification
4. Requirement Validation
5. Requirement Management



The End

Chapter 6

Architectural Design



Lesson Objectives

- Discuss why architectural design of software is important
- Explain the 3 main activities in architectural design – overall **system organization**, **modular decomposition** and **control modeling**
- Distinguish the **models/styles** involve in **System Organization** and **Control Modeling**



Architectural Design Activities

1. System Organization
2. Modular Decomposition
3. Control Modeling



Why Architectural Design is Important?



Introduction

- Large systems can be decomposed into sub-systems that provide some related set of services.
- The initial design process of **identifying** these **sub-systems** and establishing a framework for **sub-system control** and **communication** is called architectural design.
- Architectural design is the first stage in the design process and usually comes **before** detailed system specification.



Advantages of Explicit Architecture



1. Stakeholder communication

providing a clear, structured view of the system's design, ensuring all parties have a shared understanding of its components, functionalities, and dependencies.



Advantages of Explicit Architecture

2. System Analyst

- ✓ System analysts work with stakeholders to collect and understand business requirements, ensuring that the system addresses all necessary functions.



Advantages of Explicit Architecture



3. Large-scale reuse

Large-scale reuse refers to the practice of reusing software components, modules, or systems on a wide scale across different projects, applications, or organisations



Advantages of Explicit Architecture



4. Negotiation

Negotiation is the process through which two or more parties with differing interests or perspectives come together to reach a mutually agreeable solution



Advantages of Explicit Architecture



5. Complexity Management

Complexity Management refers to the practice of identifying, understanding, and controlling the various complex elements within a system, project, or process to ensure it operates efficiently and achieves its objectives



Architectural Design Activities

1. System Organization
2. Modular Decomposition
3. Control Modeling



1. System Organization/Structuring



1. System Organization/Structuring

- The system is structured into a number of **principal sub-systems** where a sub-system is an *independent software unit*.
- **Communications** between sub-systems are also identified.
- More specific models of the structure may be developed which show how sub-systems **share data**, how they are **distributed** and how they **interface** with each other.



1. System Organization/Structuring

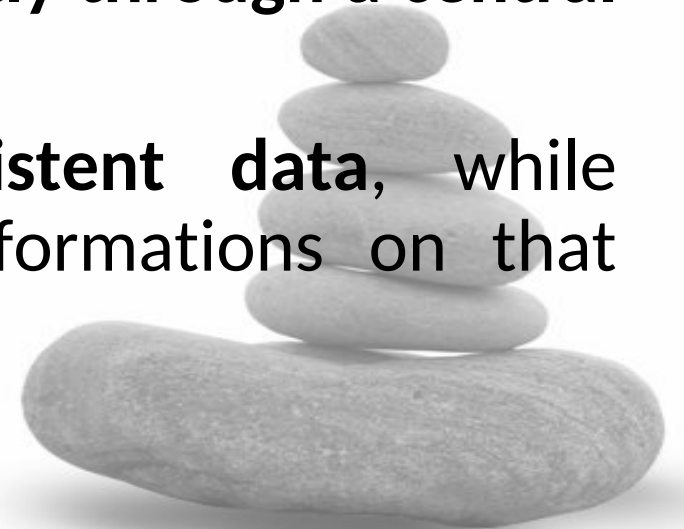
✓ 3 widely used system organization styles/models are:

- i) Repository model
- ii) Client-server model
- iii) Layered model



i)Repository model Architecture

- In software development, repository architecture plays a crucial role in managing data persistence and interaction between the application and data storage systems.
- **Repository Architecture** is a design pattern in which **all components of a system communicate indirectly through a central repository**, rather than directly with each other.
- The repository **stores the system's persistent data**, while components perform computations or transformations on that data.



Key Characteristics

1. Central Repository:

1. A single place where all data is stored, maintained, and accessed.
2. Acts as a “**source of truth**” for the system.

2. Independent Components:

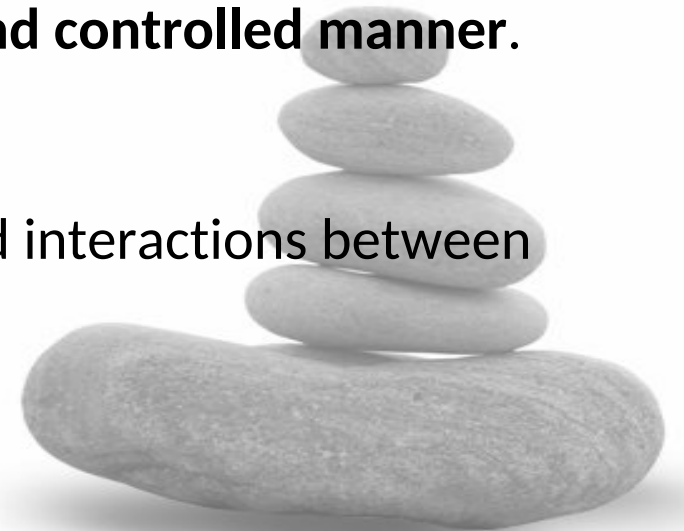
1. Modules or subsystems operate **independently**, reading from or writing to the repository.
2. Components **do not interact directly** with each other.

3. Shared Data:

1. All components can access the same data in a **consistent and controlled manner**.
2. Changes made by one component are **visible to all others**.

4. Data-Centric View:

1. The system is structured **around the repository**, not around interactions between modules.



Advantages

- **Data Consistency:** All components use the same data source.
- **Reusability:** Multiple components can use shared data without duplicating it.
- **Ease of Maintenance:** Updating the repository automatically updates data for all components.
- **Simplified Communication:** Components communicate indirectly via the repository.



Disadvantages

- **Single Point of Failure:** If the repository fails, the entire system may be affected.
- **Performance Bottleneck:** Heavy access to the central repository can slow down the system.
- **High Dependency:** Components are tightly coupled to the repository.



Examples

- **Version Control Systems:** Git, SVN.
- **Integrated Development Environments (IDEs):** Eclipse, Visual Studio.
- **Database-Centric Applications:** Enterprise Resource Planning (ERP) systems.
- **Content Management Systems (CMS):** WordPress, Drupal.



Example: Online Shopping System

- **Scenario:** An e-commerce web site platform has multiple modules:
 1. **Inventory Module:** Updates stock when a product is sold.
 2. **Order Module:** Processes customer orders.
 3. **Billing Module:** Generates invoices for purchases.
 4. **Shipping Module:** Manages delivery schedules.

Central Repository:

All modules share a **central database** that stores product details, stock levels, customer orders, and billing information.



1. System Organization/Structuring

i) Repository Model



Central DB to share data



1. System Organization/Structuring

ii) Client-Server Model

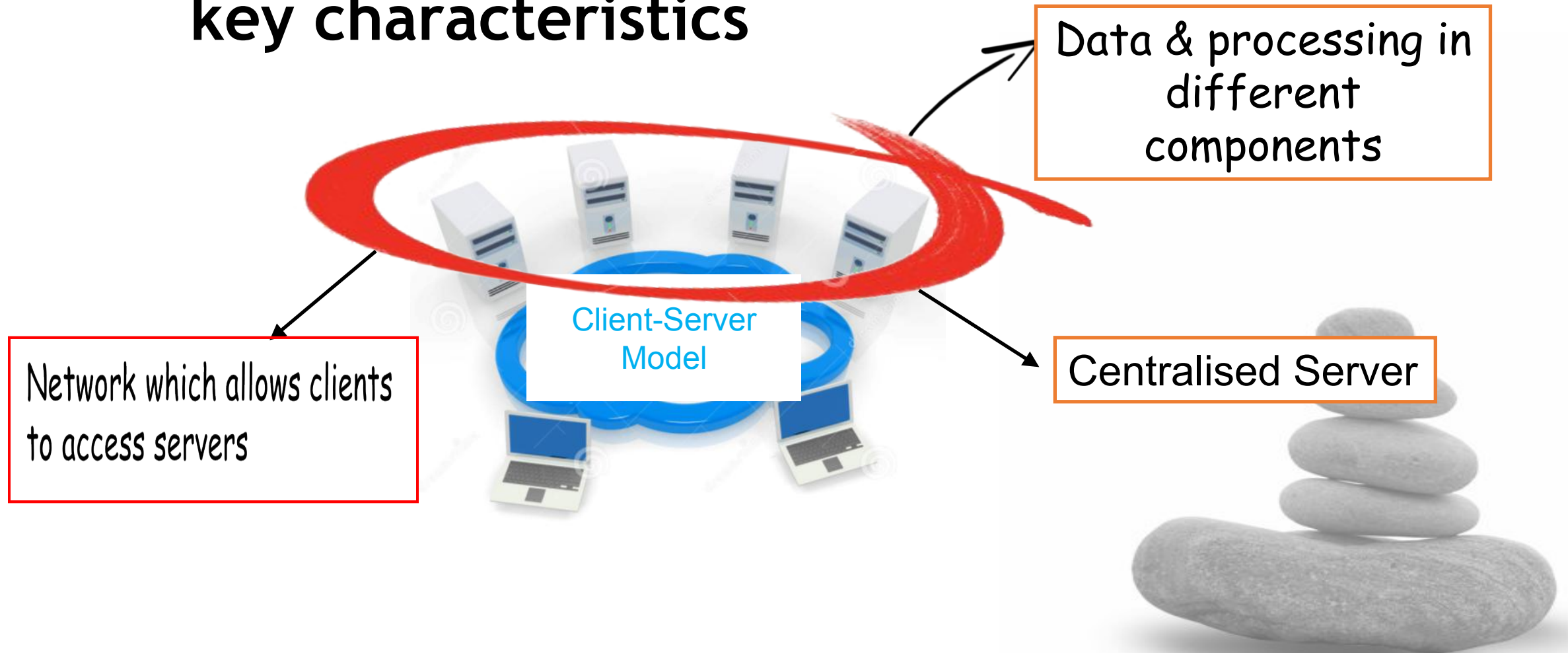


The **Client-Server Model** is one of the most widely used software architectures in which **system functionality is divided between clients and servers**. Clients request services or resources, and servers provide them.



1. System Organization/Structuring

key characteristics



1. System Organization/Structuring

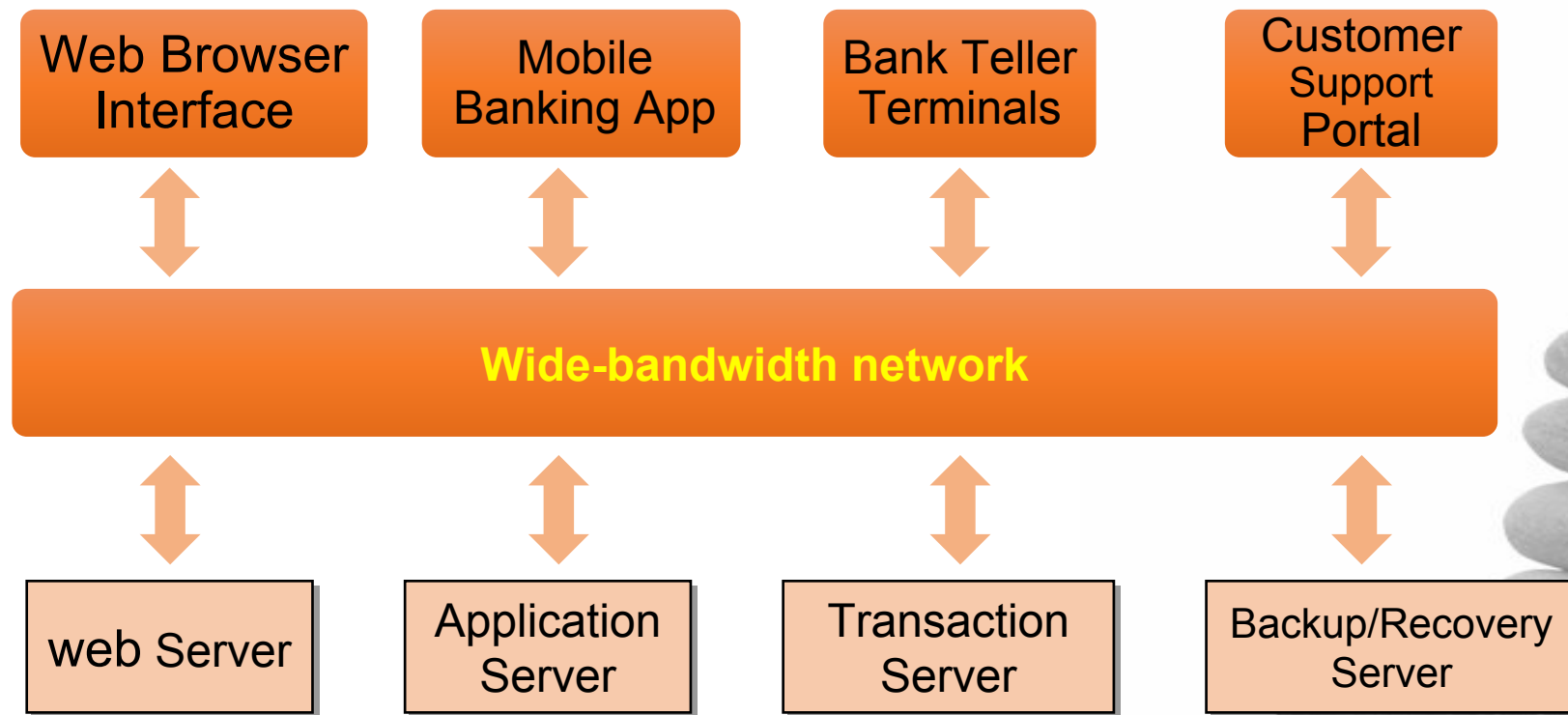
ii) Client-Server Model

- The Distributed System Model illustrates how **data and processing are shared across multiple interconnected components**. It enables several users to access applications or retrieve files from a single server, ensuring consistency and synchronization across all devices.
- Set of **centralised server** which provide **specific services** such as data management where set of clients which call on these services.
- Network which allows clients to access servers.



1. System Organization/Structuring

ii) **Client-Server Model** - In an **online banking system**, the **clients** are the front-end interfaces used by customers or bank staff, and the **servers** are the back-end systems that process transactions, store data, and manage services.



1. System Organization/Structuring

ii) Client-Server Model



✓ Distribution of data is straightforward

✓ Effective network system
✓ Cheaper hardware

✓ Easy to add or upgrade server



1. System Organization/Structuring

ii) Client-Server Model



✓ Different data model
→ inefficient data sharing

✓ Redundant mgmt

✓ No central register
of names & services

1. System Organization/Structuring

ii) Client-Server Model

- Advantages
 - **Distribution** of data is straightforward;
 - Makes effective use of **networked** systems. May require cheaper hardware;
 - Easy to add **new servers or upgrade existing servers**.
- Disadvantages
 - No shared data model so sub-systems use **different data organisation**. **Data interchange** may be **inefficient**;
 - **Redundant management** in each server;
 - **No central register of names and services** - it may be hard to find out what servers and services are available.



1. System Organization/Structuring

iii) Layered Model



Layered Model

✓ A.k.a Abstract Machine Model

✓ Model interfacing of sub-system

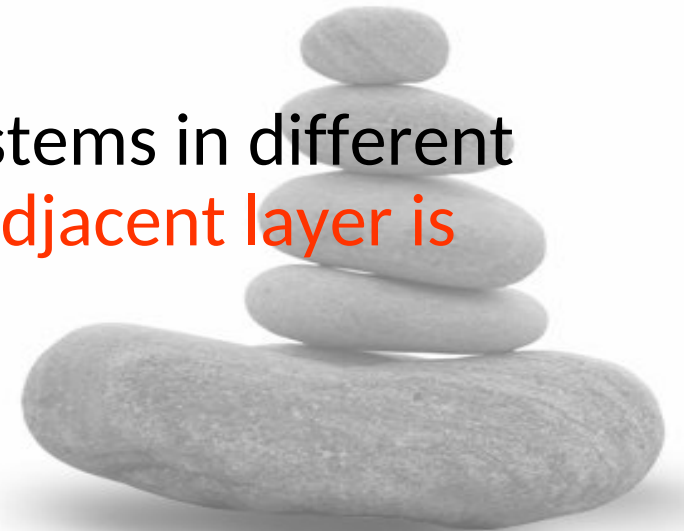
✓ Layers with services



1. System Organization/Structuring

iii) Layered Model

- Also called *abstract machine model*
- Used to model the **interfacing** of sub-systems.
- Organises the system into a **set of layers** each of which provide a **set of services**
- Supports the incremental development of sub-systems in different layers. When a layer interface changes, **only the adjacent layer is affected**



1. System Organization/Structuring

iii) Layered Model



Key characteristics

- ✓ Support incremental development

- ✓ Closed layered model
→ one layer change,
only adjacent layer is
affected



1. System Organization/Structuring

iii) Layered Model - E-commerce apps/website

Presentation layer--Web pages, mobile app interfaces

Application layer-(Service Layer)--
Controllers, APIs

Business logic layer--Account balance calculations, order processing

Database layer--MySQL, Oracle, MongoDB



Advantages of the Layered Model

- **Separation of Concerns**

Each layer handles a specific function (e.g., presentation, business logic, data access), making the system easier to understand and manage.

- **Ease of Maintenance**

Since layers are independent, changes in one layer (like the UI) do not directly affect others (like the database layer).

- **Reusability**

Components from one layer (for example, the data access layer) can be reused in other applications or projects.



Disadvantages of the Layered Model

- **Complexity in Layer Management**

Adding or maintaining multiple layers can make the system structure complex, especially for small projects.

- **Performance Overhead**

Data must pass through multiple layers, which can increase latency and reduce performance compared to simpler architectures.

- **Difficult Debugging Across Layers**

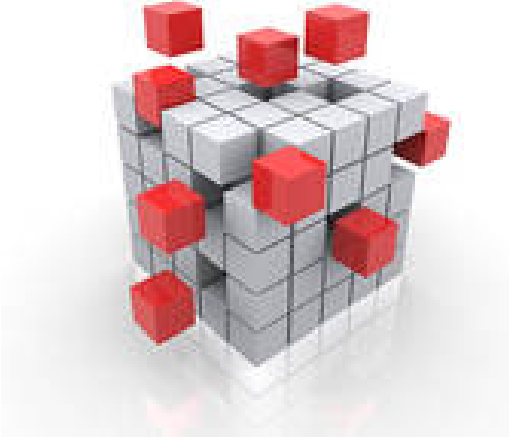
Tracing an issue through multiple layers (from UI to database) can be time-consuming and complex.



2. Modular Decomposition

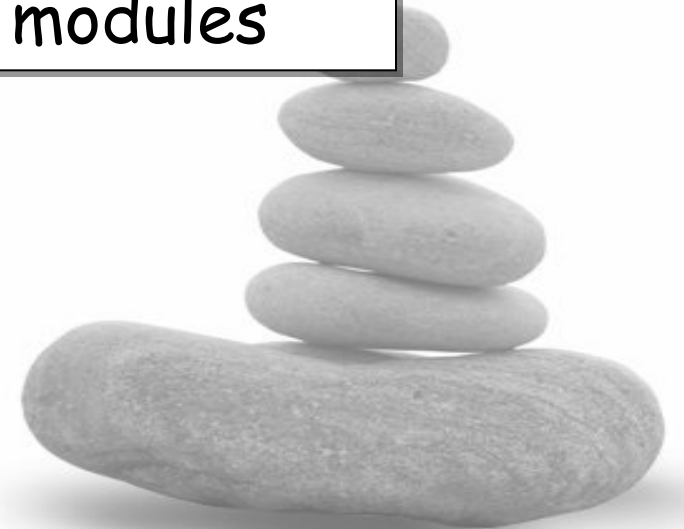


2. Modular Decomposition



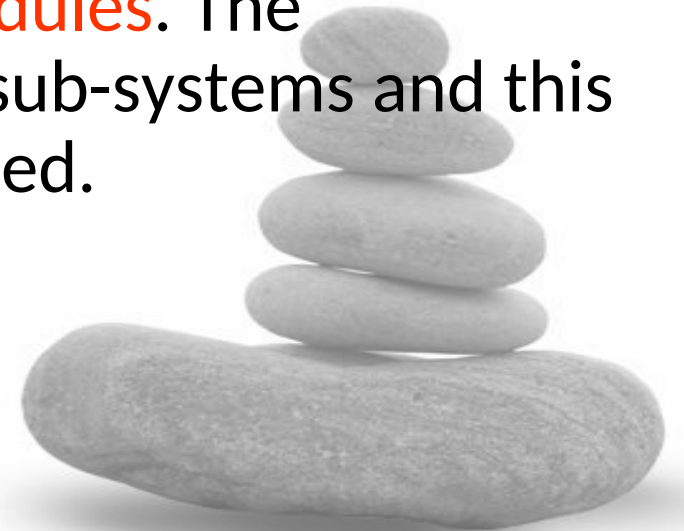
➤ Decomposition of sub-systems into **modules**

➤ Components in modules

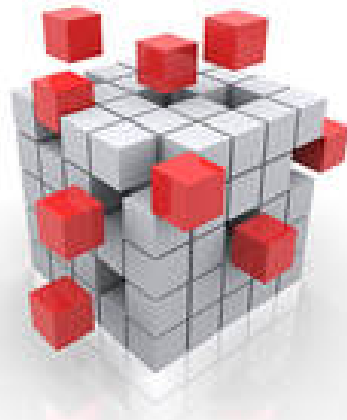


2. Modular Decomposition

- After an overall system organization has been chosen, you need to make a decision on the approaches to be used to decompose sub-systems into modules.
- This is the decomposition of sub-systems into **modules**. The components in modules are usually smaller than sub-systems and this allows alternative decomposition models to be used.



2. Modular Decomposition



Strategies

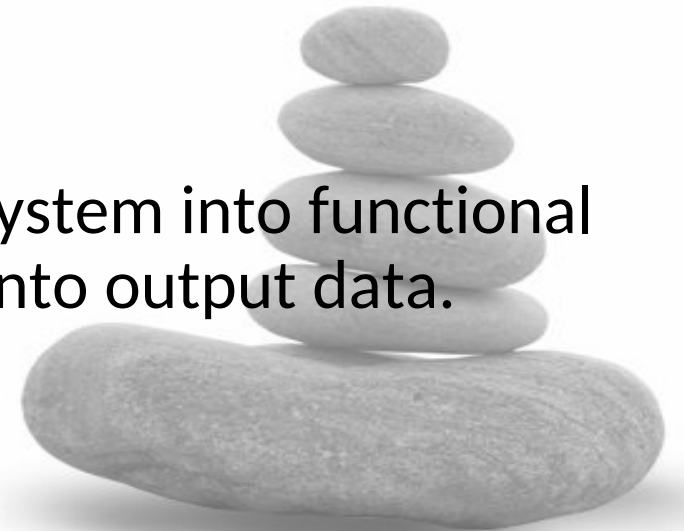
➤ Object-Oriented Decomposition

➤ Function-Oriented Decomposition



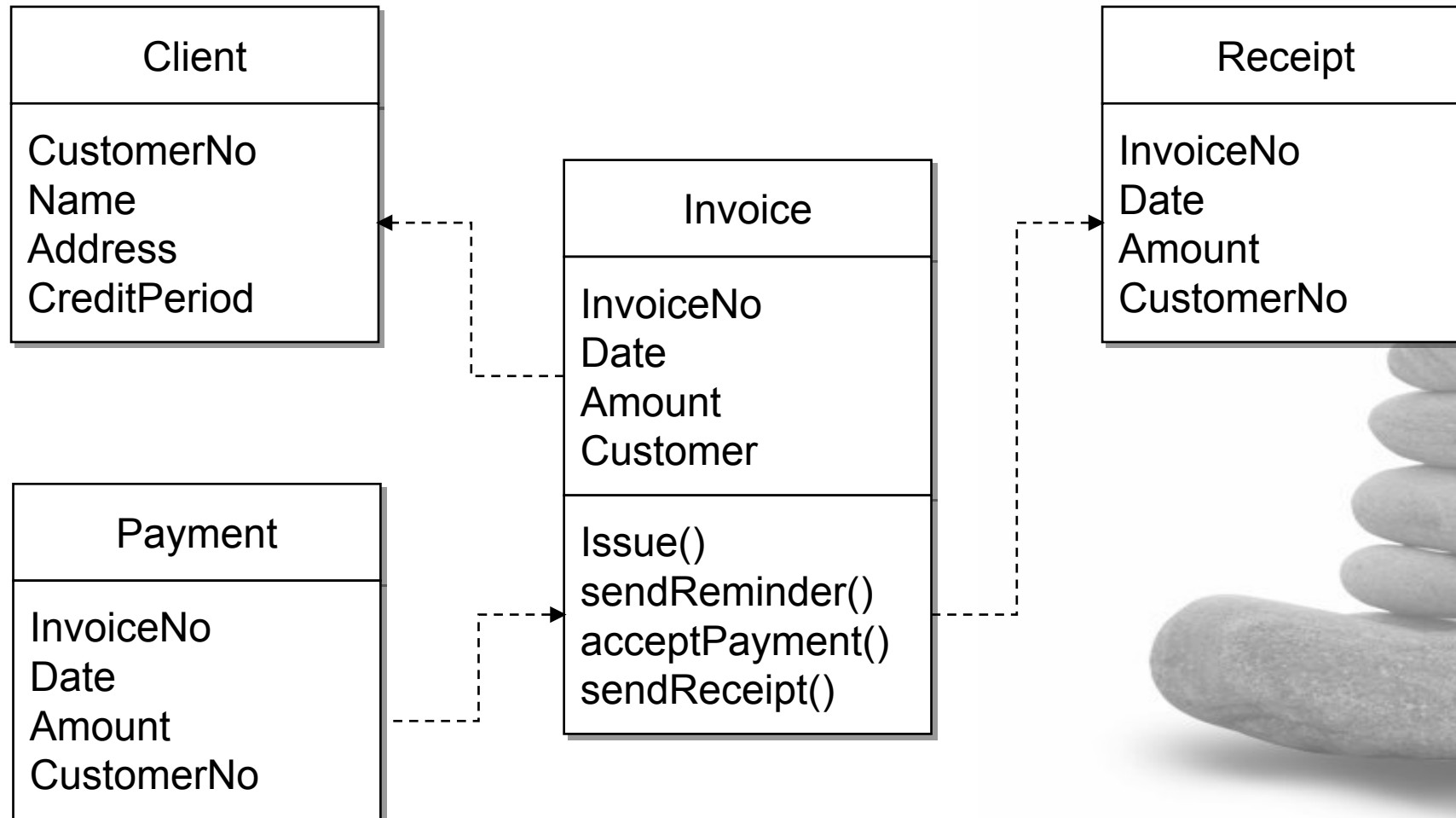
2. Modular Decomposition

- 2 main strategies on decomposing a sub-system into modules:
 - i) **Object-oriented decomposition**
 - an object model where decompose a system into a set of communicating objects
 - ii) **Function-oriented decomposition**
 - an pipeline/ data-flow model where decompose a system into functional modules that accept input data and transform them into output data.



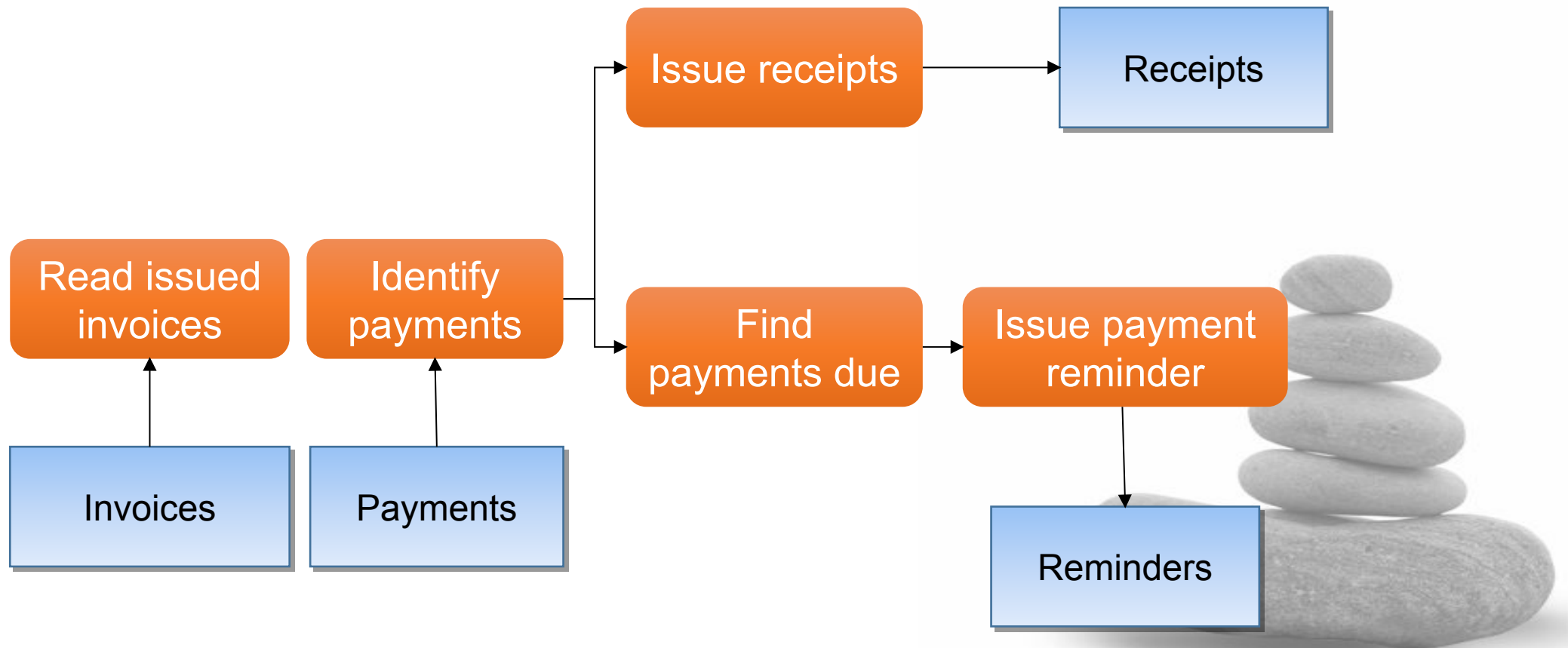
2. Modular Decomposition

Object-Oriented Decomposition - Invoice Processing System



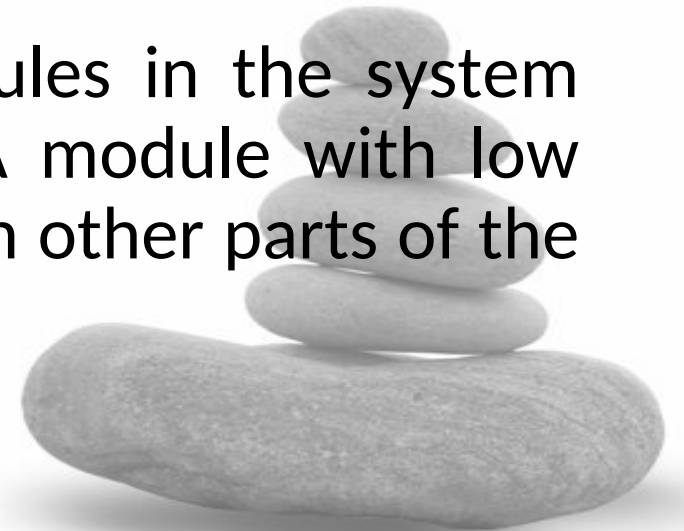
2. Modular Decomposition

Function-Oriented Decomposition - Invoice Processing System



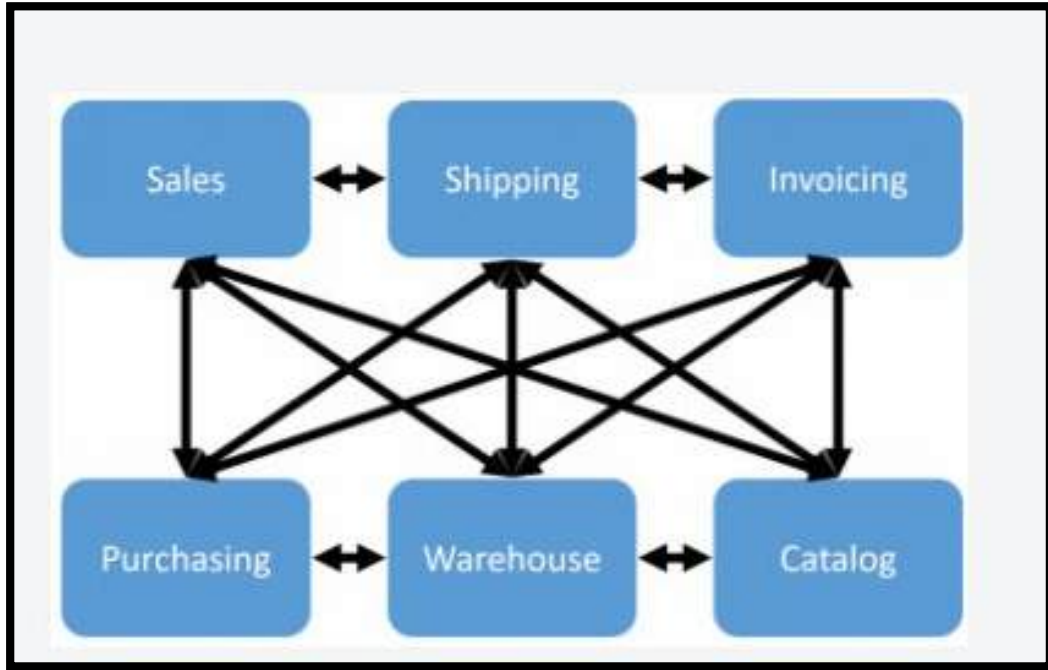
Cohesion vs. Coupling

- Cohesion and coupling are related to each other. **Each can affect the level of the other.**
- High cohesion means that the components (such as classes or modules) of your system are highly focused on a single responsibility or closely related responsibilities. A highly cohesive class or module does one thing and does it well.
- Low coupling means that components or modules in the system have minimal dependencies on one another. A module with low coupling can change without requiring changes in other parts of the system.

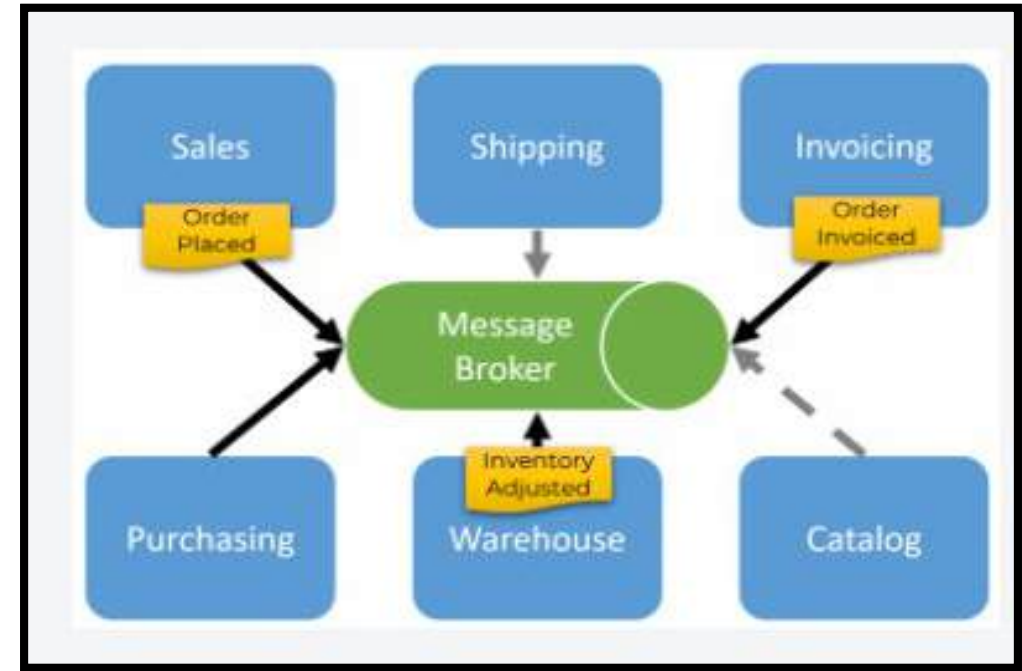


	Cohesion	Coupling
1	Cohesion is the degree to which the elements inside a module belong together.	Coupling is the degree of interdependence between the modules.
2	A module with high cohesion contains elements that are tightly related to each other and united in their purpose.	Two modules have high coupling (or tight coupling) if they are closely connected and dependent on each other.
3	A module is said to have low cohesion if it contains unrelated elements.	Modules with low coupling among them work mostly independently of each other.
4	Highly cohesive modules reflect higher quality of software design	Loose coupling reflects the higher quality of software design





cohesion



coupling

High Cohesion and Low Coupling is the ideal design choice in software development

3. Control Modeling



3. Control Modeling

✓ Sub-systems need to be controlled

Services are delivered to the right place at the right time



3. Control Modeling

- The models for structuring a system are concerned with how a system is decomposed into sub-systems. However, to work as a system, sub-systems must be **controlled** so that their **services** are **delivered** to the **right place** at the **right time**.
- Structural/ Organizational models **do not** include control information.
- Hence, a general model of the **control relationships** between the parts of the system is established
 - Concerned with the control flow between sub-systems



3. Control Modeling

Concerned with the
control flow between
sub-systems

General model of the
control relationships
between sub-systems



3. Control Modeling

i) Centralized control

- Call-return model
- Manager model



2 general control styles

ii) Event-based control

- Broadcast model
- Interrupt-driven model



3. Control Modeling

i) Centralized Control



One sub-system has **overall responsibility for control** and starts and stops other sub-systems.



3. Control Modeling

i) Centralized Control - Call-Return Model



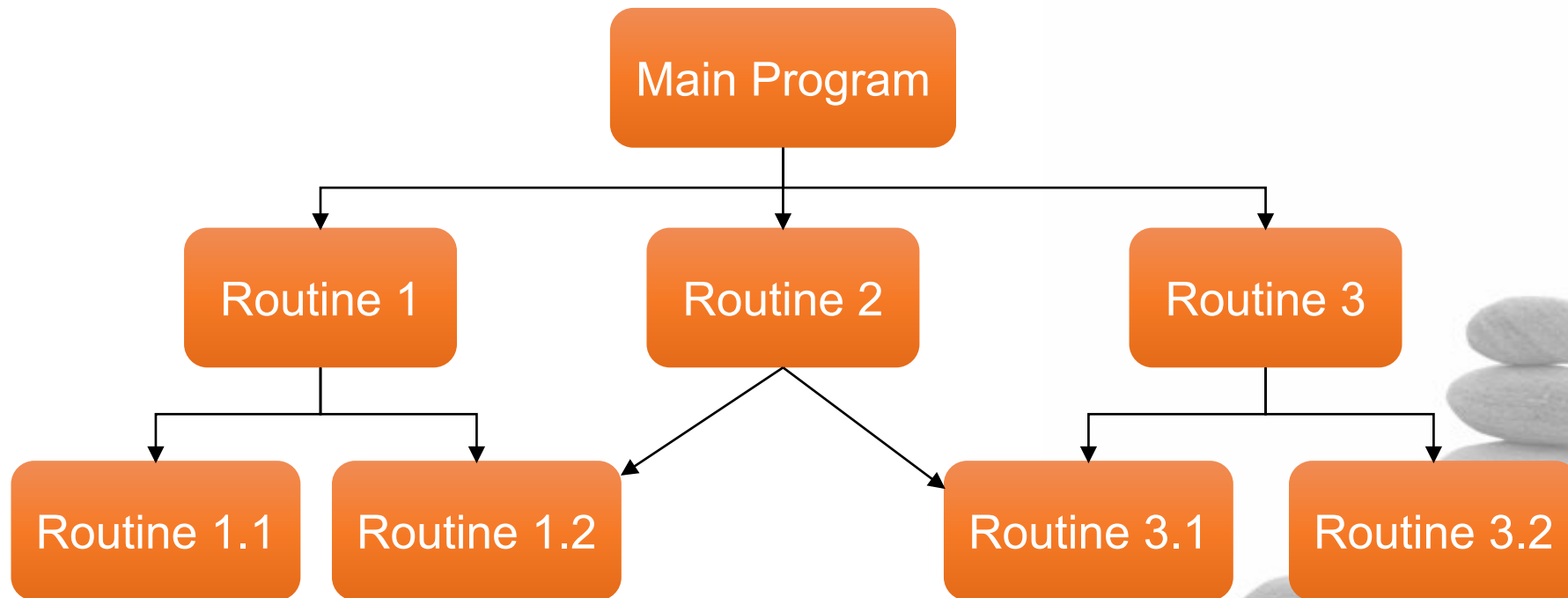
Top-down subroutine model

Sequential systems



3. Control Modeling

i) Centralized Control - Call-Return Model



- top down sub-routine model
- applicable for ***sequential systems***

3. Control Modeling

i) Centralized Control - Manager Model



One system manager

Concurrent systems

Sequential systems → case statement



3. Control Modeling

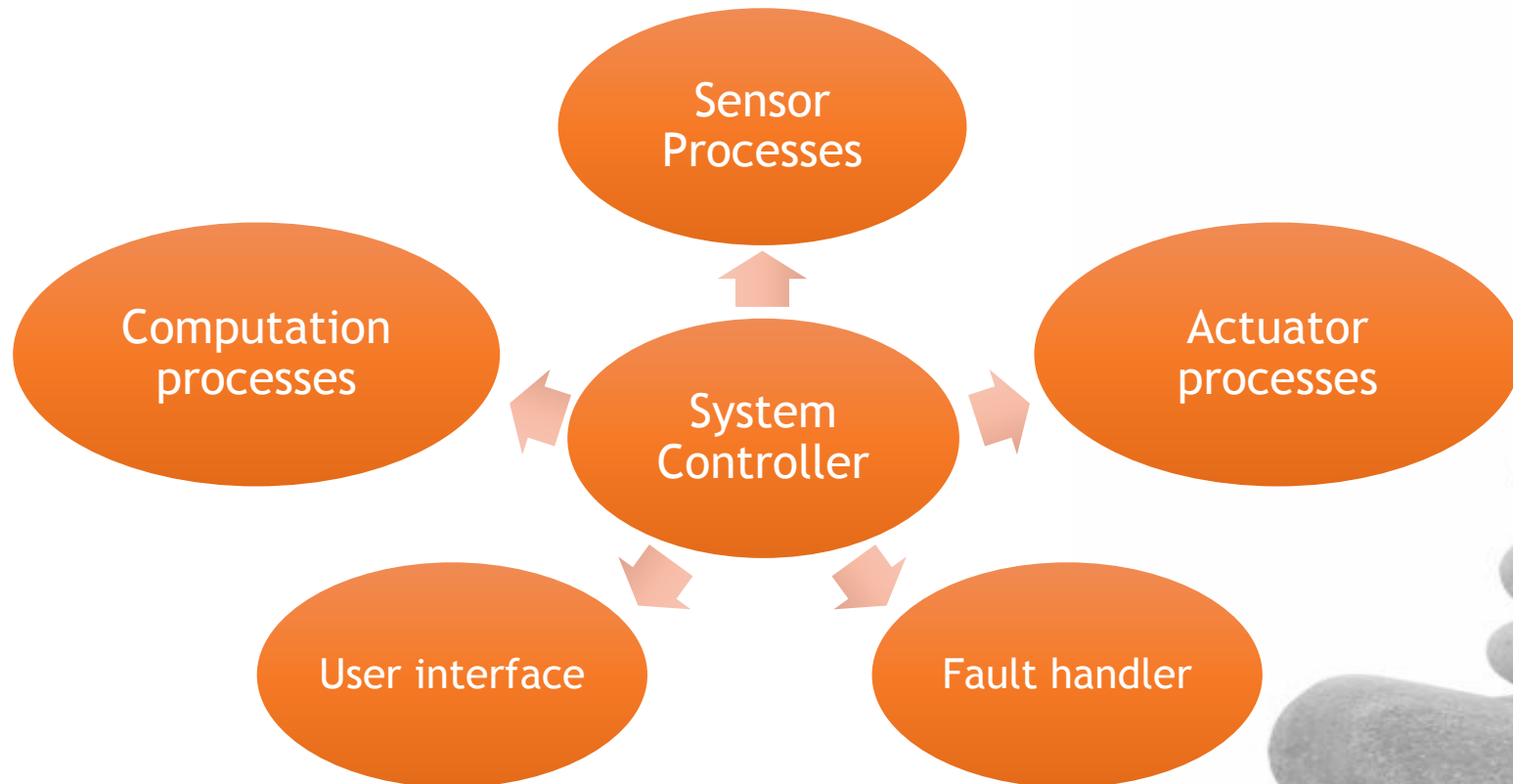
i) Centralized Control – Manager Model

- One system component is designated as system manager i.e. controls the stopping, starting and coordination of other system processes.
- Applicable to concurrent systems.
- Can be implemented in sequential systems as a case statement.



3. Control Modeling

i) Centralized Control - Manager Model



- a system component is designated as system manager
- processes(sub-systems) can execute concurrently
- applicable for **concurrent systems**

3. Control Modeling

ii) Event-based Control



Response to **event** from
other system or
environment



3. Control Modeling

ii) Event-based Control

- each sub-system can respond to externally generated events that might come from other sub-systems or environment.



3. Control Modeling

ii) Event-based Control - 1. Broadcast Model



- Event is broadcasted
- Whoever can handle will respond



3. Control Modeling

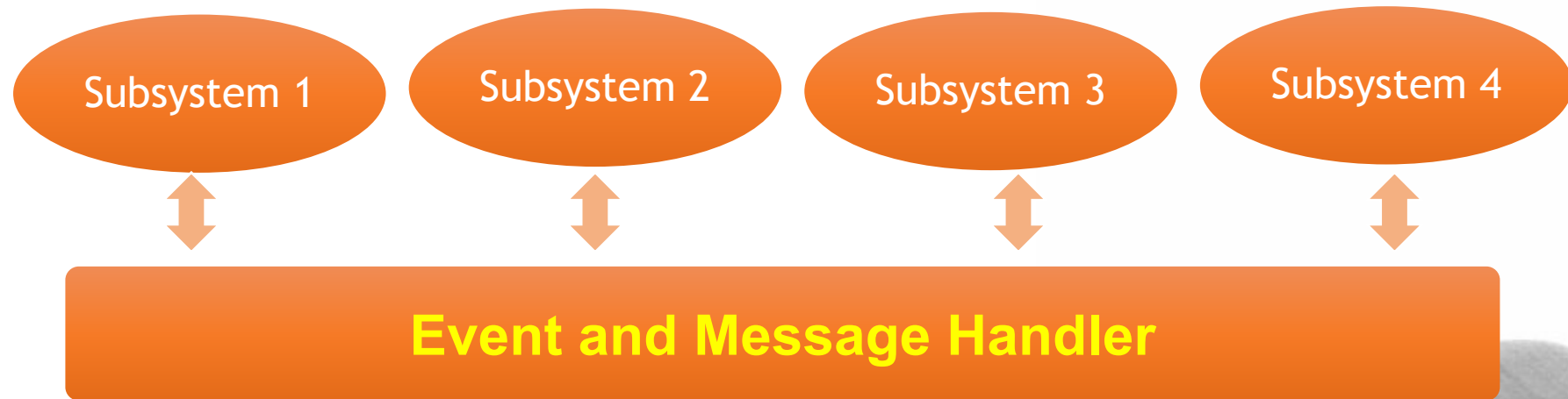
ii) Event-based Control – 1. Broadcast Model

- An event is broadcast to all sub-systems. Any sub-system which can handle the event may respond to it.
- Effective in integrating sub-systems distributed across different computers on a network



3. Control Modeling

ii) Event-based Control - 1. Broadcast Model



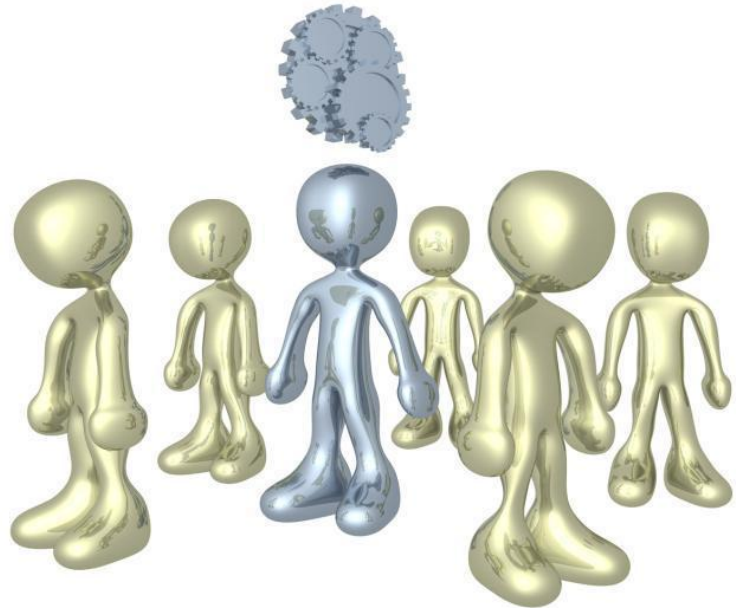
Event	Subsystem
E1	SS1
E2	SS2
E3	SS3
E4	SS4

→ an event is broadcast to all sub-systems

→ effective in integrating sub-systems distributed across different computers on a network

3. Control Modeling

ii) Event-based Control - 2. Interrupt-Driven Model



➤ Interrupt Handler

➤ Real Time System



3. Control Modeling

ii) Event-based Control –

2. Interrupt-Driven Model

- Used in real-time systems where interrupts are detected by an interrupt handler and passed to some other component for processing.



1. System Organization/Structuring

- i) Repository model
- ii) Client-server model
- iii) Layered model

2. Modular Decomposition

- i) Object-oriented Decomposition
- ii) Functional-oriented Decomposition

3. Control Modeling

- i) Centralized Control
 - a. Call-Return Model
 - b. Manager Model
- ii) Event-based Control
 - a. Broadcast Model
 - b. Interrupt-driven Model



Thank you





Chapter 7 Software Testing

Lesson Objectives

- Software testing fundamentals
- Testing techniques and strategies
- Test case design and planning
- Testing guidelines/ principles



Software Testing Fundamentals

Introduction



- The quality of a piece of software depends on its design, development, testing, and implementation
- One aspect of **software quality** is its **reliability**. A software is reliable if, when used in a reasonable manner, it does not produce failures that are dangerous or costly
- Testing can only demonstrate the presence of errors. It **cannot show** that there are **no errors in a program**.
- Testing should be scheduled as **part** of the **project planning** process. Adequate resources must be made available for testing.

Introduction

Validation Testing

- ✓ Software meets the requirements



Defect Testing

- ✓ Discover faults & defects



Main Goals of Testing

Introduction

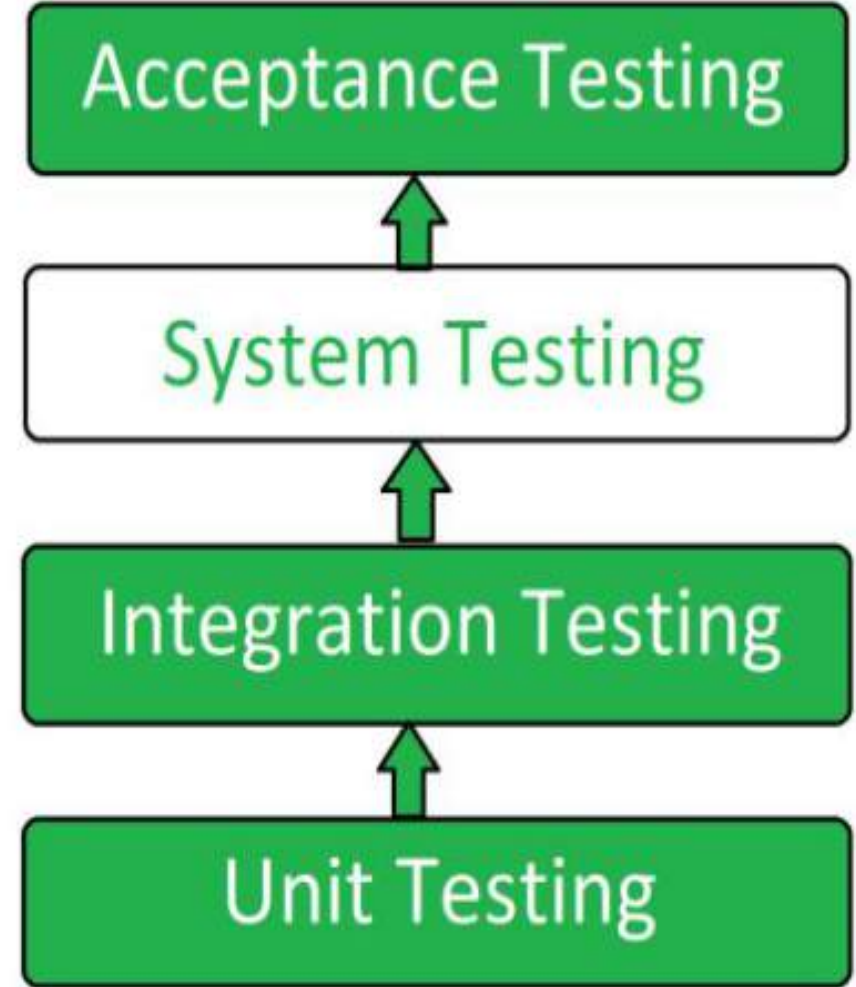
Main Goals of Testing

- To demonstrate to the developer and the customer that the **software meets the requirements**
 - validation testing
- To **discover faults or defects** in the software where the behaviour of the software is incorrect, undesirable or does not conform to its specification
 - defect testing



Levels of Testing

- Unit Testing
- Integration Testing
- System Testing
- Acceptance Testing

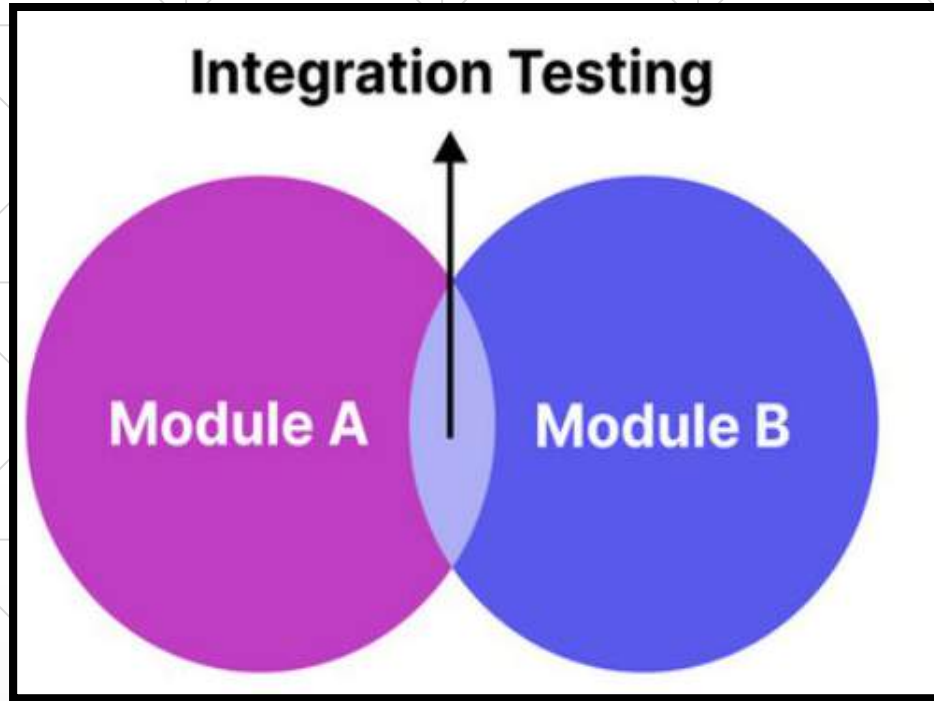


System Testing

Unit Testing

- **Unit Testing** is a software testing technique in which individual units or components of a software application are tested in isolation.
 - These units are the smallest pieces of code, typically functions or methods, ensuring they perform as expected.
 - Carried out by developers
 - Goal: Confirm that subsystems is correctly coded and carries out the intended functionality
-

Integration testing



Integration testing is the process of testing the interface between two software units or modules. It focuses on determining the correctness of the interface.

The purpose of integration testing is to expose faults in the interaction between integrated units. Once all the modules have been unit-tested, integration testing is performed.

Carried out by developers

Goal: Test the *interface* among the subsystem

Testing Techniques - Integration Testing

1. Top Down Testing



System testing



Start from upper modules
& works downwards

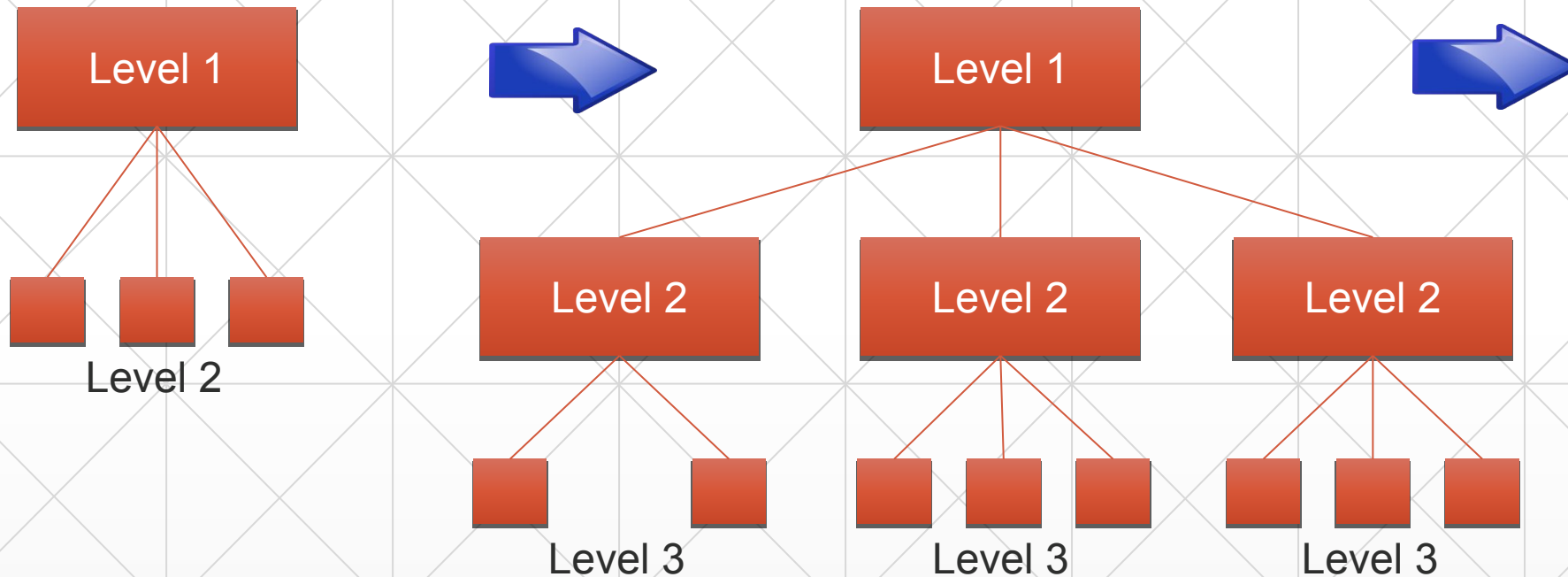
Integration Testing:
Top Down Testing

Top-down integration testing technique

- Top-down integration testing technique is used in order to simulate the behaviour of the lower-level modules that are not yet integrated. In this integration testing, testing takes place from top to bottom.
- First, high-level modules are tested and then low-level modules and finally integrating the low-level modules to a high level to ensure the system is working as intended.

Testing Techniques - Integration Testing

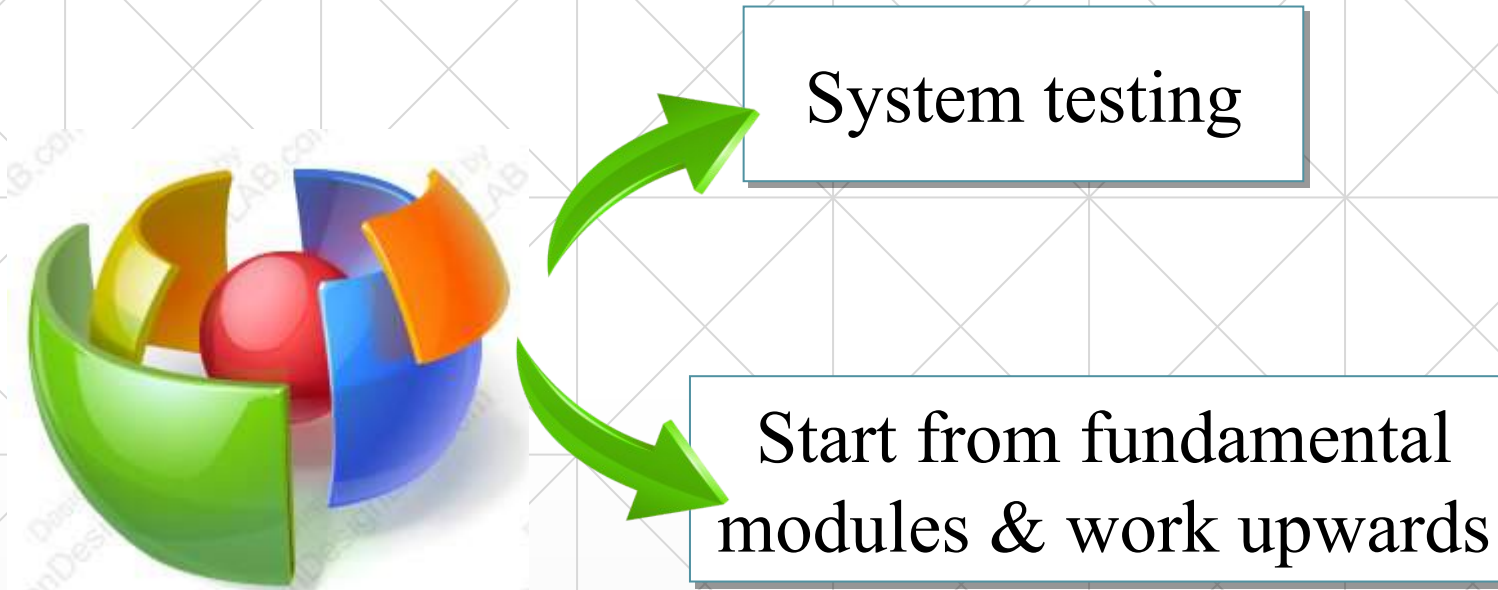
1. Top Down Testing



Integration Testing: Top Down Testing

Testing Techniques - Integration Testing

Bottom Up Testing



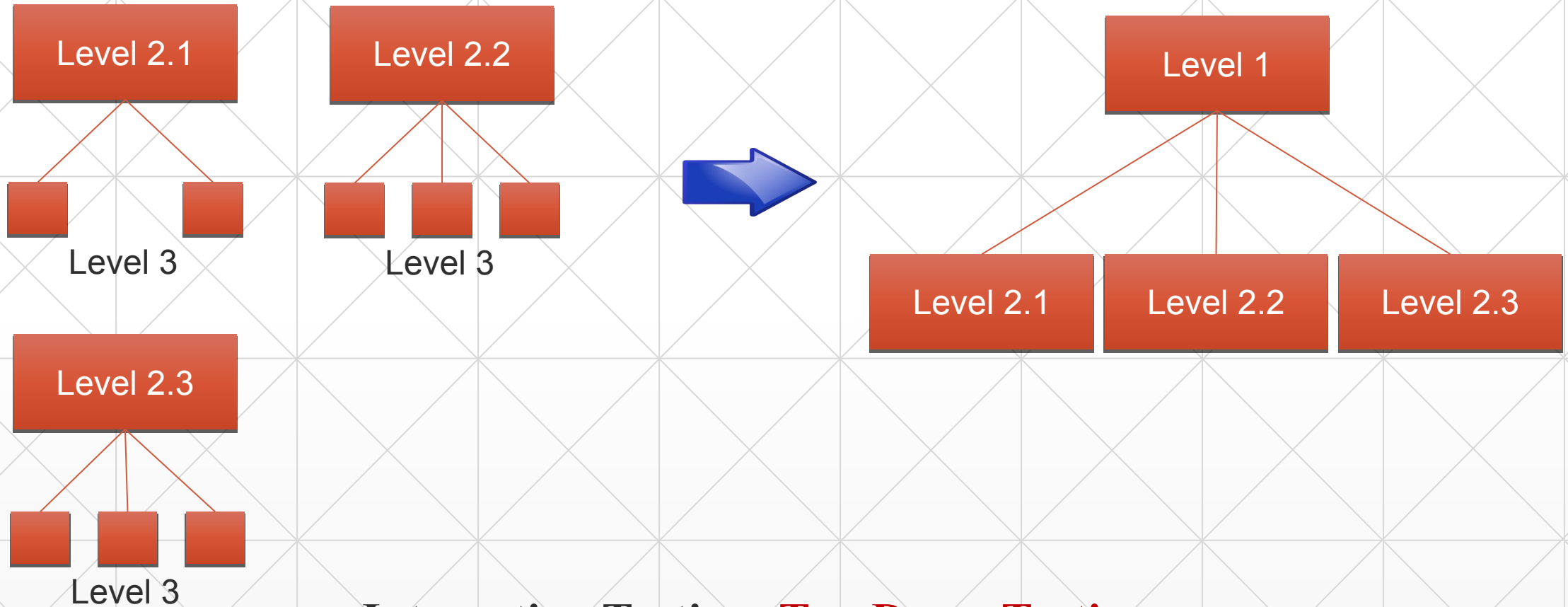
Integration Testing:
Bottom Up Testing

2. Bottom-up testing

- In bottom-up testing, each module at lower levels are tested with higher modules until all modules are tested. The primary purpose of this integration testing is that each subsystem tests the interfaces among various modules making up the subsystem.
- This integration testing uses test drivers to drive and pass appropriate data to the lower-level modules.

Testing Techniques - Integration Testing

Bottom Up Testing



Integration Testing: **Top Down Testing**

3. Regression testing

- Regression testing in integration testing involves retesting the integrated components/modules to ensure that new changes have not introduced any issues or broken existing functionality at the integration level.
- **Regression testing** is a crucial aspect of **software engineering** that ensures the stability and reliability of a software product

System testing

- System testing is a type of software testing that evaluates the overall functionality and performance of a complete and fully integrated software solution.
- It tests if the system meets the specified requirements and if it is suitable for delivery to the end-users
- System Testing is performed by a testing team that is independent of the development team and helps to test the quality of the system impartial.

Summary

1. Top-down testing

- ✓ As the name implies, begins with the upper-level modules and works downwards

2. Bottom-up testing

- ✓ Where testing starts with the fundamental components and works upwards

3. Regression testing

- ✓ Rerun the tests for previous increments when a new increment is integrated
- ✓ If problems, should check whether these problems in previous increment that the new increment has exposed or whether these are due to the added increment of functionality
- ✓ Expensive

[illegible]

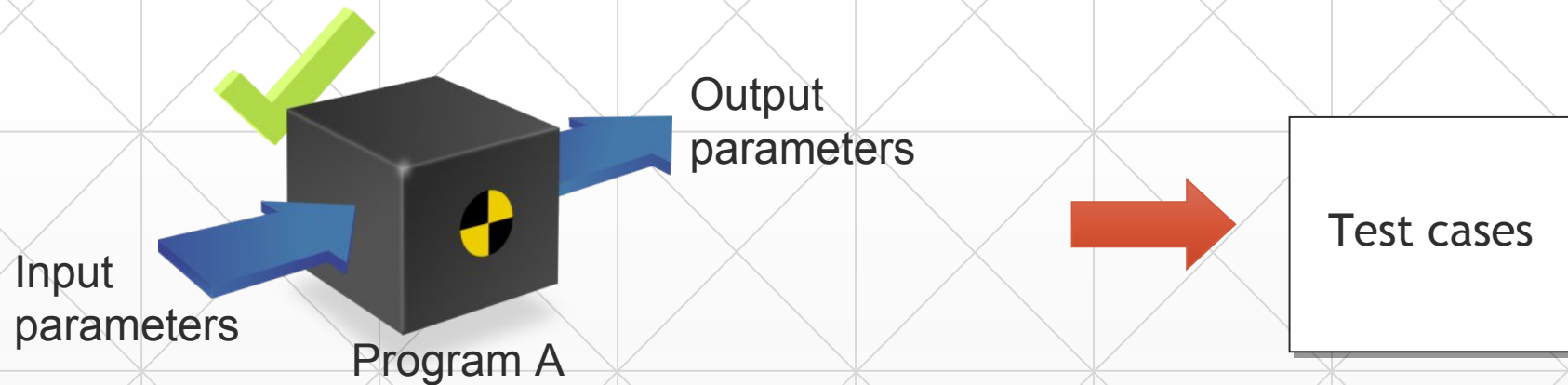
[illegible]

Testing Techniques - Release Testing

1. Black Box Testing

- **black-box testing**

- relies on the **specification** of the system or component, which is being tested to derive test cases.
- the system behaviour can only be determined by studying its inputs and the related outputs.



Introduction - Test Data

- 2 different sources of test data: -
 - live data
 - artificial data

Introduction

Test Data - Live Data



Extracted from DB

Key in data from
normal activities

Difficult to get

Introduction

Test Data - Live Data

- Live test data are those that are **actually extracted from organization files**.
- After a system is partially constructed, programmers or analysts often ask users to key in a set of data from their normal activities
- However, it is **difficult** to obtain live data in **sufficient** amounts to conduct extensive testing

Introduction

Test Data - Artificial Data



Generated to test all combinations of formats & values

Data generating utility program

Generated by person other than developer

Introduction

Test Data - Artificial Data

- Artificial test data are **created solely for test** purposes, since they can be **generated** to test **all combinations** of formats and values
- These data can be quickly prepared by a **data-generating utility program**
- The most effective test program use artificial test data generated by persons other than those who wrote the programs

Testing Techniques - Performance/Stress Test



Performance/Stress Test

- ✓ Beyond limits
- ✓ Over-load situation



Testing Techniques - Performance/Stress Test



Banking system

→ ALL tellers sign in the terminal

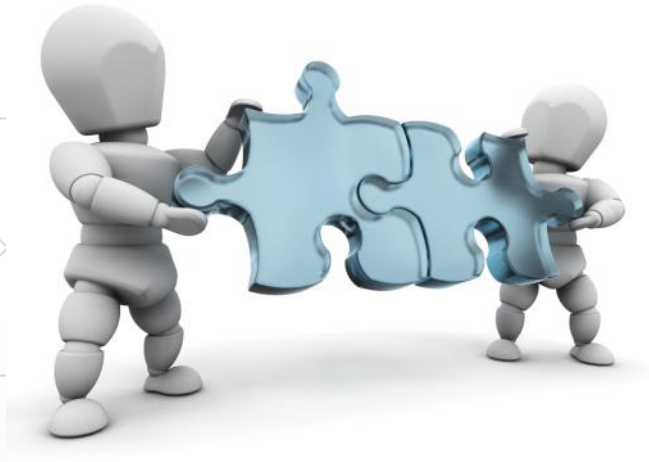
Performance/Stress Test

Testing Techniques - Performance/Stress Test

- **Performance/Stress testing:**
 - Once a system has been completely integrated, test the system to ensure it can process the intended load
 - stressing the system by going beyond its specified limits and hence testing how well the system can cope with **over-load situation**.
 - E.g. in a banking system, analysts want to know what will happen if all tellers sign on at their terminals at the same time before the start of the business day, will the system able to handle this situation?



Testing Techniques - Others

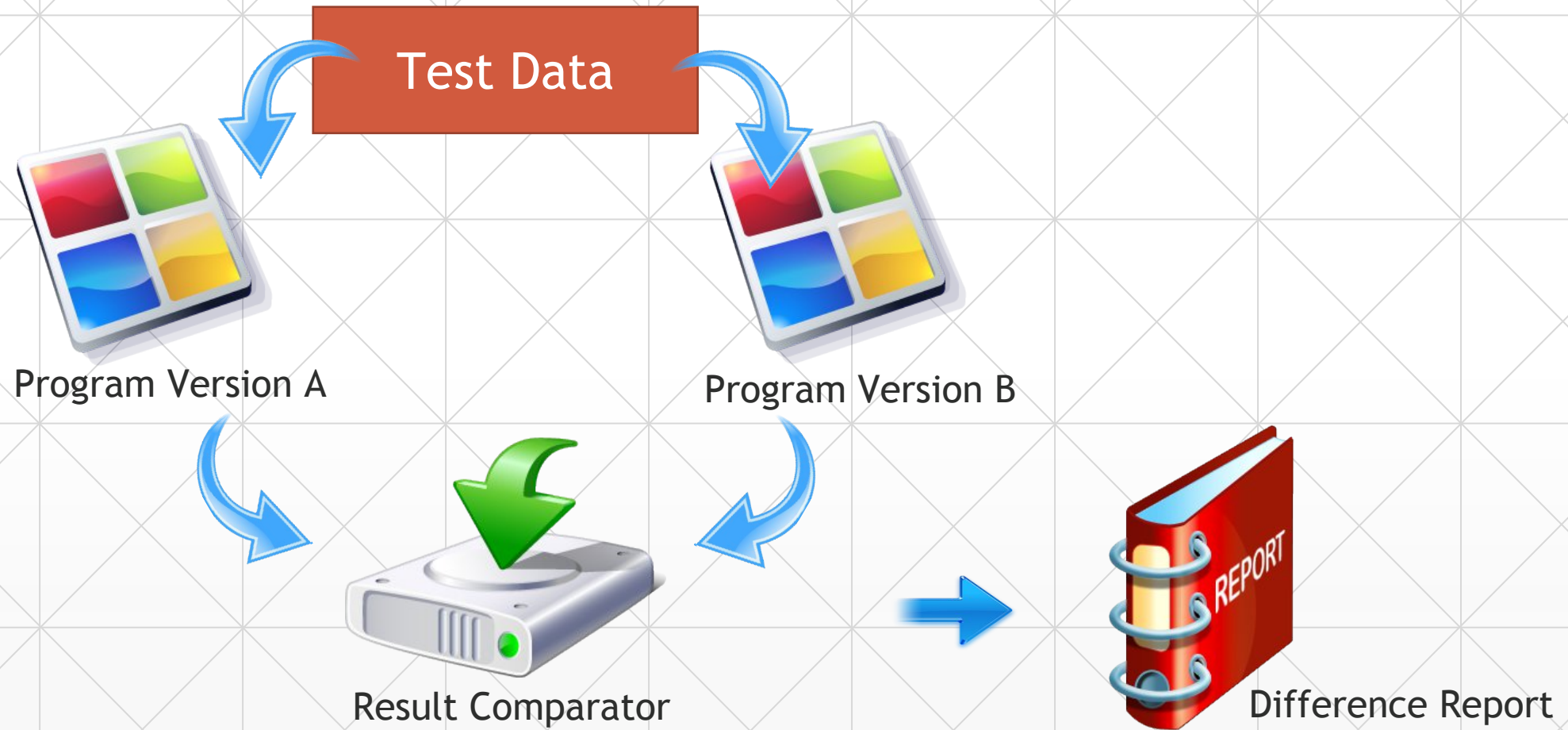


Back-to-back testing



Version 1 vs. version 2

Testing Techniques - Others



Testing Techniques - Others

Back-to-back testing

- Which is used when versions (e.g. version 1 and version 2) of a system are available. The systems are tested together and their outputs are compared.
- Differences between the test results highlight potential problems

Testing Strategy

- A testing strategy is a **general approach to the testing process**, rather than a method of devising (work out of) particular system or component tests.



Testing Strategy

Two Techniques:

- ✓ Proactive
- ✓ Reactive



Proactive Approach:

- The proactive approach involves taking anticipatory actions to prevent issues or problems from occurring in the first place.
- It focuses on planning, preparation, and strategic thinking to mitigate risks and address potential challenges before they escalate.

Scenario

- **Proactive Planning: Strategy:** Implementing automated testing processes, code reviews, and static code analysis tools as part of the development workflow to catch bugs early in the development cycle.
- **Preventive Measures:** Conducting thorough requirements analysis and creating detailed test plans to identify potential issues before coding begins.

Reactive Approach:

- The reactive approach involves responding to issues or problems after they have occurred. It focuses on addressing the immediate consequences of an event rather than preventing it from happening.

scenario

- **Issue Resolution: Bug Tracking:** Using a bug tracking system to log and prioritize reported issues, assigning them to developers for resolution.
- **Root Cause Analysis:** Conducting post-mortem analysis to understand why bugs occurred and taking corrective actions to prevent similar issues in the future.

Test Case Design and Planning

Test Case Design & Planning

- Test planning is concerned with setting out **standards** for the testing process rather than describing product tests
- In order to yield a useful testing result, a test plan should be produced. The **contents of the test plan** may include: -
 - The testing process
 - Requirement traceability
 - Tested items
 - Testing schedule
 - Testing recording procedures(example)
 - Hardware & software requirements
 - Constraints



Test Case Design & Planning

Test Plan Content



**Test
Plan**

1. The testing process

- ✓ Unit Testing
- ✓ Module Testing
- ✓ Sub-system Testing
- ✓ System Testing
- ✓ Acceptance Testing

Test Case Design & Planning

Test Plan Content



**Test
Plan**

1. The testing process
2. Requirement

- ✓ Unit Testing - SRS item x
- ✓ Module Testing - SRS item y

Test Case Design & Planning

Test Plan Content



**Test
Plan**

1. The testing process
2. Requirement traceability
3. Test cases

- ✓ Program Modules
- ✓ User Procedures
- ✓ Operator Procedures

Test Case Design & Planning

Test Plan Content



**Test
Plan**

1. The testing process
2. Requirement traceability
3. Tested items
4. Test Case 1.1.1

✓ Gantt Chart on the schedule to test each item

Test Case Design & Planning

Test Plan Content



**Test
Plan**

1. The testing process
2. Requirement traceability
3. Tested items
4. Testing Schedule
5. Testing recording procedures(example)

Test Case Design & Planning

Test Plan Content

Program Name:					
Test Date:			Tester:		
No	Objective/Test Cases	Test Data	Expected Results	Actual Results	Remarks/ Comments
	To generate a report to list selected month's sales	June to September Sales data	Monthly sales report		

Test Case Design & Planning

Test Plan Content



**Test
Plan**

1. The testing process
2. Requirement traceability
3. Tested items
4. Testing Schedule
5. Testing recording procedures(example)
6. Hardware and Software

✓ E.g. Samsung Galaxy Alpha, OS Android, 2G GSM, 3G UMT

Test Case Design & Planning

Test Plan Content



**Test
Plan**

1. The testing process
2. Requirement traceability
3. Tested items
4. Testing Schedule
5. Testing recording procedures(example)
6. Hardware & software requirements
7. **Configuration**
 - ✓ test item availability
 - ✓ test resource availability
 - ✓ time constraints

Test Case Design & Planning



- Three approaches for test case design:

1. Requirements-based testing

- Test cases are designed to test the system **requirements** are being **met**

2. Partition testing

- Identify **input and output partitions** and design tests so that the system executes inputs from all partitions and generates outputs in all partitions

3. Structural testing

- Use **knowledge** of the program's structure to design tests that exercise **all parts** of the program
 - Should try to execute each program statement at least once
-

Testing Principles/Guidelines



*What are the
guidelines to
improve software
testing?*

Testing Principles/Guidelines

- **All tests should be traceable to customer requirements** – Most severe defects → If the program fail to meet its requirements.
- **Tests should be planned long before testing begins** – Test plan can begin as soon as the req. model is complete. Test cases can begin as soon as design model has been confirmed.
- **The Pareto principles applies to software testing** – 80% of all errors found during testing will likely traceable to 20% of all program. So, isolate suspect components & thoroughly test them.
- **Testing should begin “in small” and progress toward testing “in the large”** – Firstly, the testing focuses on individual component, then, focus shifts to find errors in integrated components and finally in the entire system.

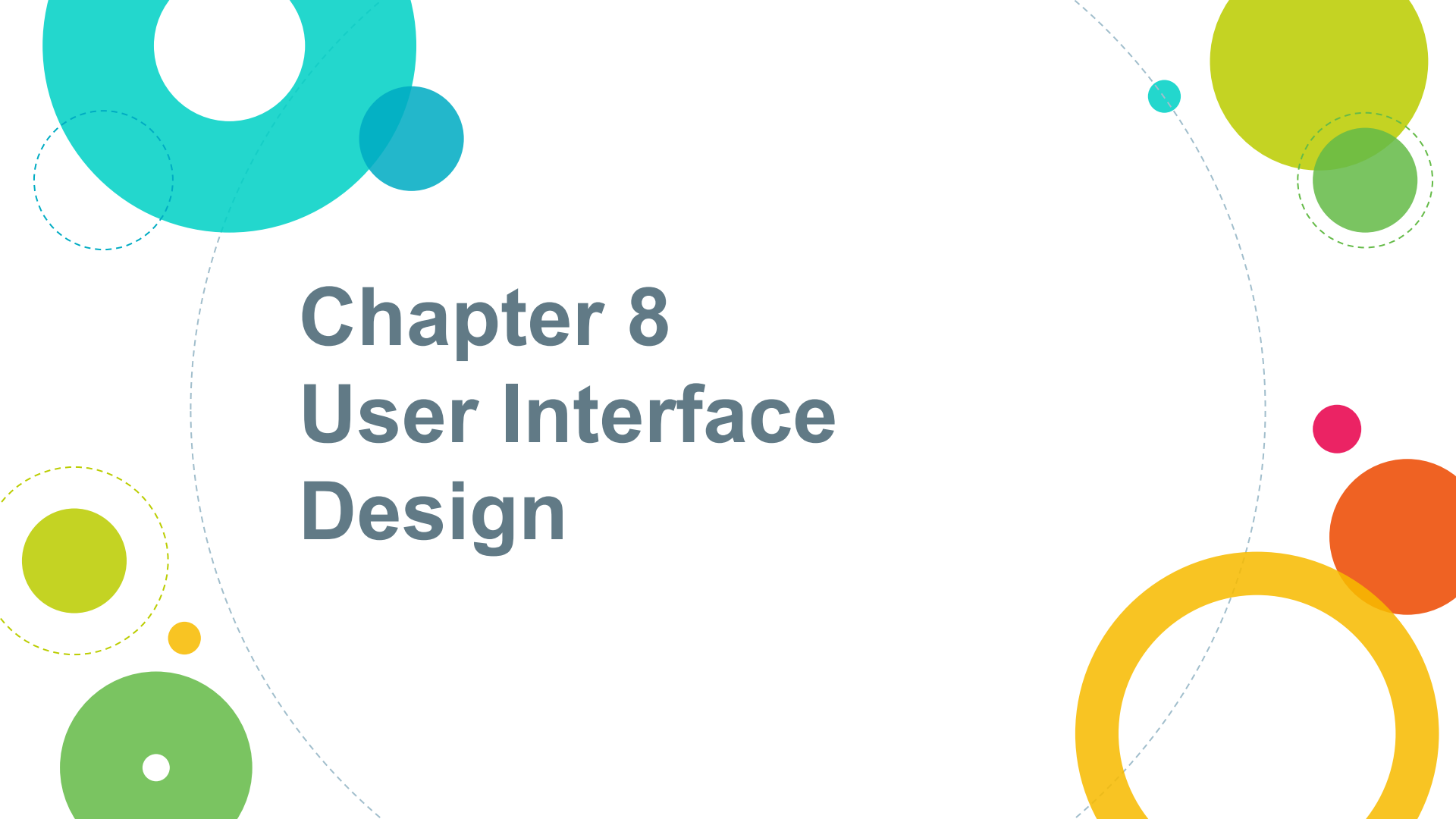
Testing Principles/Guidelines

- **Exhaustive testing is not possible** – impossible to test every combination of paths during testing because the numbers of possible paths could be very large.
- **To be most effective, testing should be conducted by an independent third party** – developers may selectively test the parts that they think have defects.

Lesson Objectives

- Software testing fundamentals
- Testing techniques and strategies
- Test case design and planning
- Testing guidelines/ principles

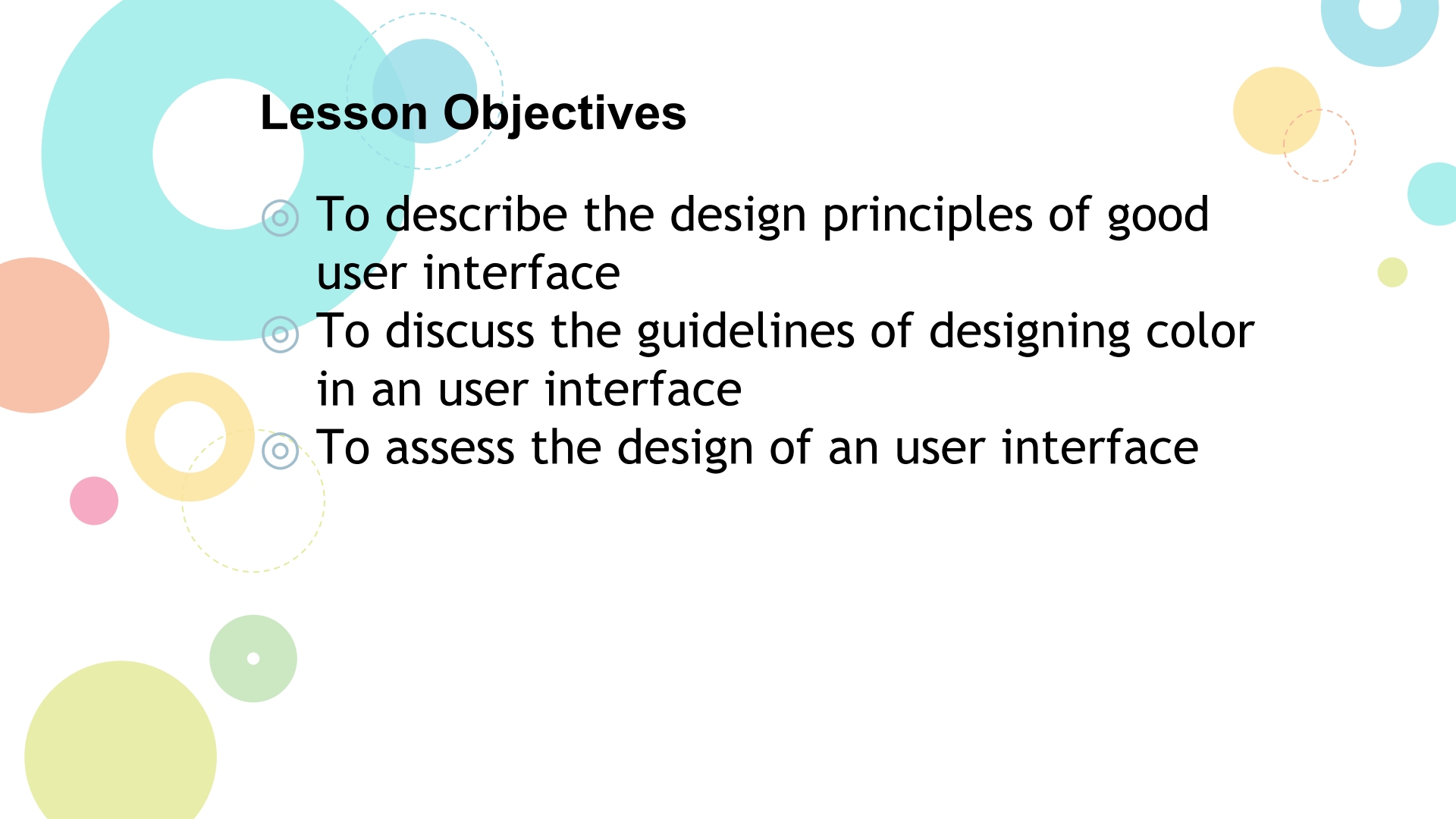


The background features a light gray background with several abstract elements. In the top left, there is a large teal circle with a white center, a smaller solid teal circle, and a dashed teal circle. In the top right, there is a large lime green circle, a smaller solid lime green circle, and a dashed lime green circle. In the bottom left, there is a large green circle with a white center, a smaller solid green circle, and a dashed green circle. In the bottom right, there is a large yellow circle, a smaller solid yellow circle, and a dashed yellow circle. A large, faint dashed gray circle is centered behind the text.

Chapter 8

User Interface

Design



Lesson Objectives

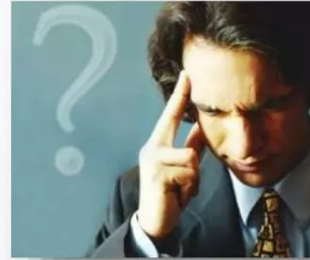
- ① To describe the design principles of good user interface
- ② To discuss the guidelines of designing color in an user interface
- ③ To assess the design of an user interface

The user interface

- ⑦ User interfaces should be designed to match the skills, experience and expectations of its anticipated users.
- ⑦ System users often judge a system by its interface rather than its functionality.
- ⑦ A poorly designed interface can cause a user to make errors.
- ⑦ Poor user interface design is the reason why so many software systems are never used.

Human factors in interface design

- Limited short-term memory
 - People can instantaneously remember about 7 items of information. If you present more than this, they are more liable to make mistakes.
- People make mistakes
 - When people make mistakes and systems go wrong, inappropriate alarms and messages can increase stress and hence the likelihood of more mistakes.
- People are different
 - People have a wide range of physical capabilities. Designers should not just design for their own capabilities.
- People have different interaction preferences
 - Some like pictures, some like text.



Benefits of good design (User view)

Better User Experience (UX)

A well-designed application is intuitive and easy to use, providing a seamless experience. The interface is clear, responsive, and organised, which reduces user frustration and increases satisfaction.

Faster Performance

Good design ensures that the software is optimised for speed, resulting in quicker load times, faster responses to user inputs, and a smoother overall experience.

Reliability and Stability

Users benefit from a stable and bug-free application that performs consistently over time. Good design minimises crashes, errors, and unexpected behaviour, leading to a more reliable system.

Customizability

Users may enjoy personalisation options, such as adjusting the interface, settings, or functionality according to their preferences, which is made easier through a well-structured design.

GUI characteristics

Characteristic	Description
Windows	Multiple windows allow different information to be displayed simultaneously on the user's screen.
Icons	Icons represent different types of information. On some systems, icons represent files; on others, icons represent processes.
Menus	Commands are selected from a menu rather than typed in a command language.
Pointing	A pointing device such as a mouse is used for selecting choices from a menu or indicating items of interest in a window.
Graphics	Graphical elements can be mixed with text on the same display.

Issues in UI design

1. How should information from the user be provided to the computer system?

Problem: The way **users interact with the system** (input methods such as typing, clicking, voice commands, etc.) needs to be designed for clarity, ease of use, and efficiency. Poor input design can lead to frustration, errors, and inefficiency.

Impact: If the user cannot easily provide information or input to the system, the interaction becomes cumbersome, and it may lead to a bad user experience (UX).

2. How should information from the computer system be presented to the user?

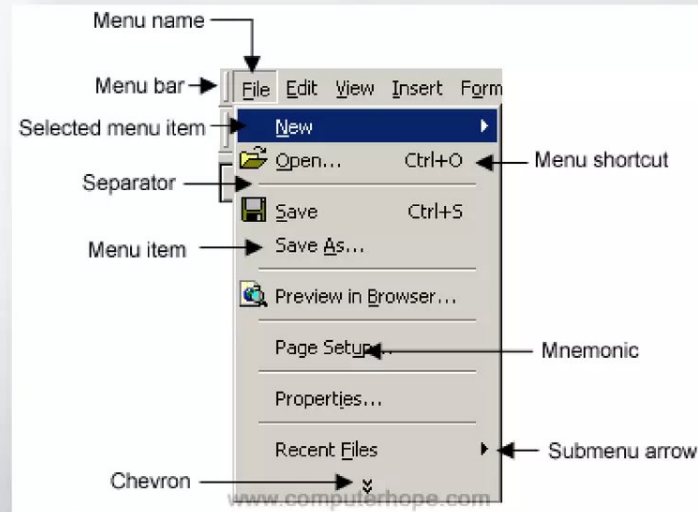
Problem: The presentation of data and information (such as feedback, results, or status updates) needs to be clear, structured, and easy to understand. Overloading the user with too much information at once or presenting it confusingly can reduce usability.

Impact: A poor design in information presentation can overwhelm the user, make important data harder to find, and cause the user to abandon the interface or make mistakes.

Example: If a system presents too much data at once without proper categorisation or visual hierarchy, the user may feel overwhelmed and unable to find the information they need.

Interaction styles

- Direct manipulation
- Menu selection
- Form fill-in
- Command language



Interaction styles

Interaction style	Main advantages	Main disadvantages	Application examples
Direct manipulation	Fast and intuitive interaction Easy to learn	May be hard to implement Only suitable where there is a visual metaphor for tasks and objects	Video games CAD systems
Menu selection	Avoids user error Little typing required	Slow for experienced users Can become complex if many menu options	Most general-purpose systems
Form fill-in	Simple data entry Easy to learn	Takes up a lot of screen space	Stock control, Personal loan processing
Command language	Powerful and flexible	Hard to learn Poor error management	Operating systems, Library information retrieval systems

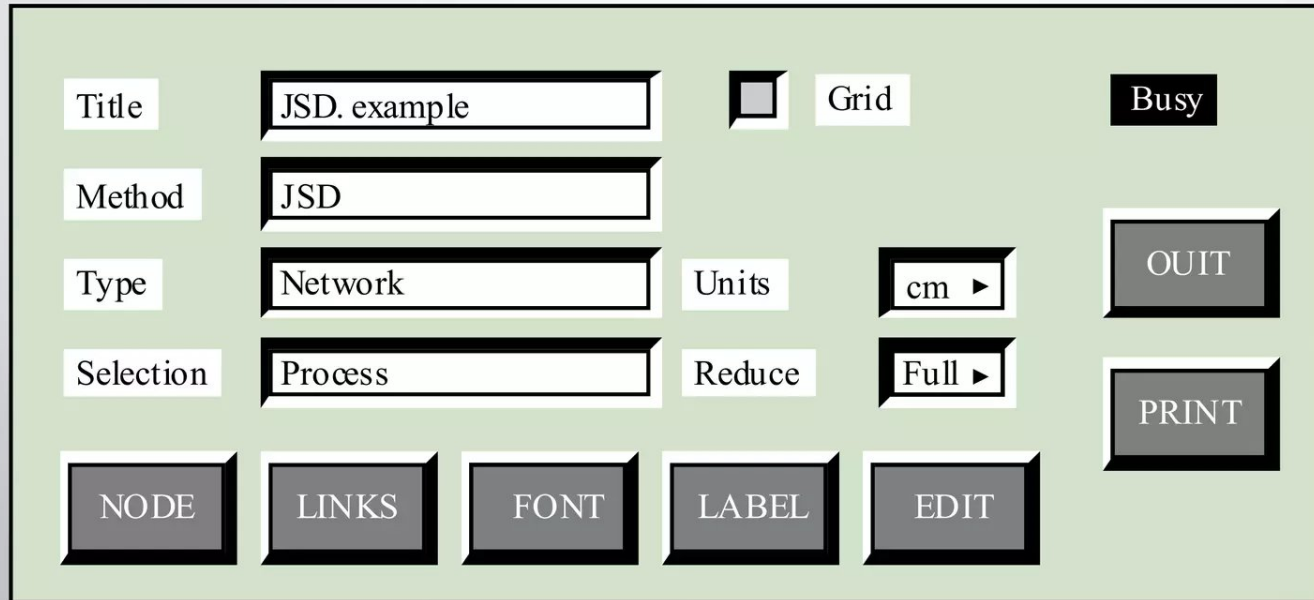
Direct manipulation advantages

- Users feel in control of the computer and are less likely to be intimidated by it
- User learning time is relatively short
- Users get immediate feedback on their actions so mistakes can be quickly detected and corrected

Direct manipulation problems

- The derivation of an appropriate information space model can be very difficult
- Given that users have a large information space, what facilities for navigating around that space should be provided?
- Direct manipulation interfaces can be complex to program and make heavy demands on the computer system

Control panel interface

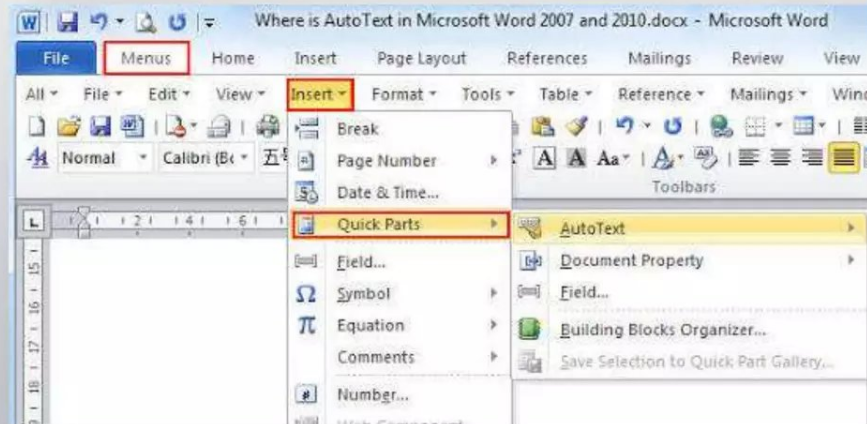


The control panel interface is a light green rectangular window with a black border. It contains various input fields, buttons, and a status indicator. On the left side, there are four labels: 'Title', 'Method', 'Type', and 'Selection', each followed by a text input field. The 'Title' field contains 'JSD. example', 'Method' contains 'JSD', 'Type' contains 'Network', and 'Selection' contains 'Process'. To the right of these fields are two checkboxes: 'Grid' (unchecked) and 'Units' (checked). Below the 'Units' checkbox is a 'Reduce' label followed by a dropdown menu showing 'Full' with a right-pointing arrow. On the far right, there is a 'Busy' status indicator (a black rectangle with white text) and two large buttons labeled 'QUIT' and 'PRINT'. At the bottom of the panel, there are five buttons labeled 'NODE', 'LINKS', 'FONT', 'LABEL', and 'EDIT'.

Title	JSD. example	☐	Grid	Busy
Method	JSD			
Type	Network	Units	cm ▶	QUIT
Selection	Process	Reduce	Full ▶	PRINT
NODE LINKS FONT LABEL EDIT				

Menu systems

- Users make a selection from a list of possibilities presented to them by the system
- The selection may be made by pointing and clicking with a mouse, using cursor keys or by typing the name of the selection
- May make use of simple-to-use terminals such as touchscreens



Advantages of menu systems

- Users need not remember command names as they are always presented with a list of valid commands
- Typing effort is minimal
- User errors are trapped by the interface
- Context-dependent help can be provided. The user's context is indicated by the current menu selection

Problems with menu systems

- Actions which involve logical conjunction (and) or disjunction (or) are awkward to represent
- Menu systems are best suited to presenting a small number of choices. If there are many choices, some menu structuring facility must be used
- Experienced users find menus slower than command language

Form-based interface

NEW BOOK			
Title		ISBN	
Author		Price	
Publisher		Publication date	
Edition		Number of copies	
Classification		Loan status	
Date of purchase		Order status	

Command interfaces

- User types commands to give instructions to the system e.g. UNIX
- May be implemented using cheap terminals.
- Easy to process using compiler techniques
- Commands of arbitrary complexity can be created by command combination
- Concise interfaces requiring minimal typing can be created



```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows XP [Version 5.1.2600.1]
(C) Copyright 1985-2001 Microsoft Corp.

C:\Documents and Settings\user>ipconfig

Windows IP Configuration

Ethernet adapter Local Area Connection:

    Connection-specific DNS Suffix  . : 
    IP Address. . . . . : 192.168.1.3
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.1.1

C:\Documents and Settings\user>
```

computer-realm.net

Problems with command interfaces

- Users have to learn and remember a command language. Command interfaces are therefore unsuitable for occasional users
- Users make errors in command. An error detection and recovery system is required
- System interaction is through a keyboard so typing ability is required

Command languages

- Often preferred by experienced users because they allow for faster interaction with the system
- Not suitable for casual or inexperienced users
- May be provided as an alternative to menu commands (keyboard shortcuts). In some cases, a command language interface and a menu-based interface are supported at the same time

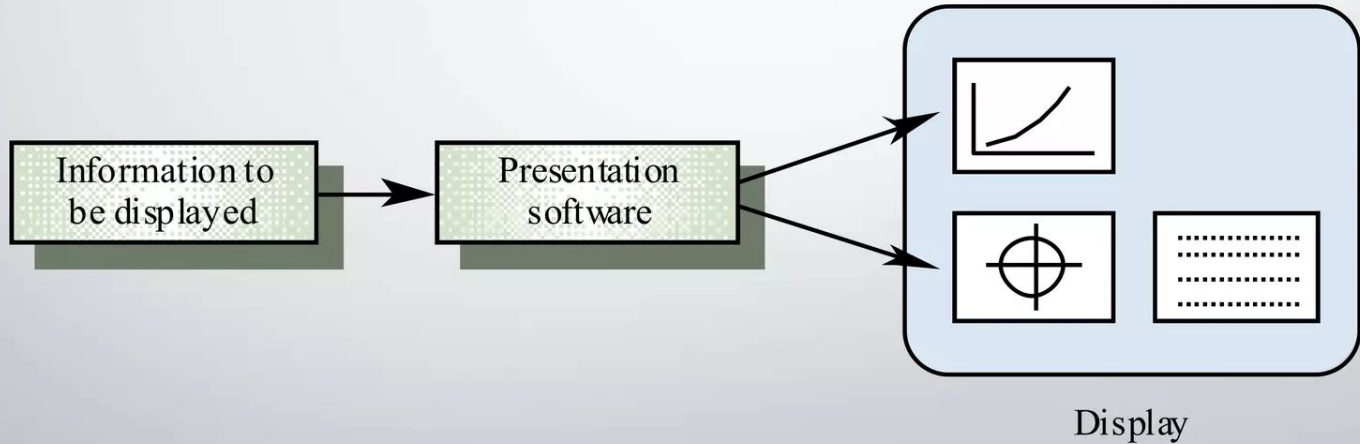
Natural language interfaces

- The user types a command in a natural language. Generally, the vocabulary is limited and these systems are confined to specific application domains (e.g. timetable enquiries)
- NL processing technology is now good enough to make these interfaces effective for casual users but experienced users find that they require too much typing

Information presentation

- Information presentation is concerned with presenting system information to system users
- The information may be presented directly (e.g. text in a word processor) or may be transformed in some way for presentation (e.g. in some graphical form)
- The Model-View-Controller approach is a way of supporting multiple presentations of data

Information presentation



Information presentation

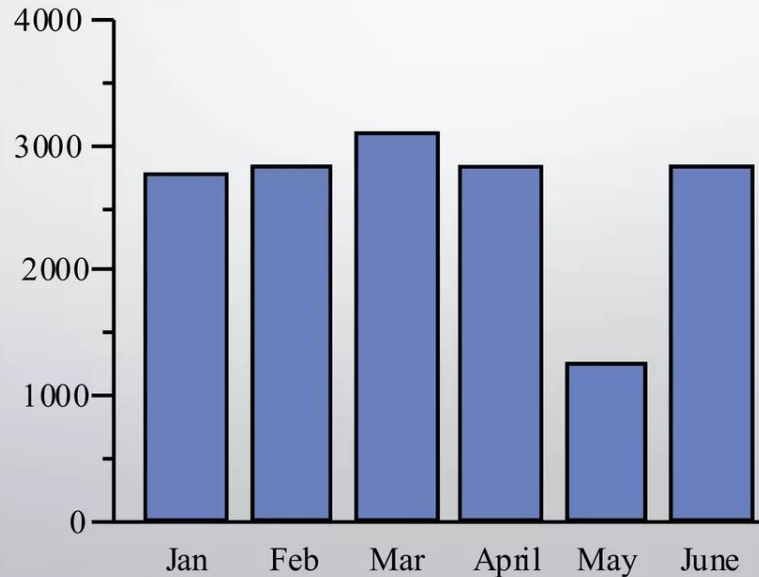
- Static information
 - Initialised at the beginning of a session. It does not change during the session
 - May be either numeric or textual
- Dynamic information
 - Changes during a session and the changes must be communicated to the system user
 - May be either numeric or textual

Information display factors

- Is the user interested in precise information or data relationships?
- How quickly do information values change?
Must the change be indicated immediately?
- Must the user take some action in response to a change?
- Does the user need to interact with the displayed info via a direct manipulation interface?
- Is the information textual or numeric? Are relative values important?

Alternative information presentations

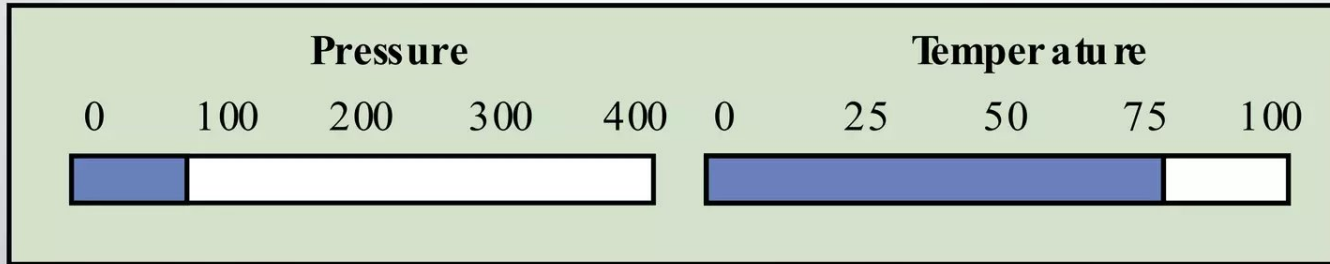
Jan	Feb	Mar	April	May	June
2842	2851	3164	2789	1273	2835



Analogue vs. digital presentation

- Digital presentation
 - Compact - takes up little screen space
 - Precise values can be communicated
- Analogue presentation
 - Easier to get an 'at a glance' impression of a value
 - Possible to show relative values
 - Easier to see exceptional data values

Displaying relative values



Data visualisation

- Concerned with techniques for displaying large amounts of information
- Visualisation can reveal relationships between entities and trends in the data

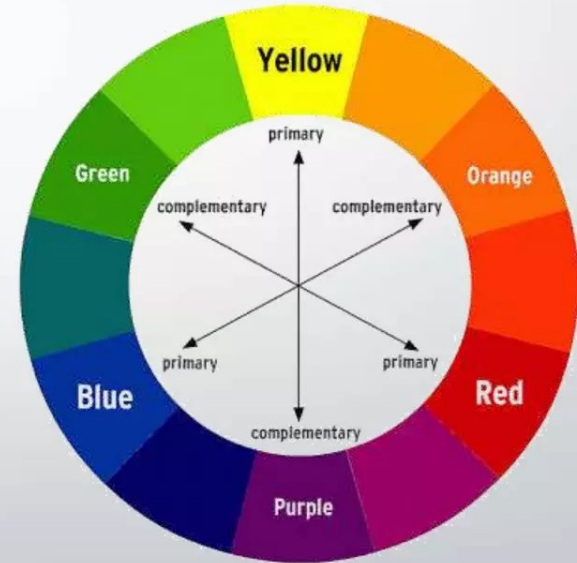


Colour displays

- Colour adds an extra dimension to an interface and can help the user understand complex information structures
- Can be used to highlight exceptional events
- Common mistakes in the use of colour in interface design include over-use of colour in the display

Colour use guidelines

- Don't use too many colours
- Use colour coding to support user tasks
- Allow users to control colour coding
- Design for monochrome then add colour
- Use colour coding consistently
- Avoid colour pairings which clash
- Use colour change to show status change
- Be aware that colour displays are usually lower resolution



The Use of Colors – Errors in Using Colors



Color is used to communicate meaning



Color blind?



Different perceptions?

The Use of Colors – Errors in Using Colors



Too many colors or colors too bright



Not comfortable



Confusion

The Use of Colors – Errors in Using Colors

- Common errors made by designers when incorporating color in a user interface are: -
 - Color is used to communicate meaning**
 - Color-blind may misinterpret the color
 - Human color perceptions are different
 - Too many colors or the colors are too bright in the display**
 - Some colors cannot viewed comfortably for long
 - Cause confusion if colors are used inconsistently

Guidelines when Using Color



Consistent



Danger → RED

Guidelines when Using Color



Color pairing

Welcome to TARUC

Welcome to TARUC

Welcome to TARUC

Design Factors in Message Wording

- ✓ Context
- ✓ Experience
- ✓ Skill level
- ✓ Style
- ✓ Culture



Design factors in message wording

Context	The user guidance system should be aware of what the user is doing and should adjust the output message to the current context.
Experience	As users become familiar with a system they become irritated by long 'meaningful' messages. However, beginners find it difficult to understand short terse statements of the problem. The user guidance system should provide both types of message and allow the user to control message conciseness.
Skill level	Messages should be tailored to the user's skills as well as their experience. Messages for the different classes of user may be expressed in different ways depending on the terminology which is familiar to the reader.
Style	Messages should be positive rather than negative. They should use the active rather than the passive mode of address. They should never be insulting or try to be funny.
Culture	Wherever possible, the designer of messages should be familiar with the culture of the country where the system is sold. There are distinct cultural differences between Europe, Asia and America. A suitable message for one culture might be unacceptable in another.

Error messages

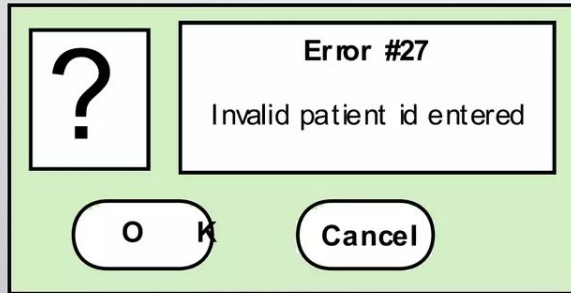
- Error message design is critically important. Poor error messages can mean that a user rejects rather than accepts a system
- Messages should be polite, concise, consistent and constructive
- The background and experience of users should be the determining factor in message design



The image shows a screenshot of a Google sign-in interface. At the top left is the text "Sign in" and at the top right is the "Google" logo. Below the logo is the "Username" label followed by a text input field containing the email address "helwerewerwetrew42342@gmail.com". Below the username field is the "Password" label followed by an empty password input field. A red error message is displayed below the password field: "The username or password you entered is incorrect." followed by a red question mark icon. At the bottom left is a blue "Sign in" button, and to its right is a checkbox labeled "Stay signed in". At the very bottom is a link that says "Can't access your account?".

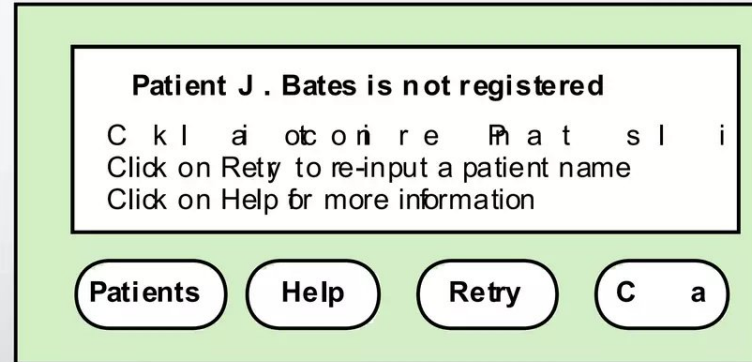
System and user-oriented error messages

System-oriented error message



A system-oriented error message dialog box with a light green background. It features a white square icon with a large black question mark on the left. To its right is a white rectangular box containing the text "Error #27" in bold and "Invalid patient id entered" below it. At the bottom, there are two white rounded buttons: "OK" and "Cancel".

User-oriented error message



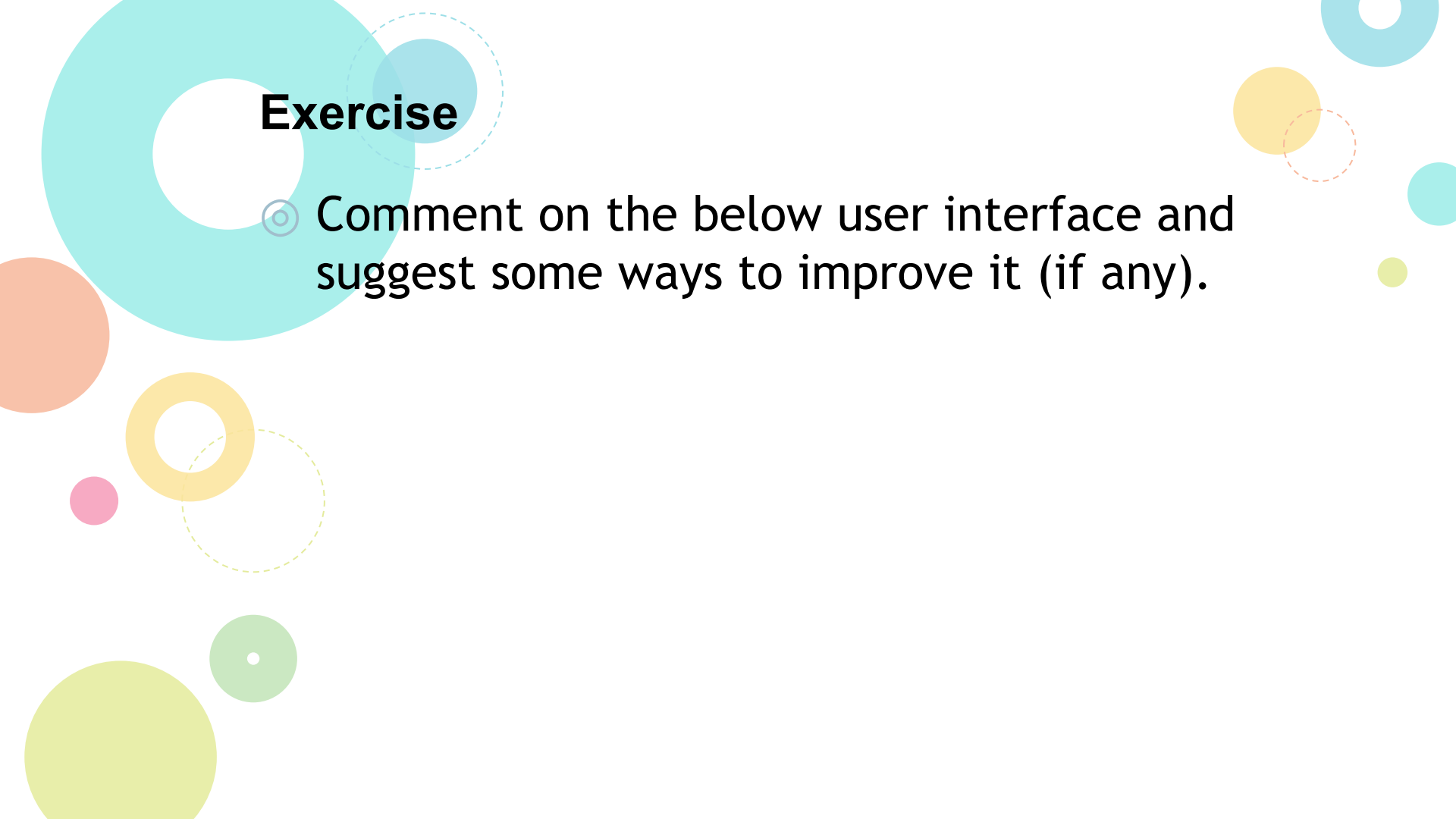
A user-oriented error message dialog box with a light green background. It contains a white rectangular box with the text "Patient J . Bates is not registered" in bold, followed by "Click on Retry to re-input a patient name" and "Click on Help for more information". Below this box are four white rounded buttons: "Patients", "Help", "Retry", and "Cancel".

Support Documentation for Users

- ① **A functional description**
 - description of services (system evaluators)
- ② **An introductory manual**
 - Getting started (beginner)
- ③ **A system reference manual**
 - facility description (experienced users)

Support Documentation for Users

- ⦿ **A system installation manual**
 - how to install system (system administrator)
- ⦿ **A system administrator's manual**
 - operation & maintenance (system administrator)



Exercise

- ◉ Comment on the below user interface and suggest some ways to improve it (if any).

Student Registration Form

[Main](#)[Add](#)[Delete](#)[Search](#)[Print](#)

Gender

D.O.B

Name

Age

I/C No.

Address

Guidance Name

SPM Result

Guidance I/C

B. Malaysia

Course(1st choice)

English

Course(2nd choice)

Mathematics

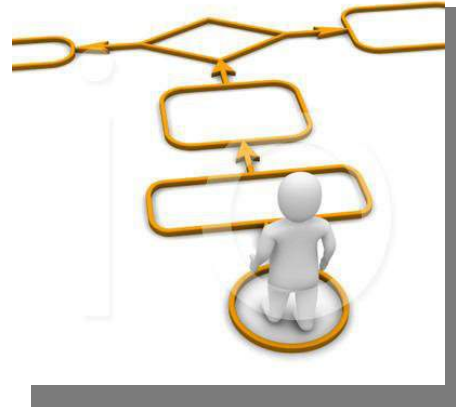
3 Golden Rules for a Good UI Design

1. **Place the user in control** - People do not like to be controlled by the machine and they like to know the **status** of the operation/ system.



3 Golden Rules for a Good UI Design

2. **Reduce the user's memory load** - people have limited short-term memory & they make mistakes when they handle too much info.



3 Golden Rules for a Good UI Design

3. **Make the interface consistent** - easy to learn and the knowledge learnt in one command or application is applicable in other parts of the system.



UI Design Principle



- ✓ Needs
- ✓ Experience
- ✓ Capabilities

UI Design Principle



- * Terms & concepts
- * Library system

1. User Familiarity

UI Design Principle



* Activate operation
in the same way

2. Consistency

UI Design Principle



* E.g. shut down immediately

3. Minimal Surprises

UI Design Principle



* E.g. undo

4. Recoverability

UI Design Principle



5. User Guidance

- * Help
- * Feedback on errors

UI Design Principle



- * Experience user
vs. beginner
- * E.g. step-by-step
installation

6. User Diversity

UI Design Principles

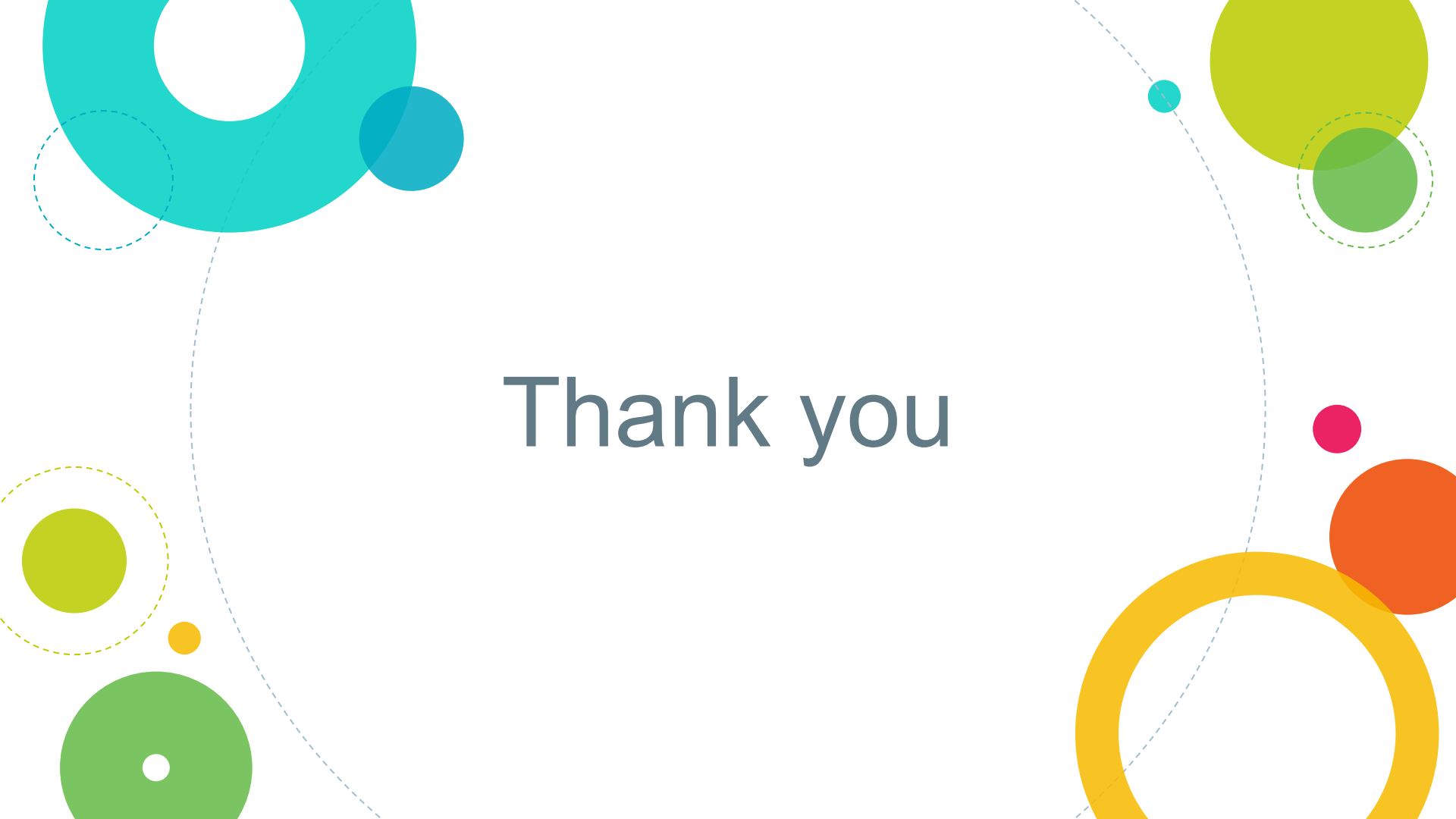
- ② User interface design must take into account the needs, experience and capabilities of the system user.
- ② **User familiarity** - should use the terms and concepts which are drawn from the experience of the users. E.g. Library system - operations: book in, book out, renew book etc.
- ② **Consistency** - wherever possible, operations should be activated in the same way
- ② **Minimal surprise** - users should never be surprised by the behaviour of a system

UI Design Principles

- **Recoverability** - should allow users to recover from errors, e.g. Undo function
- **User guidance** - provide help feature and meaningful feedback when errors occur
- **User diversity** - should provide appropriate interaction facilities for different types of system user. E.g. Experience user: very detailed step-by-step way to perform a function.

summary

Principle	Description
Use familiarity	The interface should use terms and concepts which are drawn from the experience of the people who will make most use of the system.
Consistency	The interface should be consistent in that whenever possible, comparable operations should be activated in the same way.
Minimize surprise	Users should not be surprised by the behaviour of a system.
Recoverability	The interface should include mechanisms to allow users to recover from errors.
Use guidance	The interface should provide meaningful feedback when errors occur and provide context-sensitive user assistance.
Use diversity	The interface should provide a proportionate number of failures for different types of system error.

The background features a light gray background with several abstract elements. In the top left, there is a large teal circle with a white center, a smaller solid teal circle to its right, and a dashed teal circle to its left. A dashed line curves from the top left towards the bottom left. In the bottom left, there is a large green circle with a white center, a smaller solid green circle to its right, and a dashed green circle to its left. A dashed line curves from the bottom left towards the top left. In the top right, there is a large yellow-green circle, a smaller solid yellow-green circle to its right, and a dashed yellow-green circle to its left. A dashed line curves from the top right towards the bottom right. In the bottom right, there is a large yellow circle, a smaller solid yellow circle to its right, and a dashed yellow circle to its left. A dashed line curves from the bottom right towards the top right. The text "Thank you" is centered in the middle of the image in a dark gray, sans-serif font.

Thank you

Chapter 9

REAL-TIME OPERATING SYSTEM (RTOS)

OVERVIEW

- ❖ Introduction to RTOS
- ❖ Characteristics of RTOS
- ❖ Categories of RTOS
- ❖ Time analysis
- ❖ overview of stimulus and response

WHAT IS REAL-TIME OPERATING SYSTEM?

Introduction [1]

- ❖ A real-time operating system (RTOS) is key to many embedded systems today and, provides a software platform upon which to build applications.
- ❖ Not all embedded systems, however, are designed with an RTOS.
- ❖ Some embedded systems with relatively simple hardware or a small amount of software application code might not require an RTOS.
- ❖ Many embedded systems, however, with moderate-to-large software applications require some form of scheduling, and these systems require an RTOS.

WHAT IS REAL-TIME OPERATING SYSTEM?

Real-Time Operating System Defined.

- ❖ A *Real-Time Operating System* (RTOS) is a program that schedules execution in a timely manner, manages system resources, and provides a consistent foundation for developing application code.
- ❖ A *Real-Time Operating System* is an operating system that guarantees to process events or data within a certain short amount of time

CONTINUE...

- ❖ It responds to inputs immediately(Real-Time).
- ❖ Here the task is completed within a specified time delay.
- ❖ In real life situations like controlling traffic signal or a nuclear reactor or an aircraft,
- ❖ The operating system has to respond quickly.

KEY CHARACTERISTICS OF AN RTOS

- ❖ An application's requirements define the requirements of its underlying RTOS. Some of the more common attributes are
 - Reliability,
 - Predictability,
 - Performance,
 - Compactness, and
 - Scalability.

RELIABILITY

- ❖ Real-Time Operating System must be reliable. Depending on the application, the system might need to operate for long periods without human intervention.
- ❖ Different degrees of reliability may be required. For example, a digital solar-powered calculator might reset itself if it does not get enough light, yet the calculator might still be considered acceptable.

PREDICTABILITY

- ❖ Because many embedded systems are also real-time systems, meeting time requirements is key to ensuring proper operation.
- ❖ The RTOS used in this case needs to be predictable to a certain degree.
- ❖ The term deterministic describes Real-Time Operating Systems (RTOSs) with predictable behavior, in which the completion of operating system calls occurs within known timeframes.

CONTINUE....

- ❖ Developers can write simple benchmark programs to validate the determinism of an RTOS.
 - ✓ Determinism means more faster processors
- ❖ The result is based on timed responses to specific RTOS calls. In a good deterministic RTOS, the variance of the response times for each type of system call is very

PERFORMANCE

- ❖ This requirement dictates that an embedded system must perform fast enough to fulfill its timing requirements.
- ❖ Processor's performance
 - ❖ MIPS: million instructions per second
- ❖ It may be used to measure the overall performance of a system
 - ✓ The rate at which a system can generate output based on the inputs coming in
- ❖ Call-by-call method *may be* used to measure RTOS performance.
 - ✓ Produce timestamps when a system call starts and when it completes

COMPACTNESS

- To determine how compact an embedded system be
 - **Application design constraints:** *The concept of constraints in design can be defined as the practice of limiting user actions on a system.[2]*
 - **Cost constraints:** *The cost constraint refers to the budgeted amount available for the project.[3]*
- For example, a cell phone must be small, portable and low cost.
 - Limit system memory
- Limit the size of application and operating system

CONTINUE....

- To meet total system requirements, designers must understand both the static and dynamic memory consumption of the RTOS and the application that will run on it.
 - Static memory:** Static memory allocation is the allocation of memory at compile-time before the associated program is executed.
 - Dynamic memory:** Memory allocated during run-time. [4]

CONTINUE....

- To meet total system requirements, designers must understand both the static and dynamic memory consumption of the RTOS and the application that will run on it.
 - Static memory:** Static memory allocation is the allocation of memory at compile-time before the associated program is executed.
 - Dynamic memory:** Memory allocated during run-time. [4]

SCALABILITY

- RTOSs can be used in a wide variety of embedded systems, they must be able to scale up or down to meet application-specific requirements.
- Depending on how much functionality is required, an RTOS should be capable of adding or deleting modular components, including file systems and protocol stacks.

Real-time Operating system components

- Real-time clock
 - Provides information for process scheduling.
- Interrupt handler
 - Manages aperiodic requests for service.
- Scheduler
 - Chooses the next process to be run.
- Resource manager
 - Allocates memory and processor resources.
- Dispatcher
 - Starts process execution.

REAL-TIME OPERATING
SYSTEM

```
graph TD; A[REAL-TIME OPERATING SYSTEM] --> B[HARD REAL-TIME SYSTEM]; A --> C[SOFT-REAL TIME SYSTEM]
```

The diagram is a flowchart illustrating the classification of Real-Time Operating Systems. At the top, a box labeled 'REAL-TIME OPERATING SYSTEM' has a vertical arrow pointing down to a horizontal line. From this line, two arrows branch out to two separate boxes below: 'HARD REAL-TIME SYSTEM' on the left and 'SOFT-REAL TIME SYSTEM' on the right. The entire diagram is enclosed in a large rectangular frame with a blue border. A dark blue decorative element, resembling a folded corner or a tab, is located on the right side of the frame.

HARD REAL-TIME
SYSTEM

SOFT-REAL TIME
SYSTEM

Categories of RTOS

- A **soft real-time system** is a system whose operation is degraded if results are not produced according to the specified timing requirements.
- A **hard real-time system** is a system whose operation is incorrect if results are not produced according to the timing specification.

Soft and Hard Real Time tasks

- Hard real time task:
 - Task must complete before or at deadline.
 - Value of completing the task after deadline is zero.
- Software real time
 - Missing deadline incurs a penalty
 - Penalty increases as tardiness increases

Timing analysis

- The correctness of a real-time system depends not just on the correctness of its outputs but also on the time at which these outputs were produced.
- In a timing analysis, you calculate how often each process in the system must be executed to ensure that all inputs are processed and all system responses produced in a timely way.
- The results of the timing analysis are used to decide how frequently each process should execute and how these processes should be scheduled by the real-time operating system.

Factors in timing analysis

- *Deadlines*
 - The times by which stimuli must be processed and some response produced by the system.
- *Frequency*
 - The number of times per second that a process must execute so that you are confident that it can always meet its deadlines.
- *Execution time*
 - The time required to process a stimulus and produce a response.

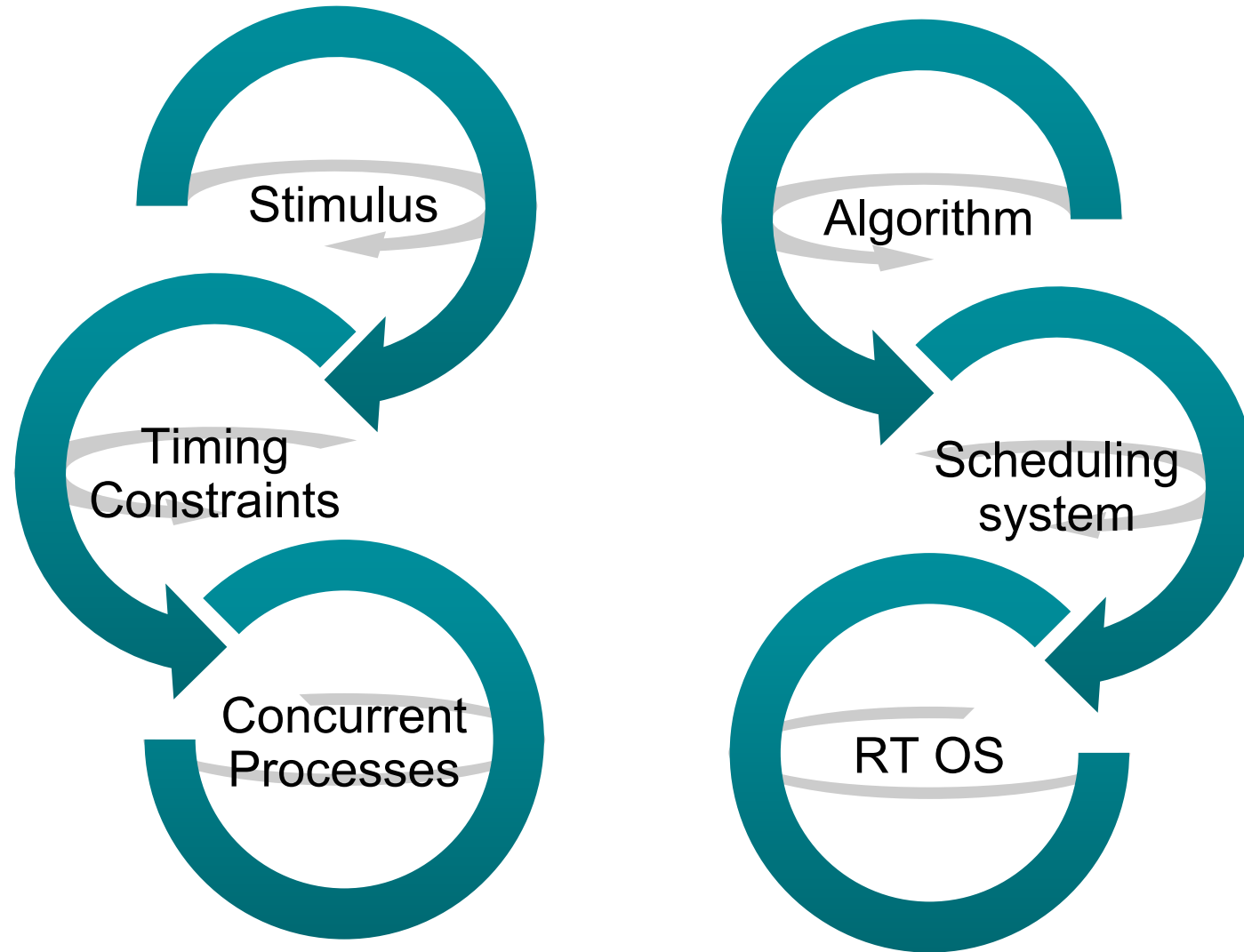
Real Time System Design

- Real time system must respond to stimulus at different time. Its architecture must therefore be organized so the **control** is transferred to the appropriate handler for that stimulus as soon as it is received.



*Which **control model** is suitable for real-time system?*

Stages of RT System Design



Stages of RT System Design

1. Identify the stimuli that the system must process and the associated responses
2. For each stimulus and associated response, identify the timing constraints which apply to both stimulus and response processing
3. Aggregate the stimulus and response processing into a number of concurrent processes.
4. For each stimulus and response, design algorithm to carry out the required computations
5. Design a scheduling system which will ensure that processes are started in time to meet their deadlines
6. Integrate the system under the control of a real-time operating system

Monitoring & Control Systems

Real-time system design process with example

1. Identify the **stimuli** that the system must process and the **associated responses**

Stimuli → Response

Example:

Stimuli (Intruder Alarm) → Response (computer room no., call to police, initiate voice synthesizer, switch on audible alarm, switch on light)

Stimuli (power failure) → Response (switch to backup power)

Monitoring & Control Systems

Real-time system design process with example

2. For each stimulus and associated response, identify the **timing constraints** which apply to both stimulus and response processing

timing
Stimuli → Response

Monitoring & Control Systems

Real-time system design process with example

3. Aggregate the stimulus and response processing into a number of **concurrent processes**

A set of Stimuli ^{timing} → Response } A set of concurrent processes

Example:

Window Sensors → Window Sensors Process

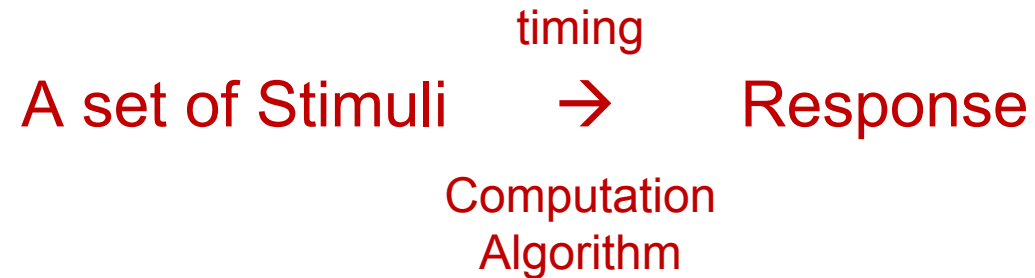
Door Sensors → Door Sensors Process

Motion Sensors → Motion Sensors Process

Monitoring & Control Systems

Real-time system design process with example

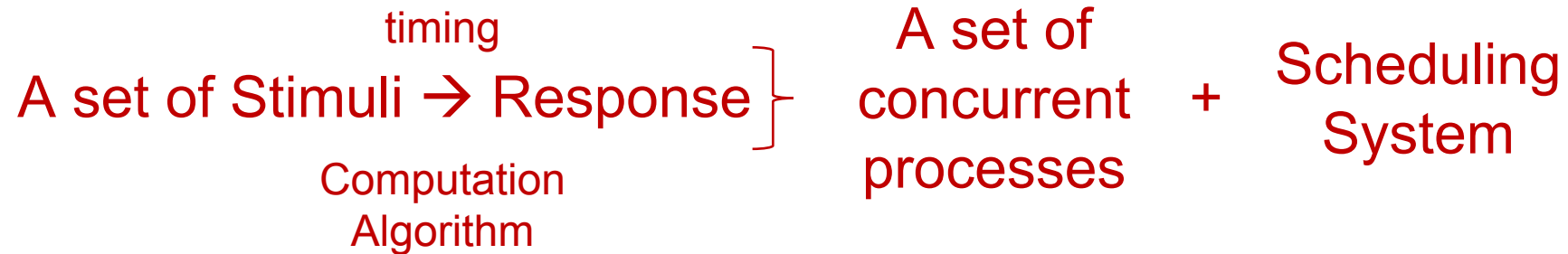
4. For each stimulus and response, **design algorithms** to carry out the required computations



Monitoring & Control Systems

Real-time system design process with example

5. Design a **scheduling system** which will ensure that processes are started in time to meet their deadlines



Monitoring & Control Systems

Real-time system design process with example

5. Design a **scheduling system** which will ensure that processes are started in time to meet their deadlines

Time (second)	Action
Every 0.5 sc	Poll door sensor
	Poll window sensor
Every 1 sc	Poll motion sensor

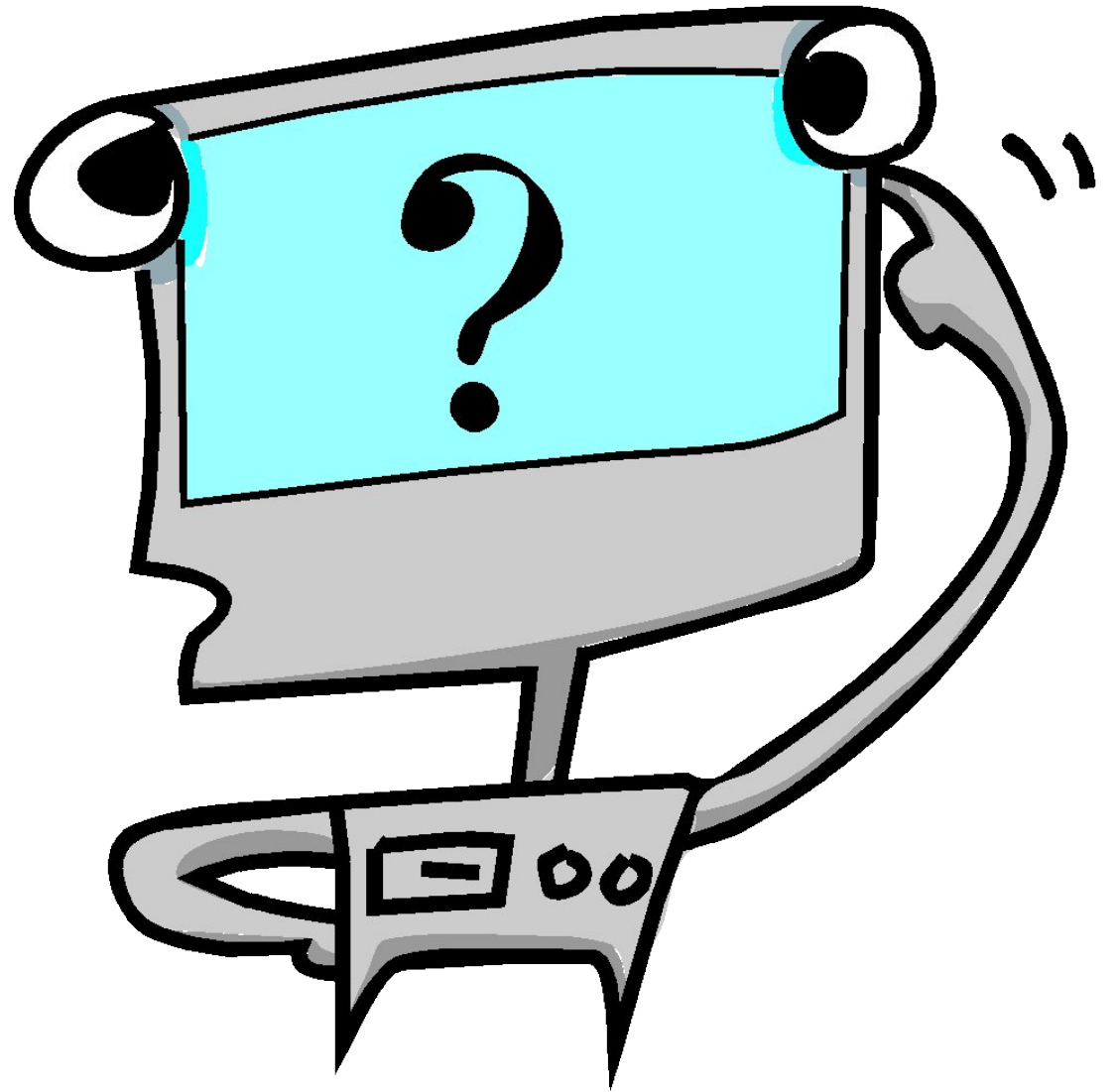
Monitoring & Control Systems

Real-time system design process with example

6. Integrate the system under the control of a real-time operating system (manages processes and resources allocation such as processor & memory in real-time system)
-

Stimuli of Real-time System

- Real-time systems have to respond to events occurring at irregular intervals. These events (or stimuli) often cause the system to move to a different state.
- Thus, may be used as a way of describing a state machine modeling real-time system.



Stimuli of Real-time System

- A **state model** of a system assumes that, at any time, the system is in one of a number of possible states.
- An example of state model of a simple microwave oven, extracted from the text book.

Stimuli of Real Time System



System Change States

Stimuli of Real Time System

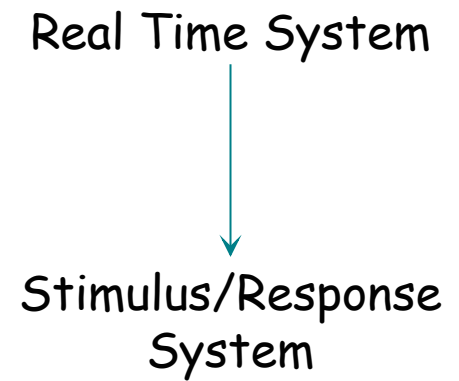


System Change States

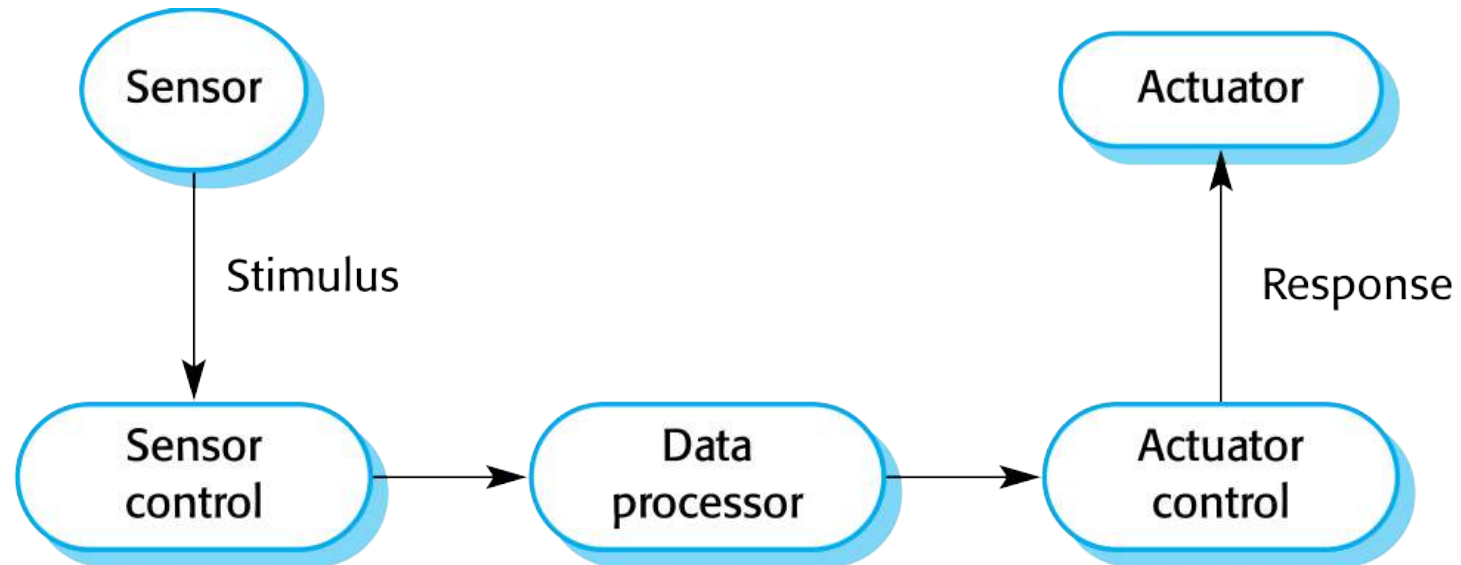


State Machine Modeling

Stimuli of Real Time System



Sensor and actuator processes



Stimuli of Real Time System



Two classes of
Stimulus

1. Periodic Stimulus
2. Aperiodic Stimulus

Stimulus/response System

- One way of looking at a real-time system is as a stimulus/response system.
- Stimuli fall into two classes: -
 - 1) Periodic stimuli
 - These occurs at predictable time intervals. E.g. the system will examine a sensor every 50ms & take action (respond) depending on the sensor value (stimulus).

Stimulus/response System

- Stimuli fall into two classes: -
 - 2) Aperiodic stimuli
 - These occurs irregularly (usually signaled using the computer's interrupt mechanism).
 - E.g. I/O interrupt in a comp system → indicating that an I/O transfer was complete & that data was available in a buffer.

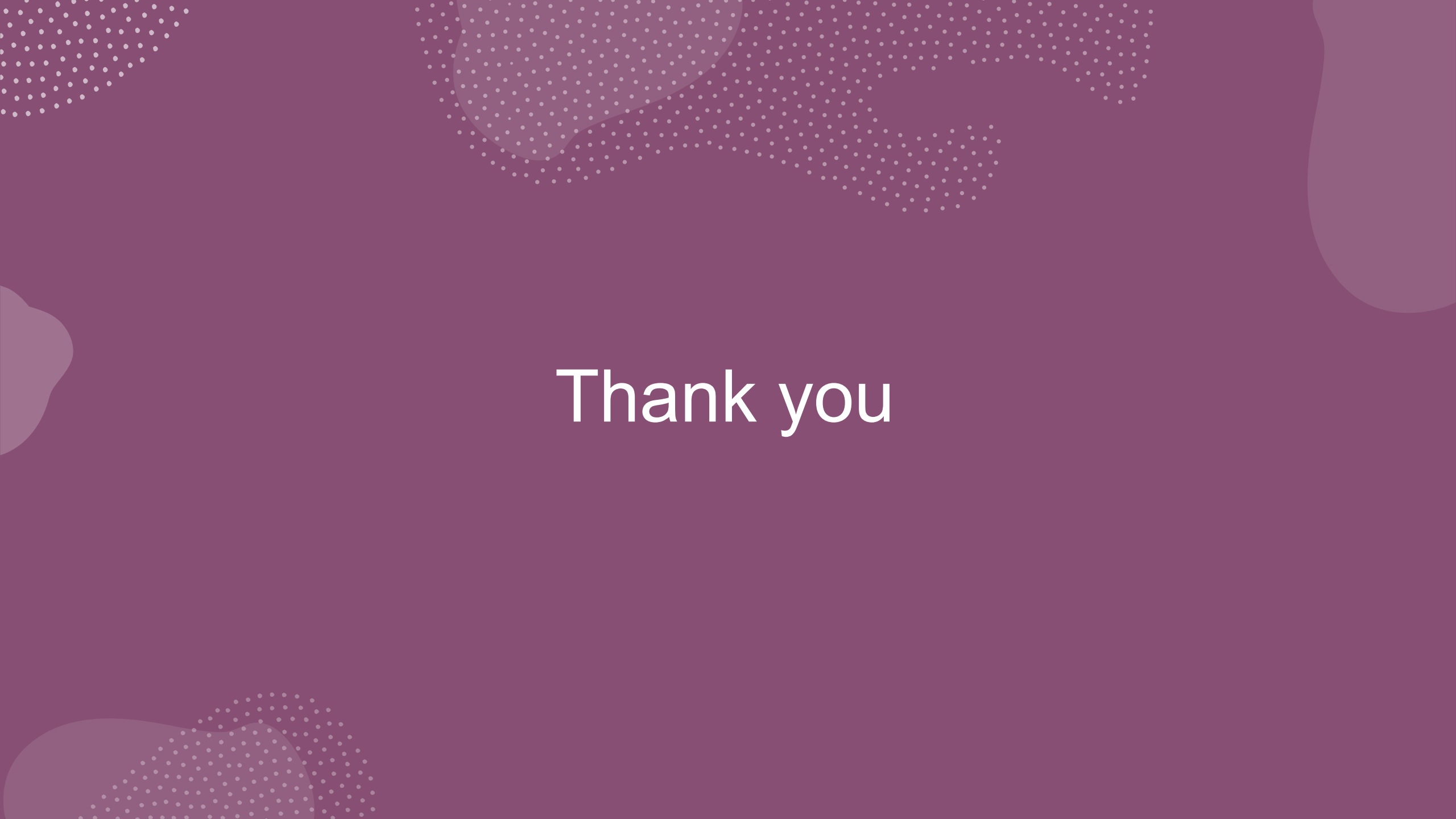
Stimuli and responses for a burglar alarm system

Stimulus	Response
Clear alarms	Switch off all active alarms; switch off all lights that have been switched on.
Console panic button positive	Initiate alarm; turn on lights around console; call police.
Power supply failure	Call service technician.
Sensor failure	Call service technician.
Single sensor positive	Initiate alarm; turn on lights around site of positive sensor.
Two or more sensors positive	Initiate alarm; turn on lights around sites of positive sensors; call police with location of suspected break-in.
Voltage drop of between 10% and 20%	Switch to battery backup; run power supply test.
Voltage drop of more than 20%	Switch to battery backup; initiate alarm; call police; run power supply test.

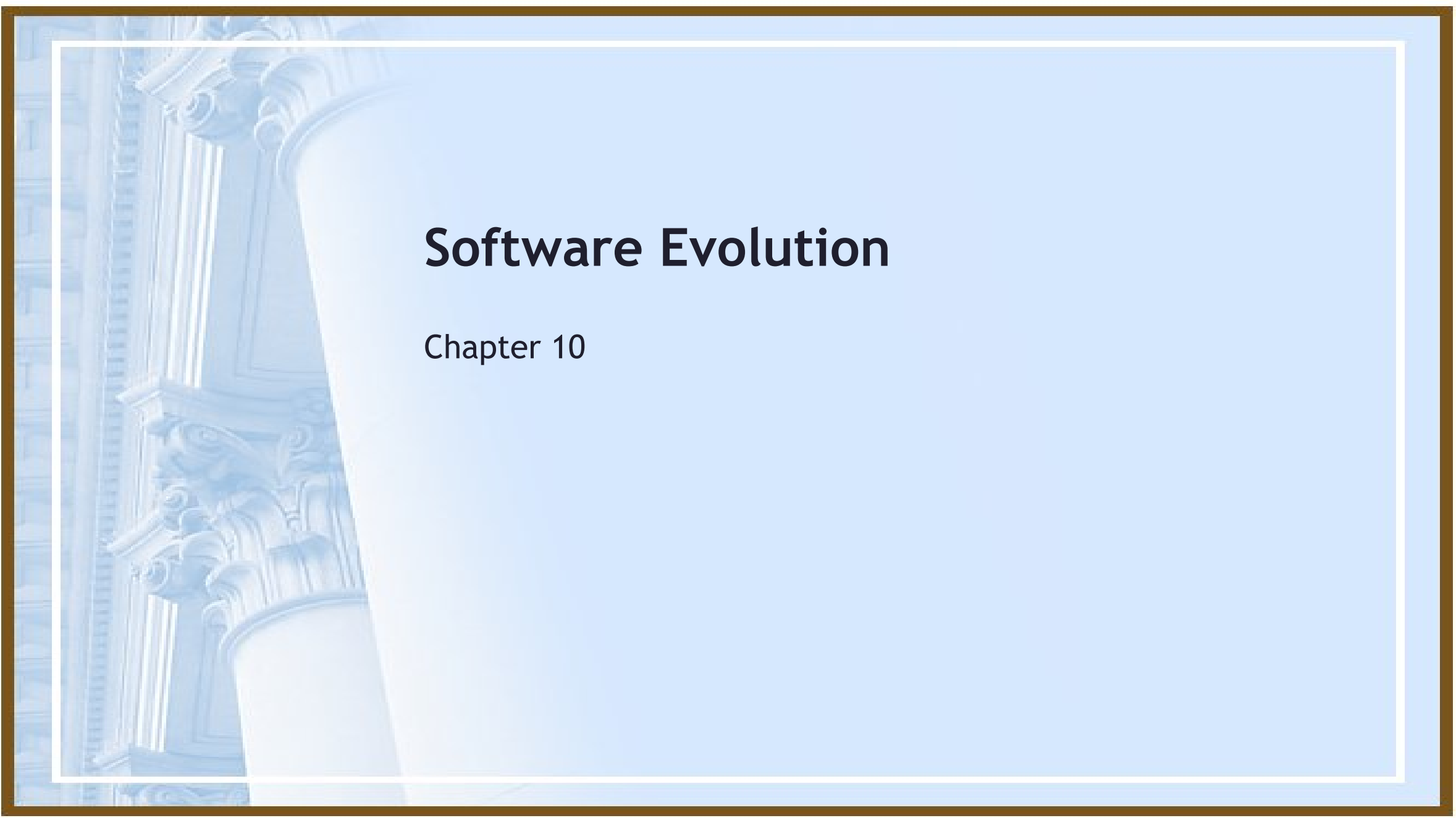
Quiz

- Identify ONE stimuli and corresponding/ associated responses for a carpark system.
- A carpark system consists of the following hardware:
 - Sensor
 - Barrier
 - Ticket machine

Stimuli (... ..) → Response (... ..)

The background is a solid medium purple color. It features several decorative elements: a cluster of white dots in the top-left corner, a large, irregular shape filled with white dots in the top-center, a solid purple organic shape on the right side, a solid purple organic shape on the left side, and a cluster of white dots in the bottom-left corner.

Thank you

The background of the slide features a low-angle, upward-looking perspective of several tall, white classical columns with ornate Corinthian capitals. The columns are set against a clear, bright blue sky. The entire image is overlaid with a semi-transparent light blue rectangle, which serves as a backdrop for the text. The slide is framed by a thin brown border.

Software Evolution

Chapter 10

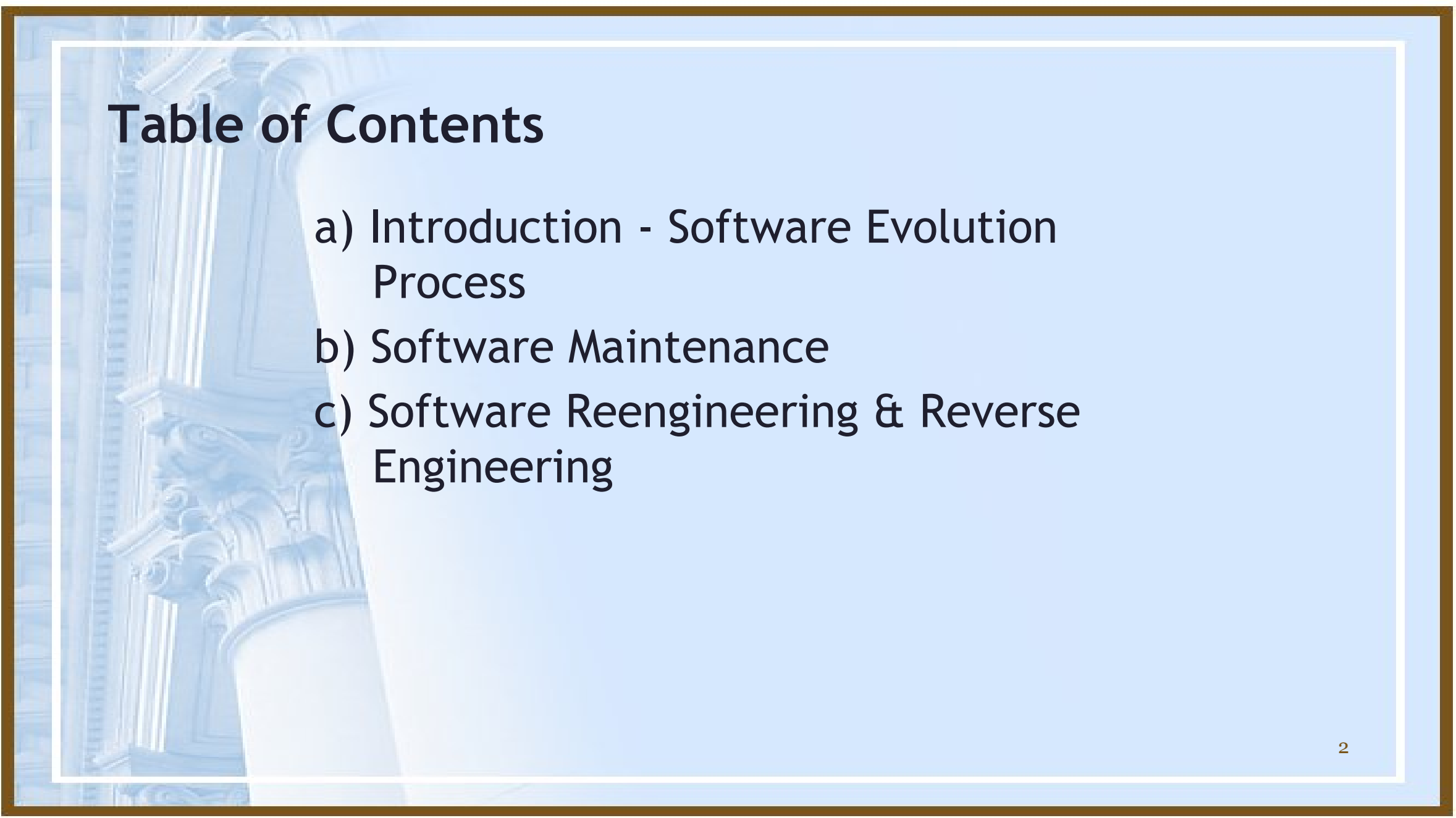


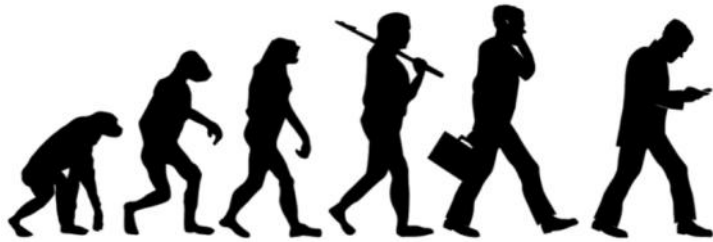
Table of Contents

- a) Introduction - Software Evolution Process
- b) Software Maintenance
- c) Software Reengineering & Reverse Engineering

The background of the slide features a light blue gradient. On the left side, there is a faint, semi-transparent image of classical architectural columns, likely from a government building, which adds a sense of formality and tradition to the presentation.

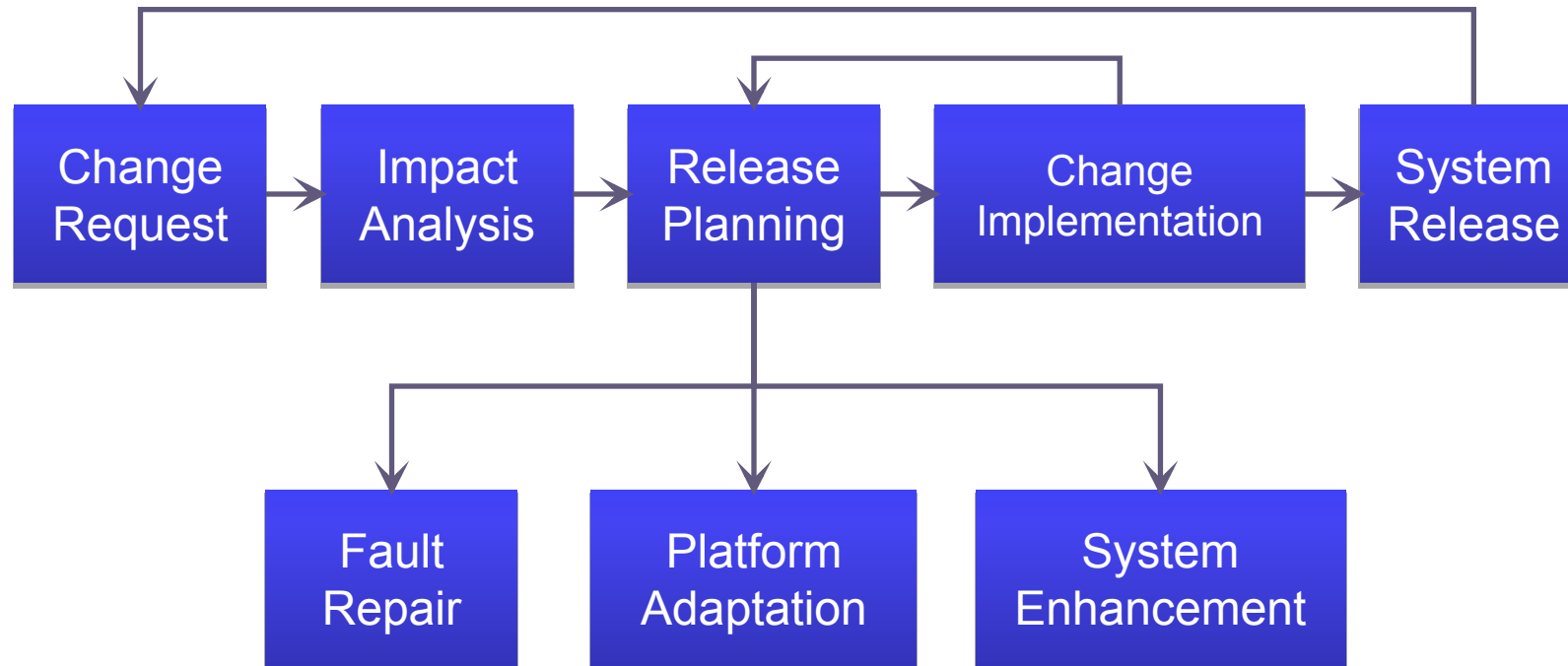
a) Introduction - Software Evolution Process

a) Introduction - Software Evolution Process



Software Evolution is a term that refers to the process of developing software initially, and then timely updating it for various reasons.

a) Introduction - Software Evolution Process



CHANGE REQUESTS

- Change requests are requests for system changes from users, customers or management
- In principle, all change requests should be carefully analysed as part of the maintenance process and then implemented
- In practice, some change requests must be implemented urgently
 - Fault repair
 - Changes to the system's environment
 - Urgently required business changes



Software Evolution Process

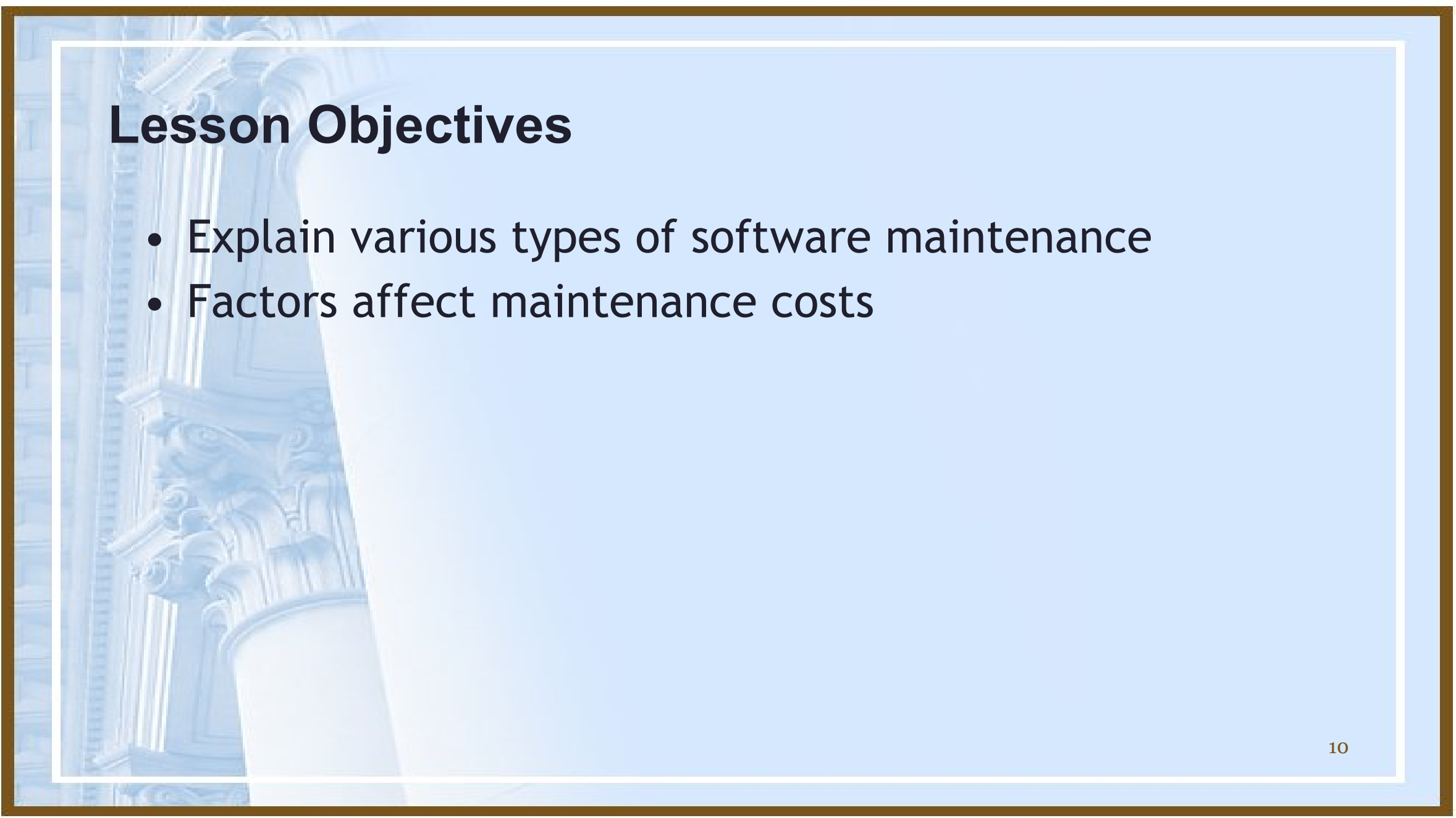
- Evolution starts from the requirement-gathering process. After which developers create a prototype of the intended software and show it to the users to get their feedback at the early stage of software product development.
- The users suggest changes, on which several consecutive updates and maintenance keep on changing too. This process changes to the original software, till the desired software is accomplished.

Software Evolution Process

- Even after the user has the desired software in hand, the advancing technology and the changing requirements force the software product to change accordingly.
- Re-creating software from scratch and to go one-on-one with requirements is not feasible. The only feasible and economical solution is to update the existing software so that it matches the latest requirements

The background of the slide features a low-angle photograph of several tall, white classical columns with ornate capitals, reaching towards a clear, bright blue sky. The image is slightly faded and serves as a backdrop for the text.

b) Software Maintenance

The background of the slide features a low-angle, perspective view of several tall, white classical columns with ornate capitals. The columns are set against a clear blue sky. The entire image is overlaid with a semi-transparent light blue rectangle that serves as the background for the text.

Lesson Objectives

- Explain various types of software maintenance
- Factors affect maintenance costs

Software Maintenance

Software Maintenance refers to the process of modifying and updating a software system after it has been delivered to the customer. This involves fixing bugs, adding new features, and adapting to new hardware or software environments



Software Maintenance

- ✓ Software errors
- ✓ Installation of new hardware
- ✓ Customer needs



Reasons for changes

Types of Software Maintenance



Corrective Maintenance

CM is a type of maintenance task or action is usually performed after equipment failure.

Types of Software Maintenance



Adaptive Maintenance

This involves modifying the software system to adapt it to changes in the environment, such as changes in hardware or software, government policies, and business rules.

New environment:

- ✓ hardware platform
- ✓ OS

Types of Software Maintenance



Perfective Maintenance

This involves improving functionality, performance, and reliability, and restructuring the software system to improve changeability.

New functional/non-functional requirements

Factors that Affecting Maintenance Cost

Module independence

- ✓ High independency,
low cost



Factors that Affecting Maintenance Cost

Programming Language

- ✓ High level programming language



Factors that Affecting Maintenance Cost

Programming Style

✓ Good structure → low cost



Factors that Affecting Maintenance Cost

Program Validation & Testing

- ✓ More time spent in testing, ↓ error, ↓ cost



Factors that Affecting Maintenance Cost

Documentation Quality

- ✓ *Good quality, good understanding, ↓ cost*



Factors that Affecting Maintenance Cost

Configuration Management Techniques

- ✓ Effect technique →
easy to keep track all
versions, ↓ cost



Factors that Affecting Maintenance Cost

Application Domain

- ✓ New application domain,
less understanding,
↑cost



Factors that Affecting Maintenance Cost

Staff Stability

- ✓ New project reassignment, $\uparrow$ cost
- ✓ Same staff maintain, $\downarrow$ cost



Factors that Affecting Maintenance Cost

Age of the program

- ✓ > older, > maintenance it receives, > cost



Factors that Affecting Maintenance Cost

The dependence of the program on its external environment

- ✓ > depend on environment changes, > cost



Factors that Affecting Maintenance Cost

Hardware stability

- ✓ If hardware no need to change, < cost (rarely happen)



Maintenance Cost

The costs of adding functionality to a system after it has been put into operation are usually much greater than providing similar functionality when software is originally developed.

WHY?



Maintenance Cost

- Maintenance staff are often relatively **inexperienced and unfamiliar** with the application domain
- The **programs** being maintained may have been developed many years ago **without modern software engineering techniques**. Therefore they may be unstructured and difficult to understand.
- **Changes** made to a program may **introduce new faults**, which trigger further change requests.



Maintenance Cost

Maintenance staff:

- Inexperience
- Unfamiliar with application domain



Maintenance Cost

Old program:

- Old engineering technique
- Unstructured
- Difficult to understand



Maintenance Cost

Changes → new faults → further change request



Maintenance Cost

Change → structure degraded → harder to understand, less cohesive



Maintenance Cost

Change → documentation link missing →
unreliable document → difficult to
understand program



The background of the slide features a low-angle photograph of several tall, white classical columns with ornate capitals, reaching towards a clear, bright blue sky. The image is slightly faded and serves as a backdrop for the text.

c) Software Reengineering

Lesson Objectives

- What is software reengineering
- Differentiate software reengineering to reverse engineering
- The advantages and disadvantages of software reengineering
- Legacy system management

Introduction



Legacy System

80% of Software Expenditure
is on system maintenance &
evolution

Introduction



Legacy System

1990 → 120 billion LOC
being maintained

2000 → 250 billion LOC
being maintained

Introduction



Legacy System

Why exist?

Business procedures & knowledge of system may not documented elsewhere

Too risky to rewrite them

Introduction

Problems:



Legacy System

- Poor structure
- Outdated documentation
- Inconsistent documentation
- Etc.

Introduction



Legacy System

Solution?

Software Reengineering

Software Reengineering - Objectives

Improve system structure

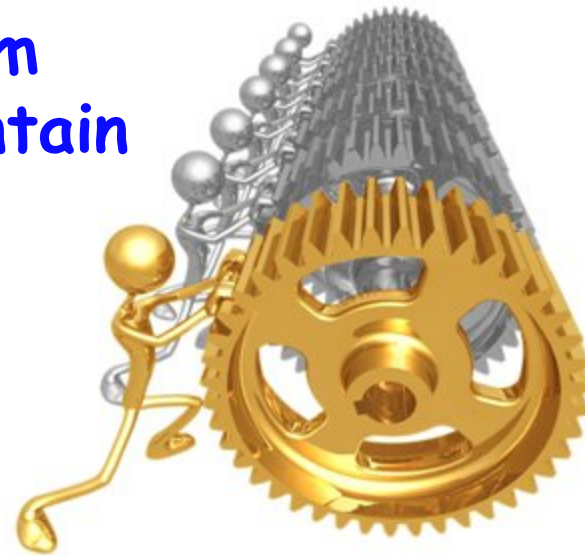
New system documentation

Easy understand system



Software Reengineering - Objectives

Re-implement legacy system
to make them easy to maintain



Software Reengineering - Objectives

How?

- Source code translation
- Reverse Engineering
- Program restructuring
- Program modularization
- Data reengineering



Introduction

- The main objectives of system re-engineering are to:
 - improve the system structure
 - create new system documentations
 - make it easier to understand.

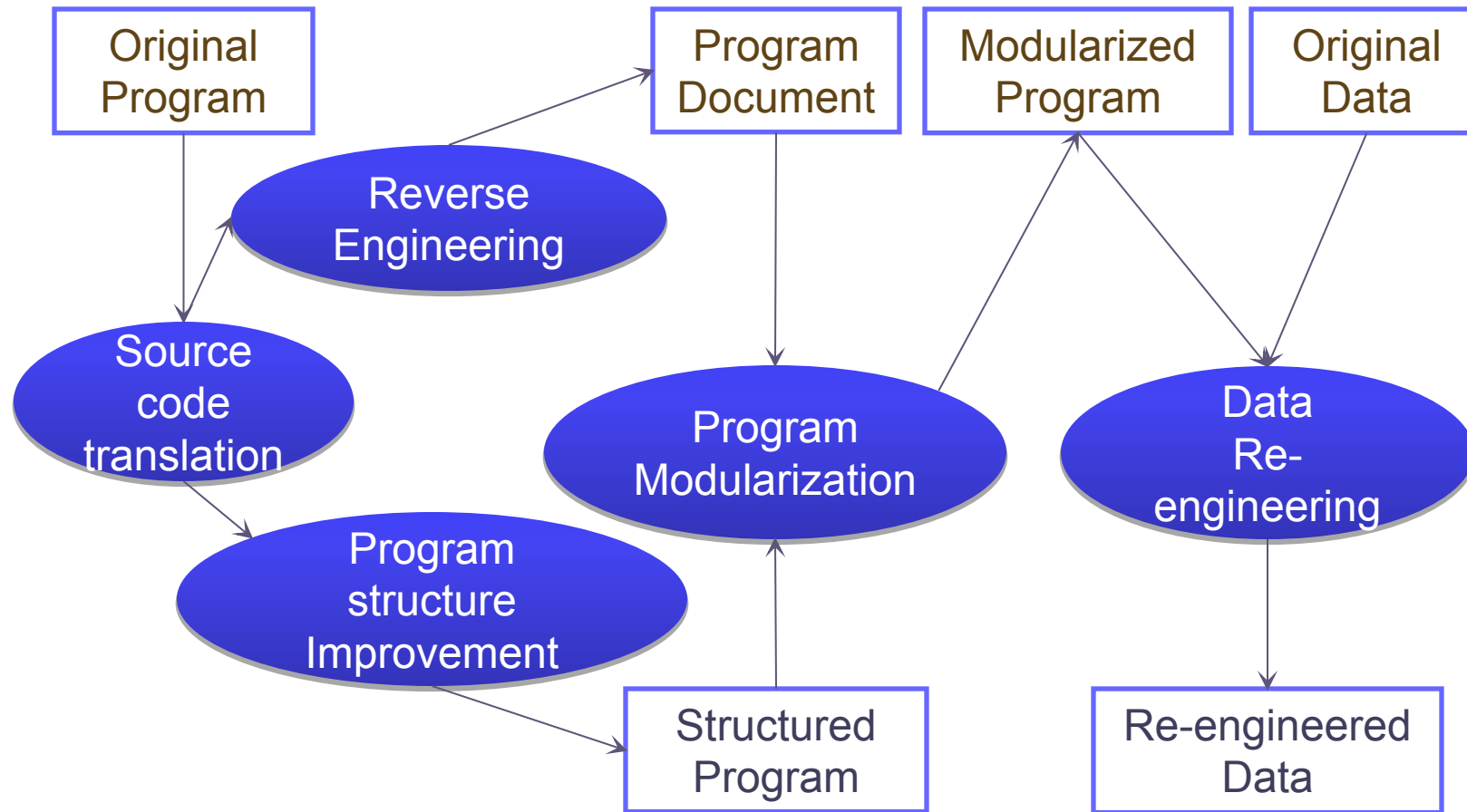
So, the cost of future system maintenance should therefore be reduced.

- In other words, software re-engineering is concerned with taking existing legacy systems and **re-implementing** them to make them more maintainable.

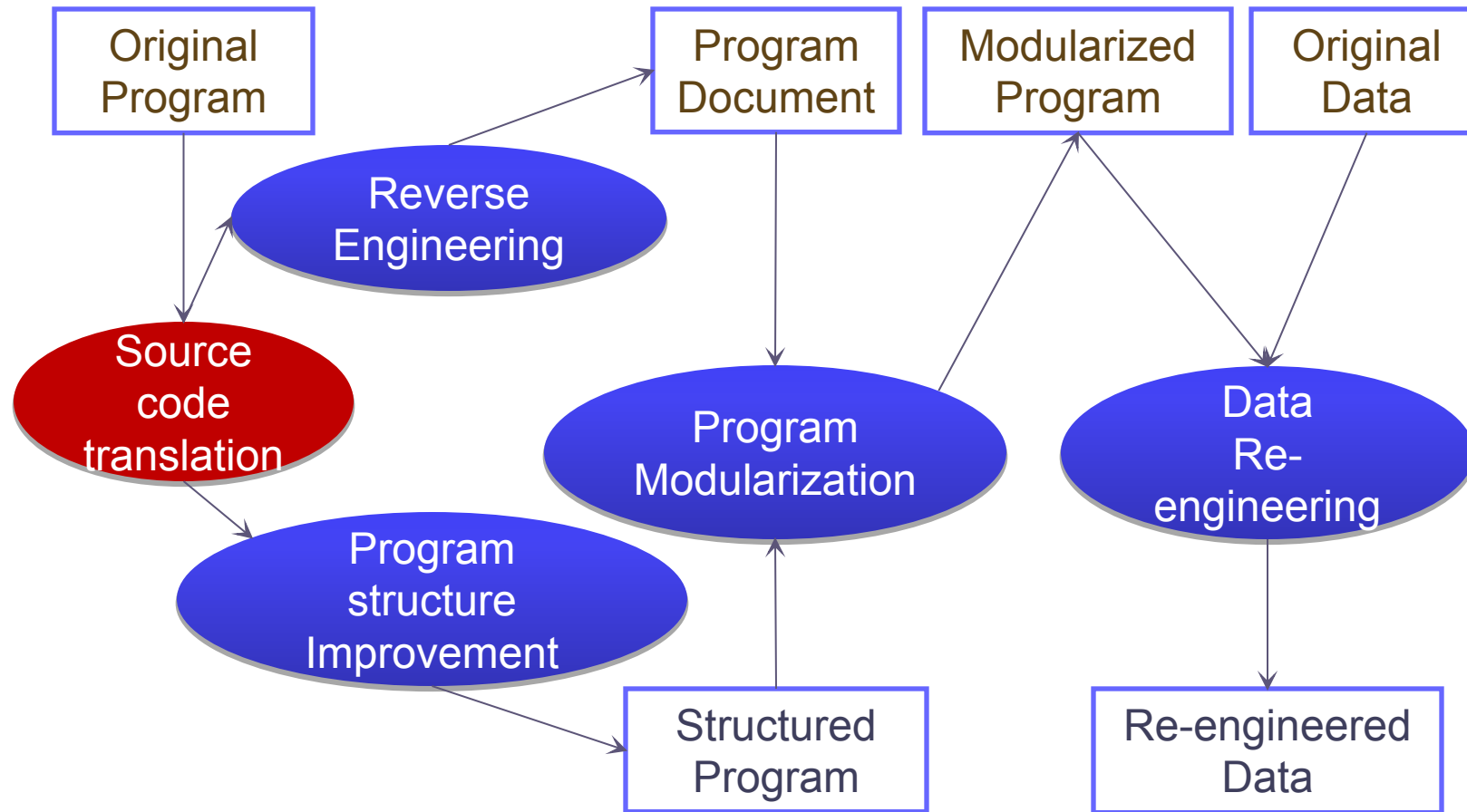
How?

- Source code translation
- Program restructuring
- Data reengineering etc

Software Re-engineering Process



Software Re-engineering Process



Software Re-engineering Process



Translate source code into
another programming language

Fortran → C

Source Code Translation

Software Re-engineering Process



Source Code Translation

Why?

- Hardware platform update
- Staff skill shortage
- Organizational policy change

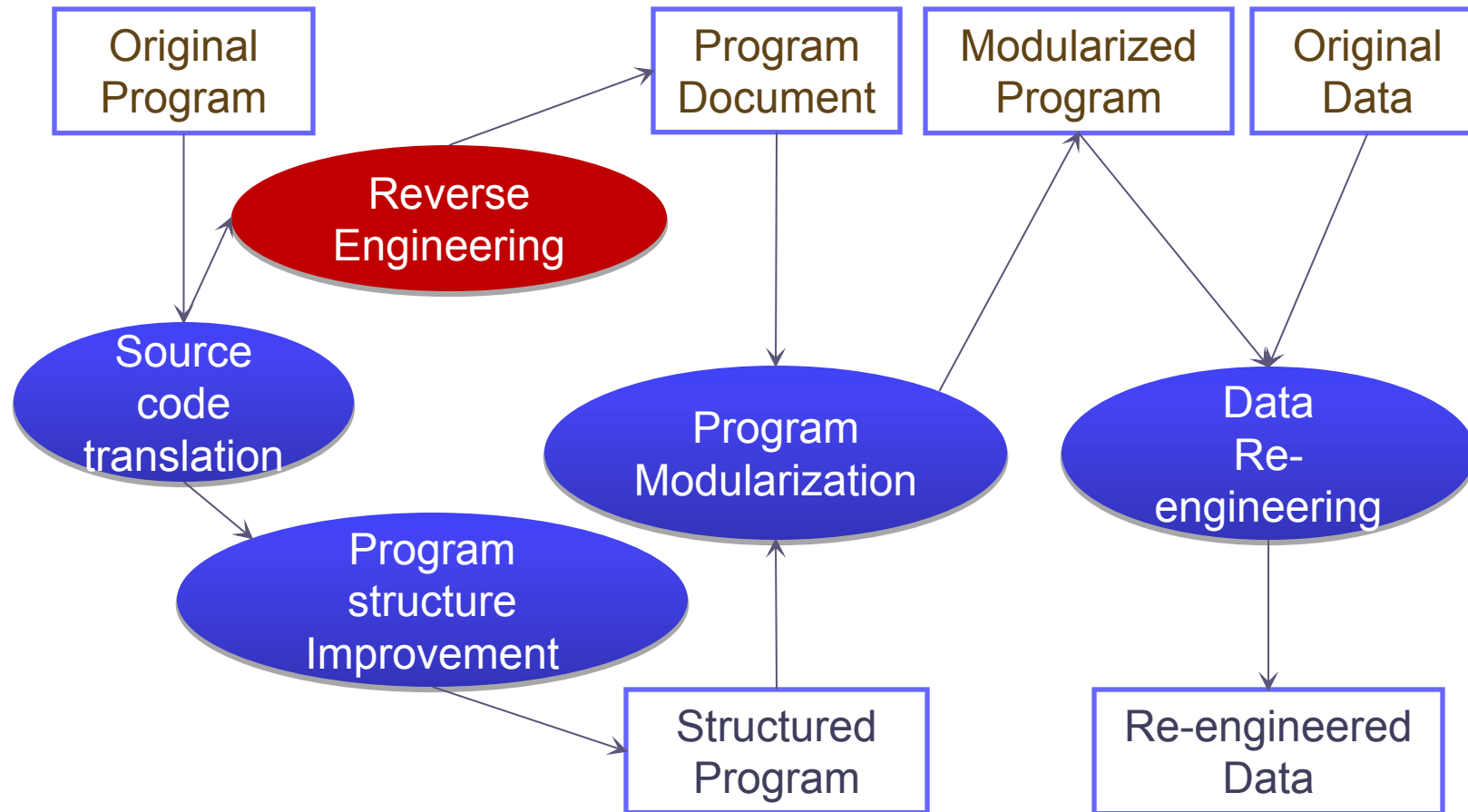
Software Re-engineering Process



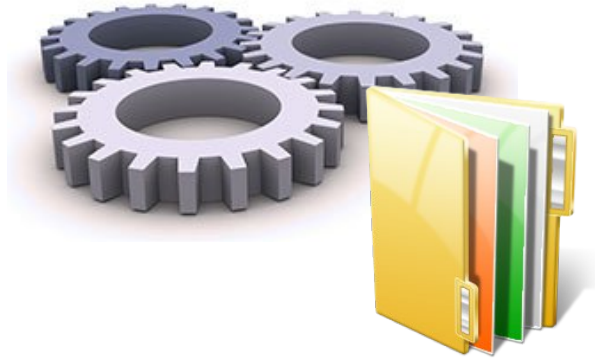
Source Code Translation

- CASE tool
- E.g. REFINE
- Still need manual work

Software Re-engineering Process



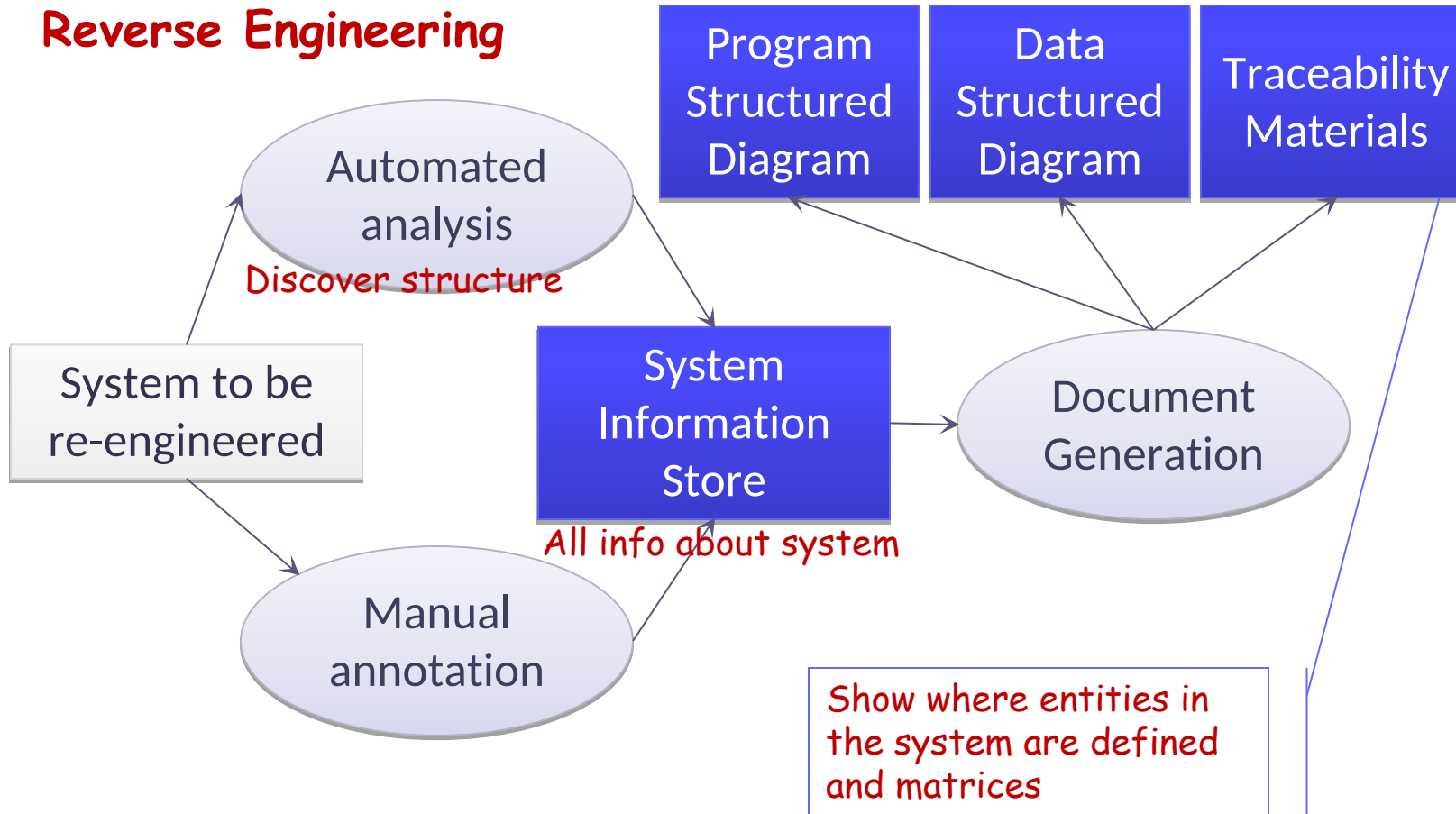
Software Re-engineering Process



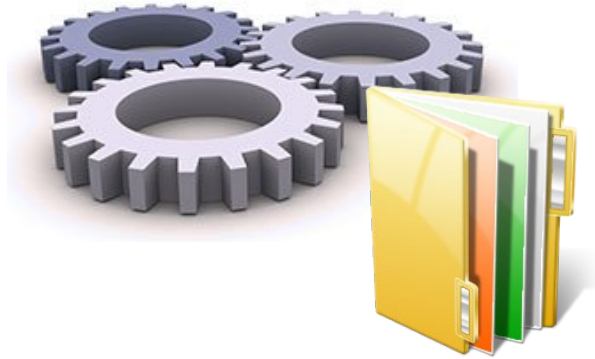
Reverse Engineering

Derive system design &
specification →
documentation

Software Re-engineering Process



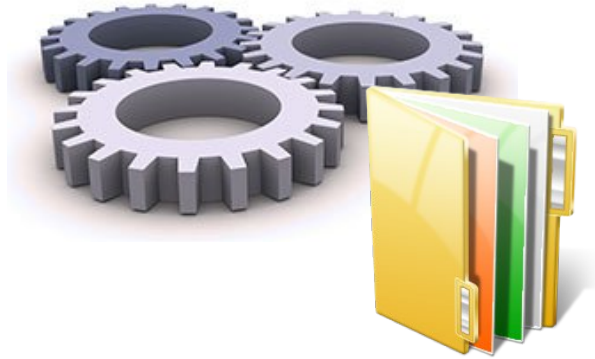
Software Re-engineering Process



Reverse Engineering

In other words, reverse engineering is the process of analyzing software with the objective of recovering its design and specification (**documentation**).

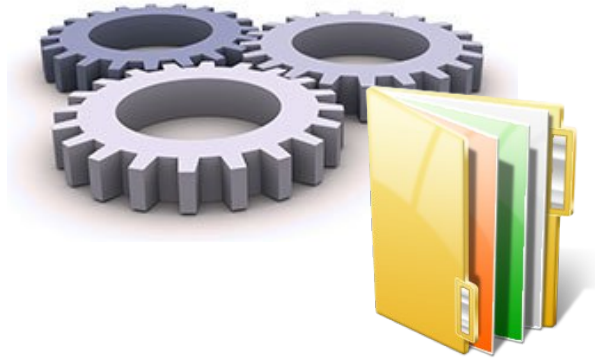
Software Re-engineering Process



Reverse Engineering

Documentation → input to
Program Modularization

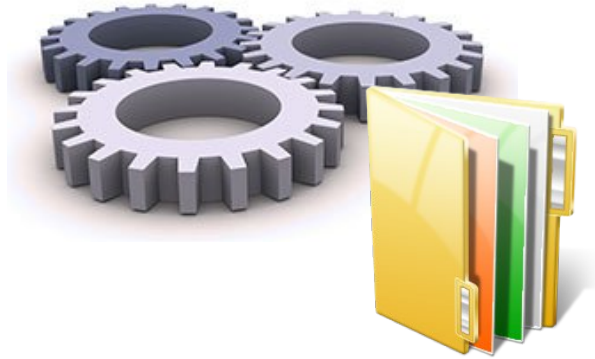
Software Re-engineering Process



Reverse Engineering

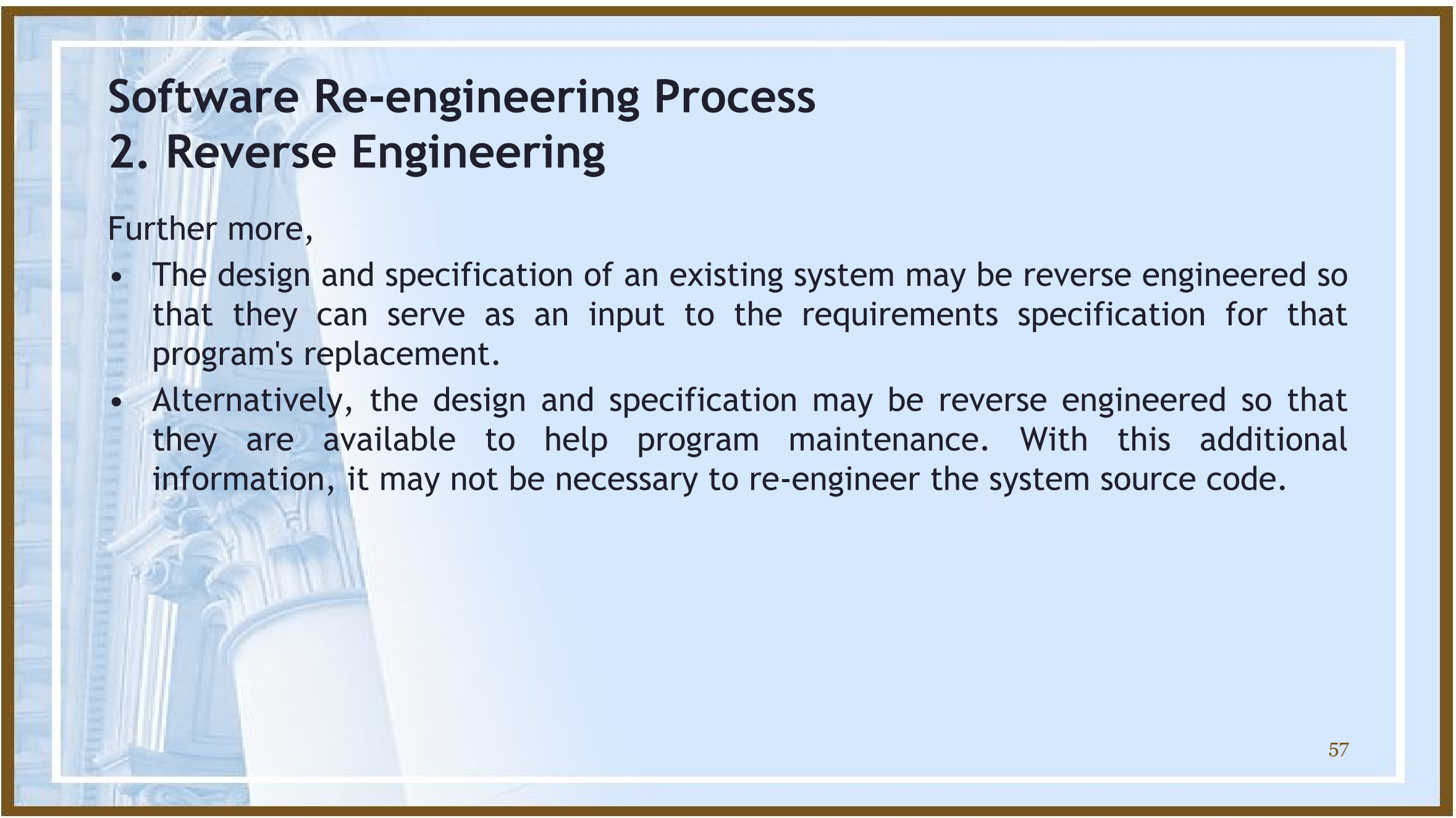
Documentation → input to
requirement specification
for program replacement

Software Re-engineering Process



Reverse Engineering

Documentation → help
program maintenance



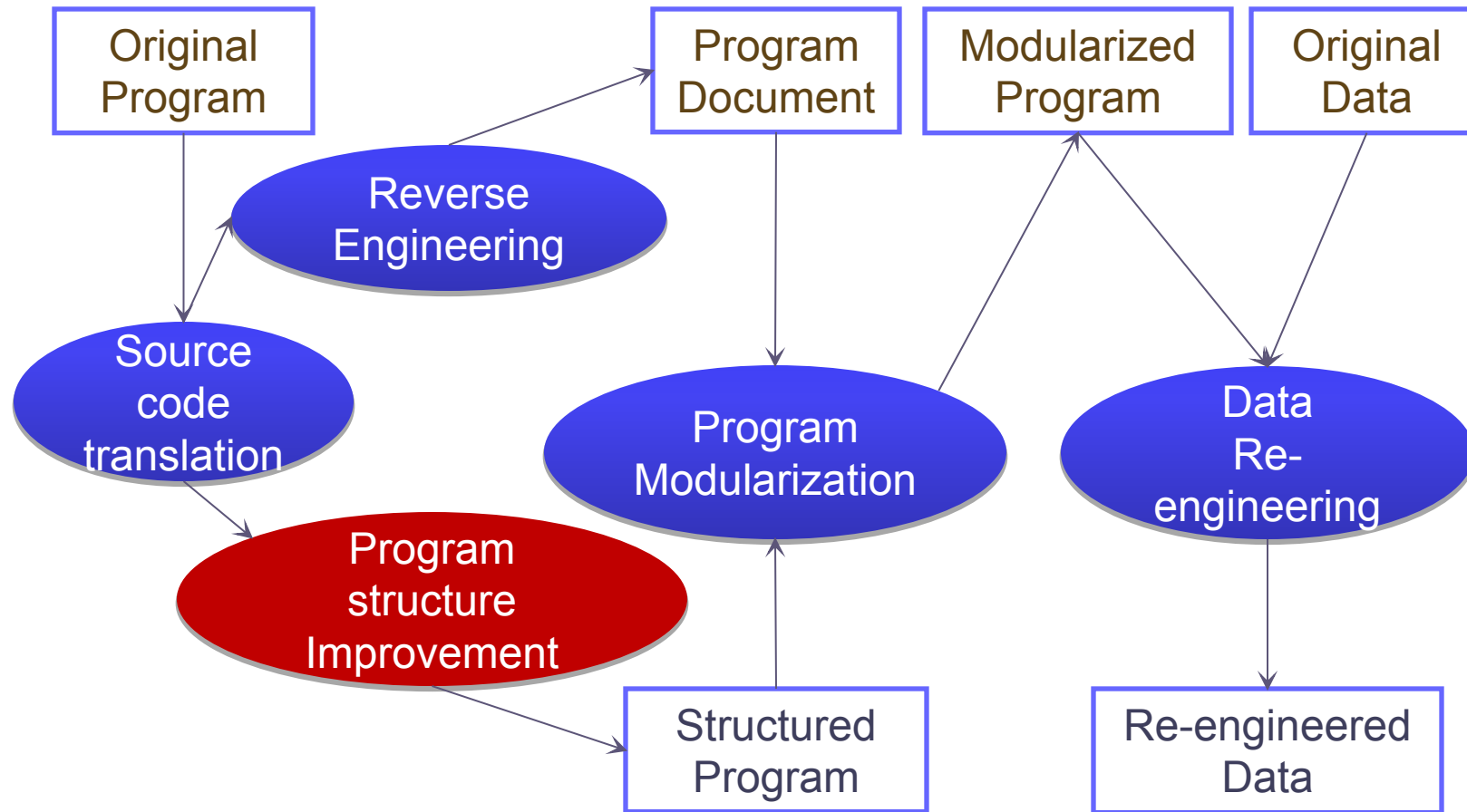
Software Re-engineering Process

2. Reverse Engineering

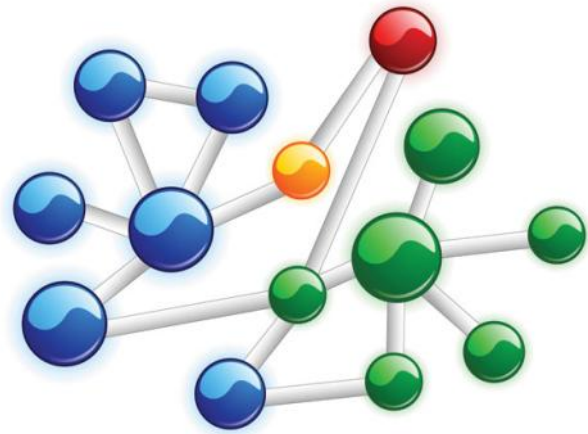
Further more,

- The design and specification of an existing system may be reverse engineered so that they can serve as an input to the requirements specification for that program's replacement.
- Alternatively, the design and specification may be reverse engineered so that they are available to help program maintenance. With this additional information, it may not be necessary to re-engineer the system source code.

Software Re-engineering Process



Software Re-engineering Process

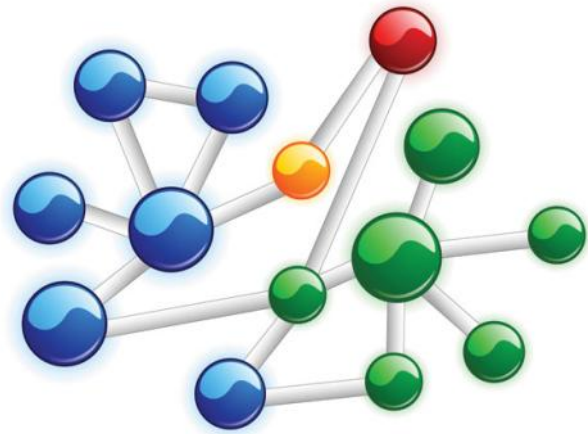


**Program Structure
Improvement**

Control Structure

**E.g. 'spaghetti' program
structure**

Software Re-engineering Process



**Program Structure
Improvement**

Control Structure

**Simplify complex
conditions**



```
Start:  Get (Time-on, Time-off, Setting, Temp, Switch)
        if Switch = off goto off
        if Switch = on goto on
        goto CntrlD
off:    if Heating-status = on goto Sw-off
        goto loop
on:     if Heating-status = off goto Sw-on
        goto loop
CntrlD: if Time = Time-on goto on
        if Time = Time-off goto off
        if Time < Time-on goto Start
        if Time > Time-off goto Start
        if Temp > Setting then goto off
        if Temp < Setting then goto on
Sw-off: Heating-status := off
        goto Switch
Sw-on:  Heating-status := on
Switch: Switch-heating
loop:   goto Start
```

A control program with spaghetti logic

Loop

Get (...)

Case Switch off

When On ->

When Off →

When Controlled →

End Case

End Loop

a **structured** control program

Example of Condition Simplification

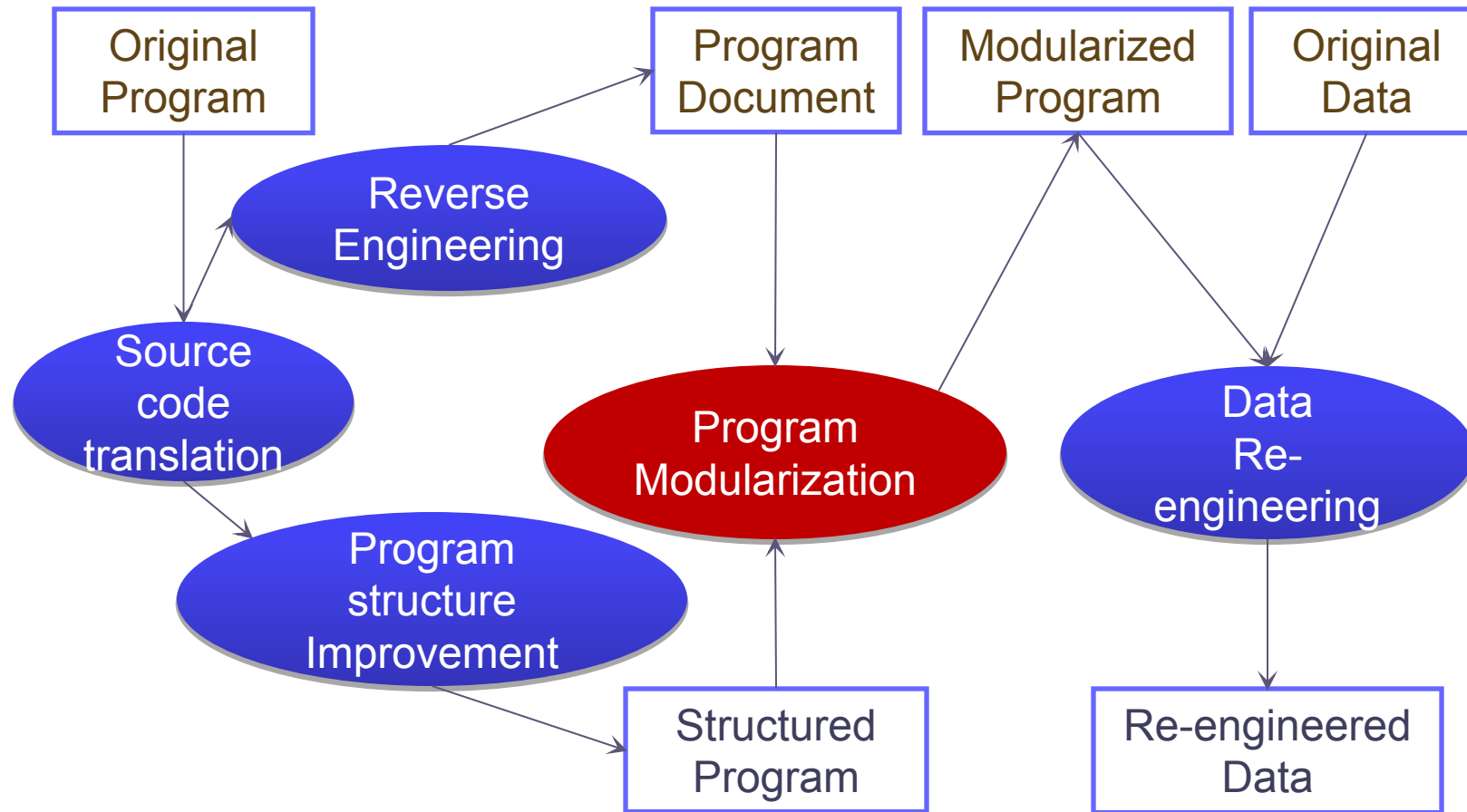
Complex condition

if not (A > B and (C < D or not (E > F))) ...

Simplified condition

if A <= B and (C >= D or E > F)...

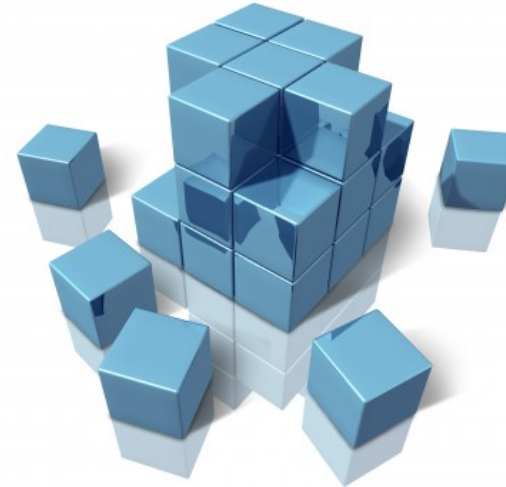
Software Re-engineering Process



Software Re-engineering Process

Group related parts

Remove redundancy

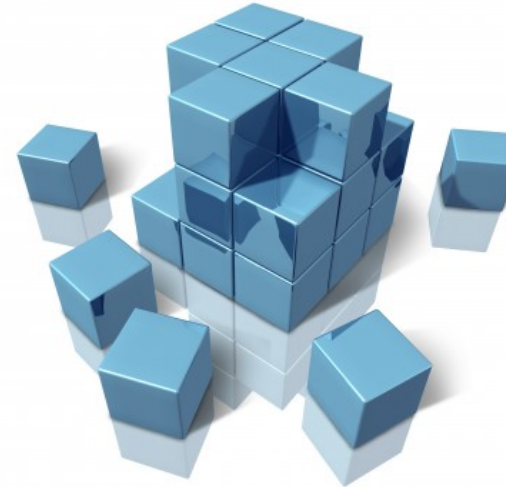


Program Modularization

Software Re-engineering Process

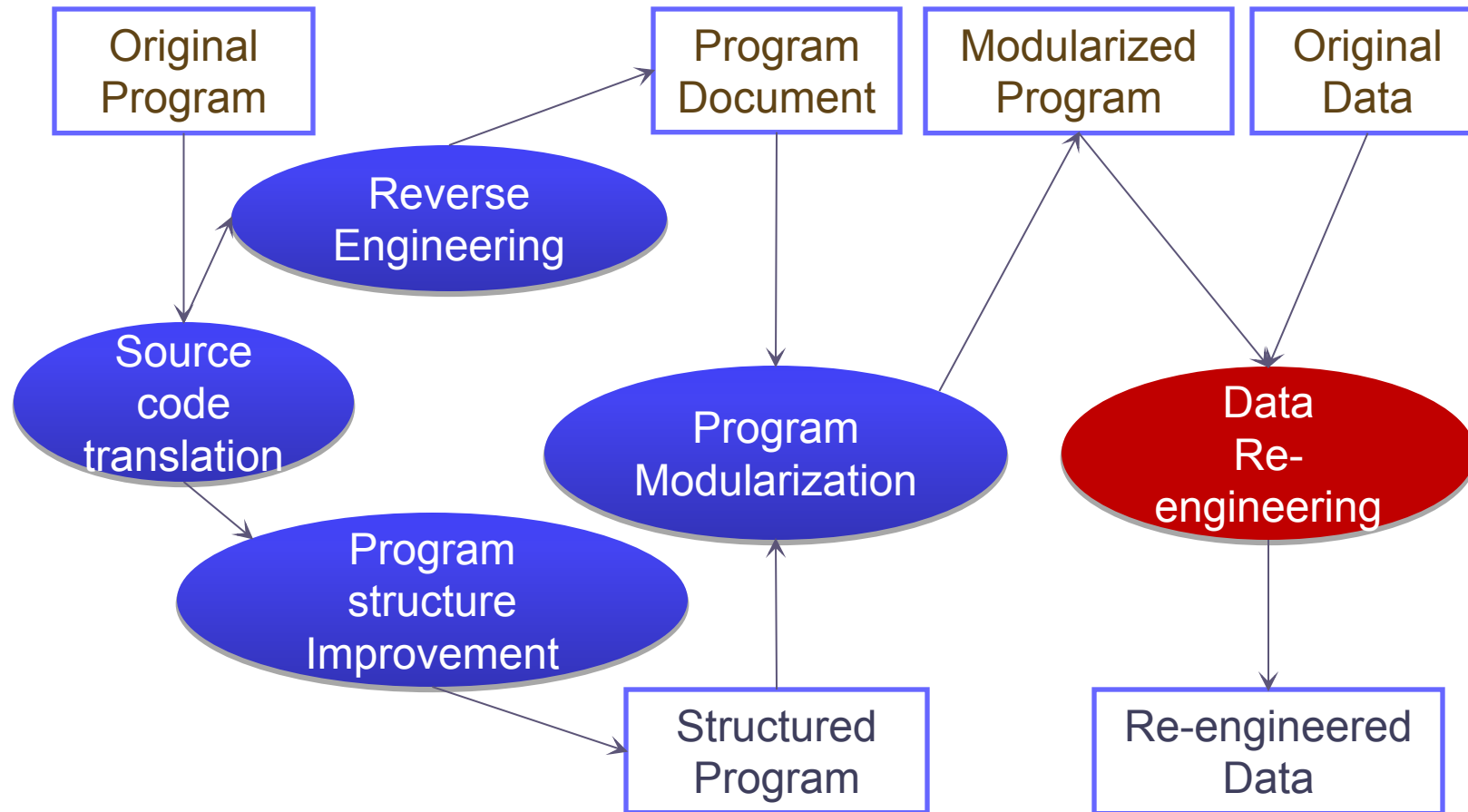
Architectural transformation

E.g. centralized system
run on distributed
platform



Program Modularization

The Re-engineering Process



Software Re-engineering Process

5. Data Re-engineering

- How to re-engineer? (refer diagram)
- Some of the problems (b4 data re-engineering) with data which can arise in legacy systems made up of several cooperating programs are: -
 - data naming (for files/attributes) problems
 - field length problems: assigned diff length in diff program!
 - record organization problems : same entity may be org differently in diff programs, e.g. COBOL language.
 - hard-coded literals/values: such as tax rate
 - no data dictionary: to define their representation etc.

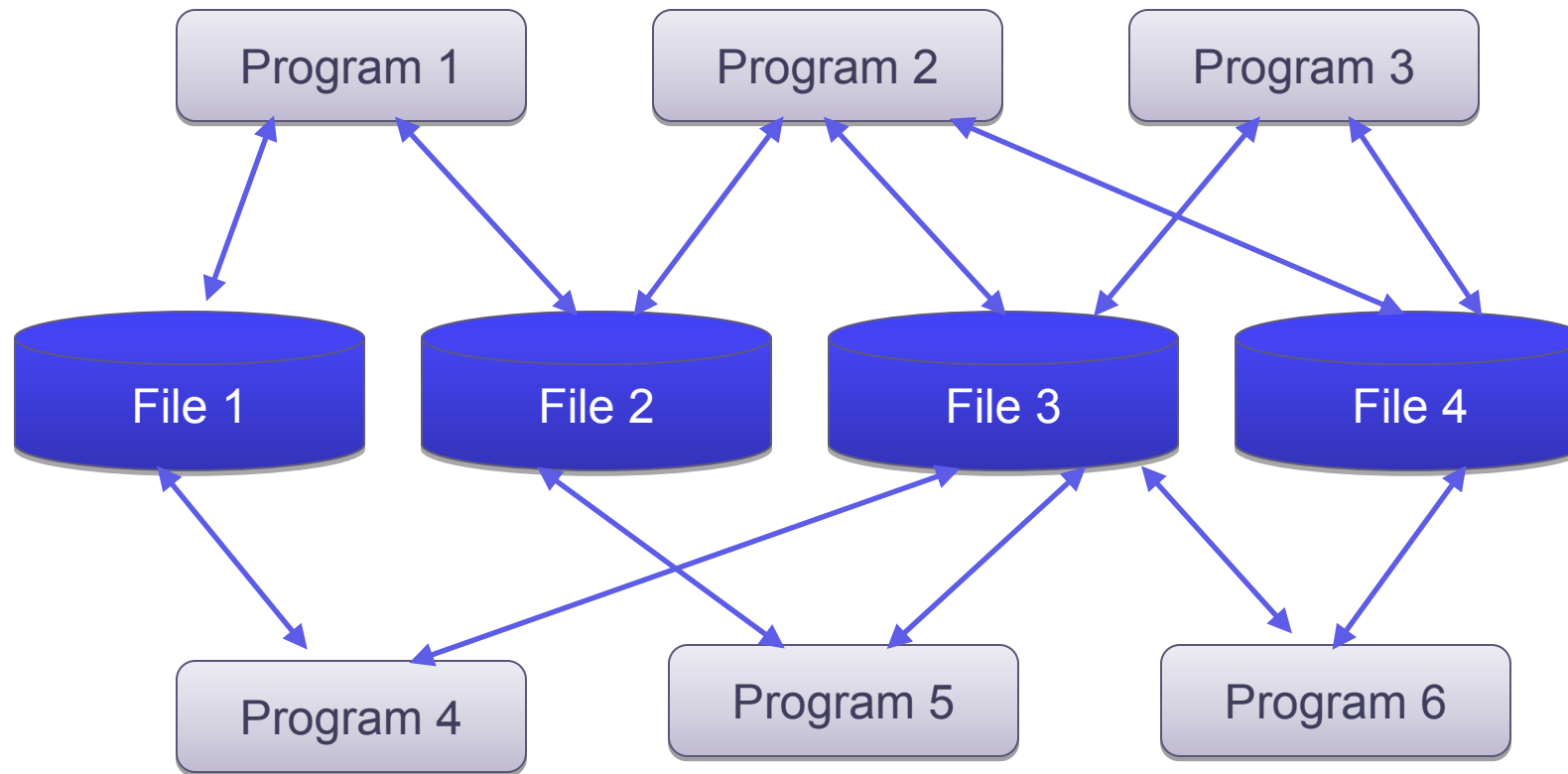
Software Re-engineering Process

5. Data Re-engineering

- **Data values** may also be stored in an inconsistent way. After the data definitions have been re-engineered, the data values must also be converted to conform to the new structure. Some of the data value inconsistencies problems (before data re-engineering) are:
 - Inconsistent default values: different date format?
 - Inconsistent validation rules
 - Inconsistent units: kg vs pound
 - Inconsistent representation semantic
 - Inconsistent handling of negative values

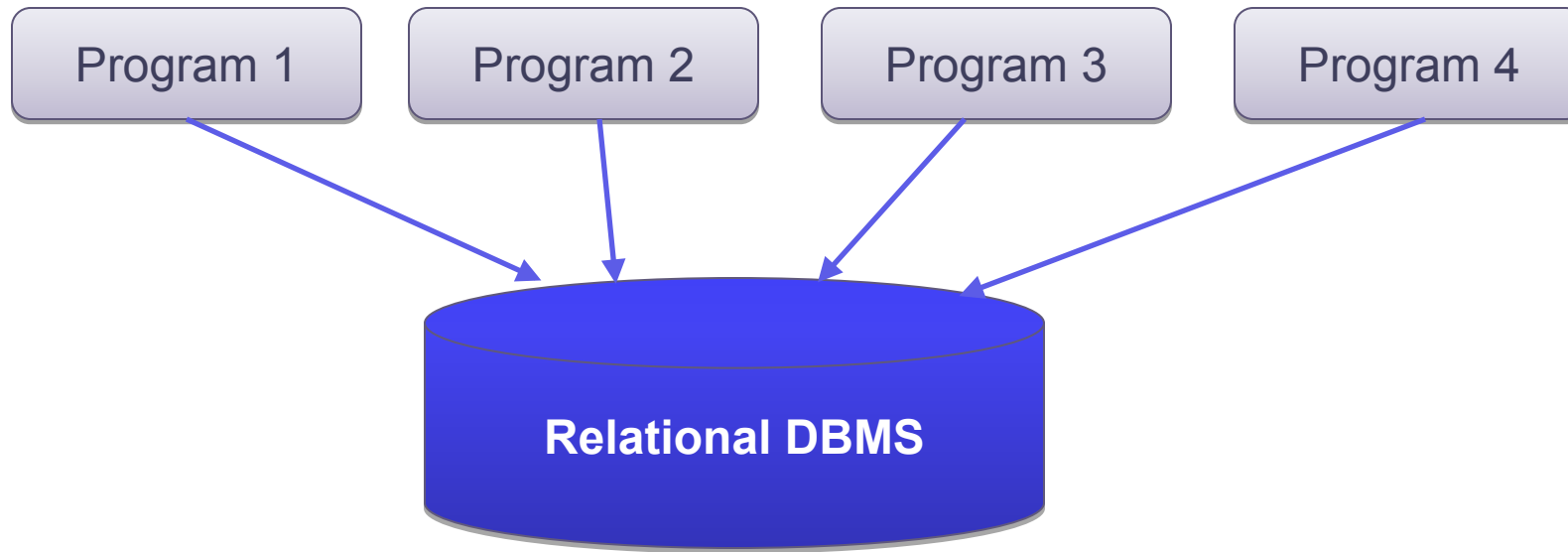
Software Re-engineering Process

Data Re-engineering (Before)



Software Re-engineering Process

Data Re-engineering (After)



Software Re-engineering Process

Problems with data before re-engineering:

Data naming problems
(file/attributes)



Data Re-engineering

Software Re-engineering Process

Problems with data before re-engineering:
Field length problems



Data Re-engineering

Software Re-engineering Process

Problems with data before re-engineering:
Record organization problems



Data Re-engineering

Software Re-engineering Process

Problems with data before re-engineering:
Hard-coded literals/values



Data Re-engineering

Software Re-engineering Process

Problems with data before re-engineering:

No data dictionary



Data Re-engineering

Software Re-engineering Process

Problems with **data value inconsistencies** before re-engineering:

Inconsistent default value

Different program assign different default value to the same logical data item.



Data Re-engineering



Software Re-engineering Process

Problems with **data value inconsistencies**
before re-engineering:

Inconsistent validation rules

Data written by one program
may be rejected by another



Data Re-engineering



Software Re-engineering Process

Problems with **data value inconsistencies**
before re-engineering:

Inconsistent units



Data Re-engineering

Software Re-engineering Process

Problems with **data value inconsistencies**
before re-engineering:

Inconsistent representation
semantic

Multiple ways of using
uppercase to convey meaning
in text strings



Data Re-engineering



Software Re-engineering Process

Problems with **data value inconsistencies** before re-engineering:

Inconsistent handling of negative values

Some programs do not accept -ve value, some programs accept & convert to +ve value.



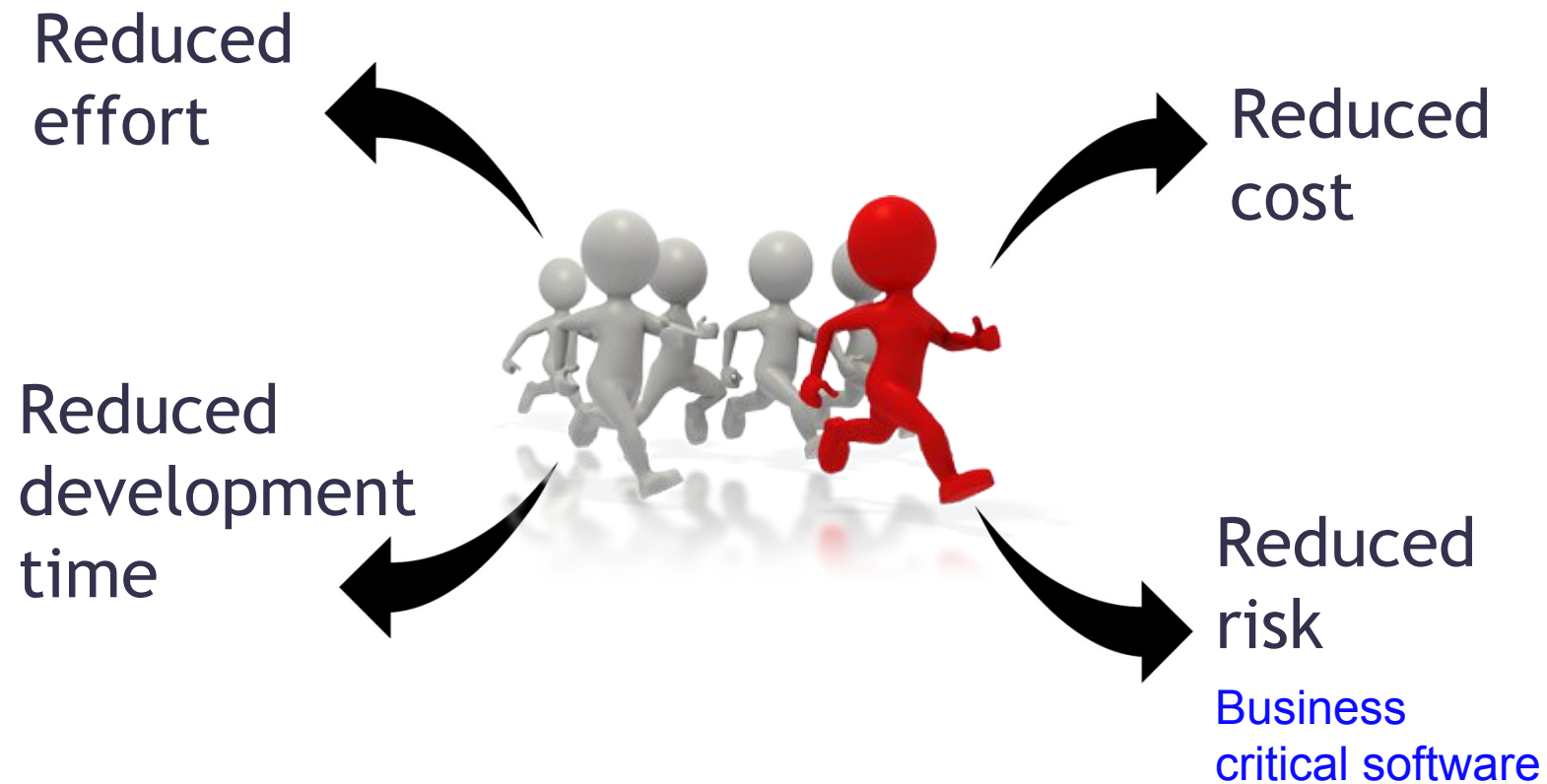
Data Re-engineering



Cost Factors of Re-engineering

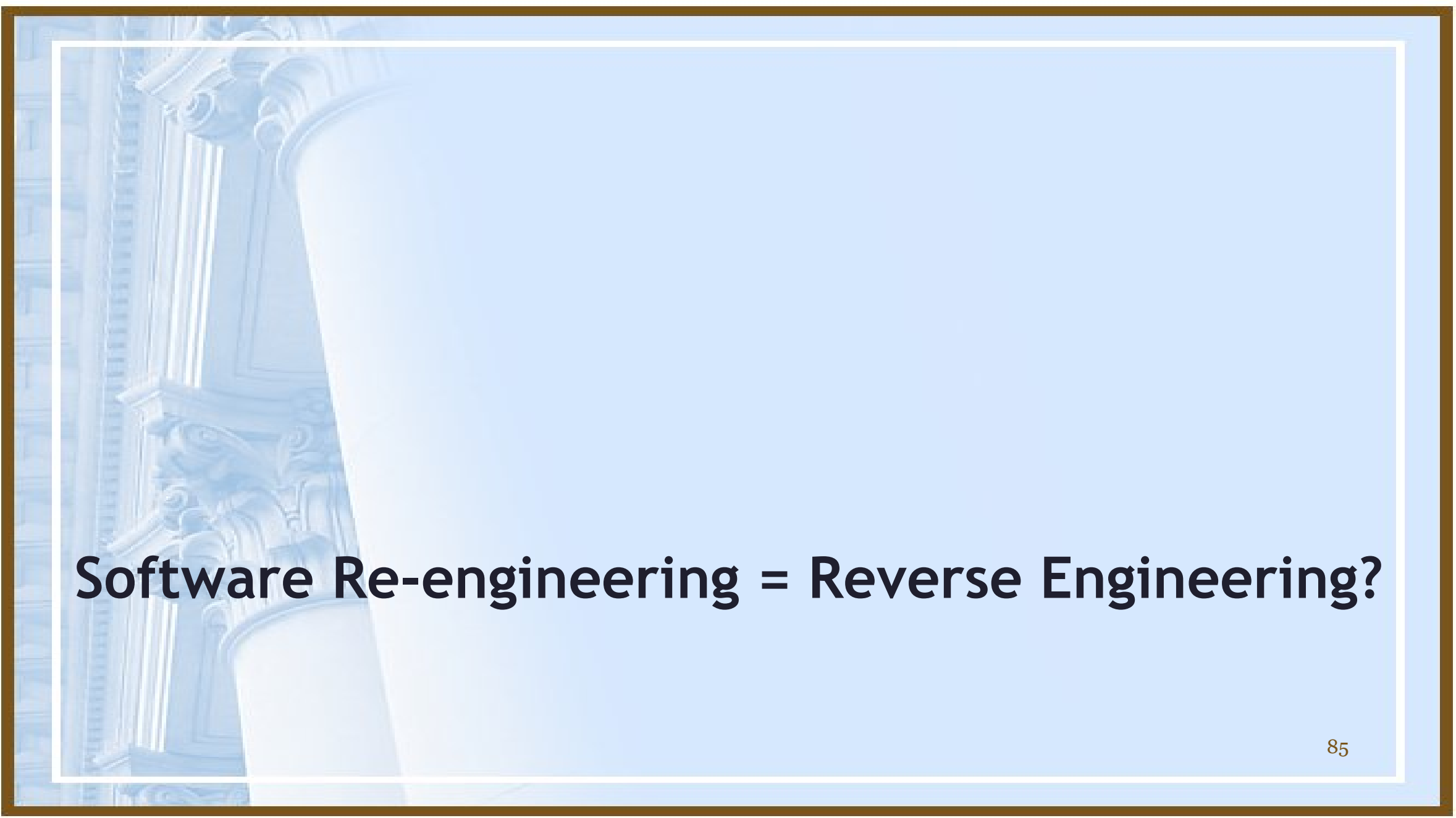


Advantages of Re-engineering



Disadvantages of Re-engineering



The background of the slide features a low-angle, perspective view of several tall, white classical columns with ornate capitals. The columns are set against a clear, bright blue sky. The image is slightly faded and serves as a decorative backdrop for the text.

Software Re-engineering = Reverse Engineering?

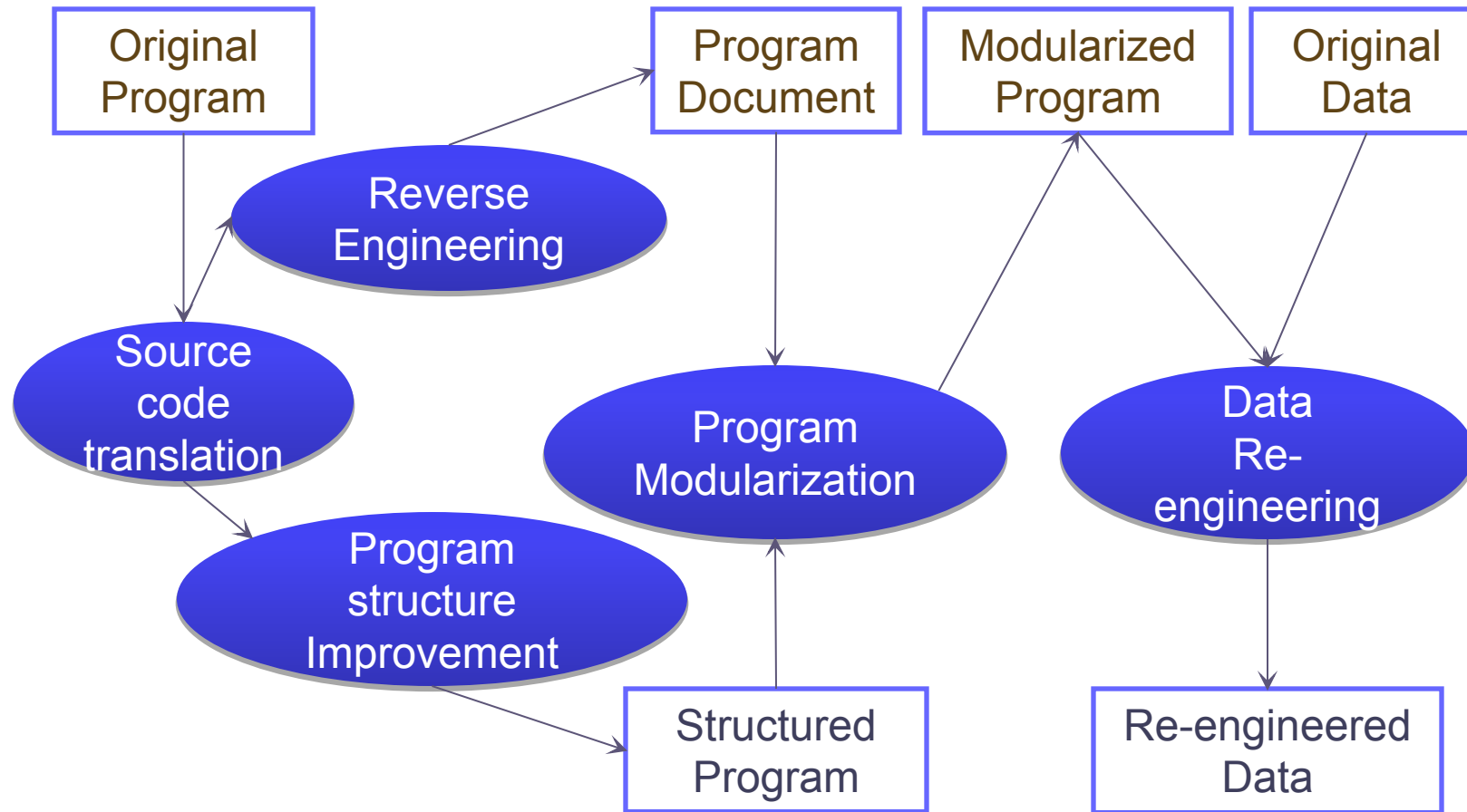
Software Re-engineering = Reverse Engineering?

- NO!
- The main objectives of system re-engineering are to **improve the system structure, create new system documentations and make it easier to understand**. The main methods of s/w re-engineering :
 - i) source code translation, ii) program restructuring, iii) data re-engineering and iv) reverse engineering
- However, the main objective of reverse engineering is to **derive system's design and specification** from its source code. It is normally part of s/w re-engineering process.

Software Revolution

- a) Introduction - Software Evolution Process
- b) Software Maintenance
- c) Software Reengineering
 - Source Code Translation
 - Program Structure Improvement
 - Reverse Engineering
 - Program Modularization
 - Data Re-engineering

Software Re-engineering Process



Legacy System Management



Scrap the system

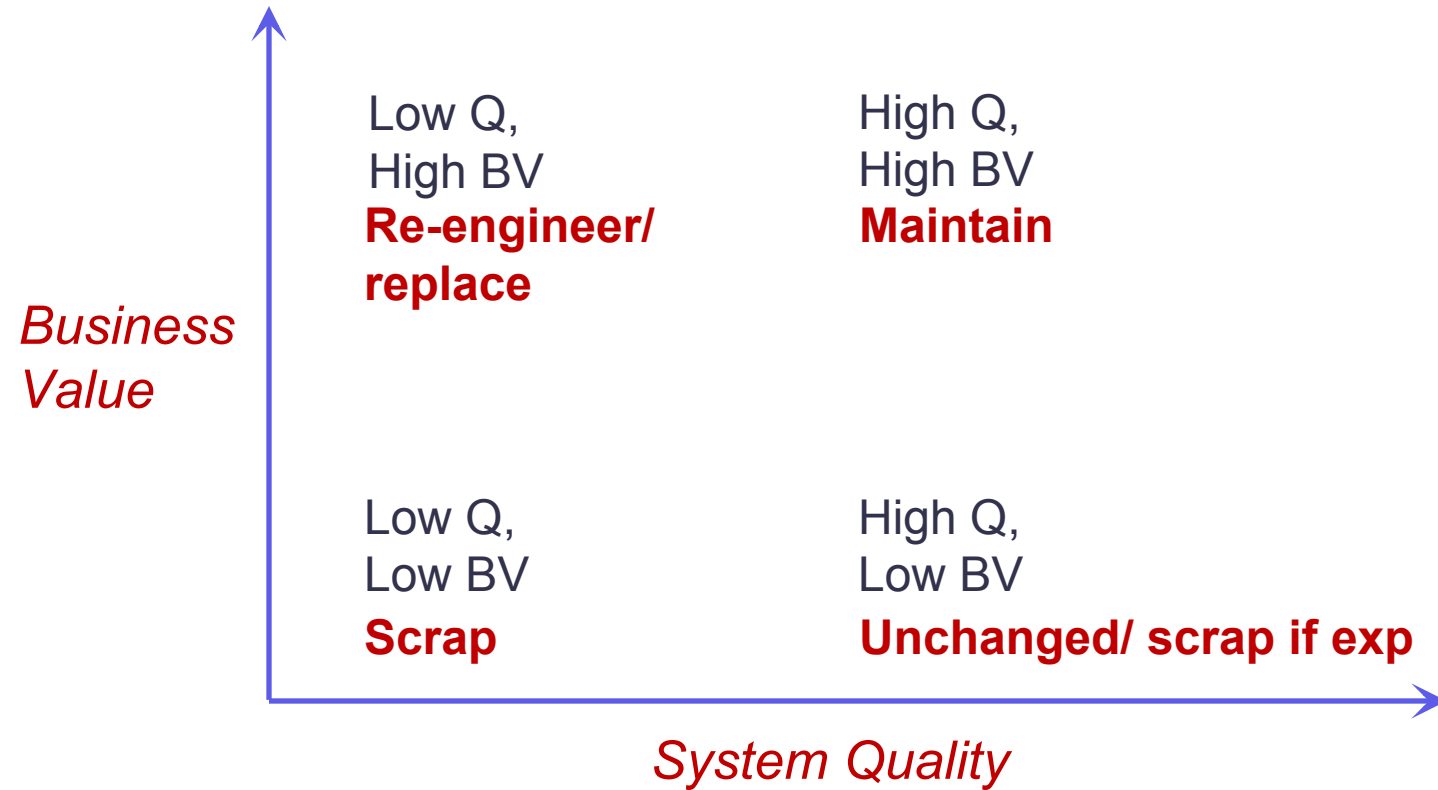
Leave it unchanged

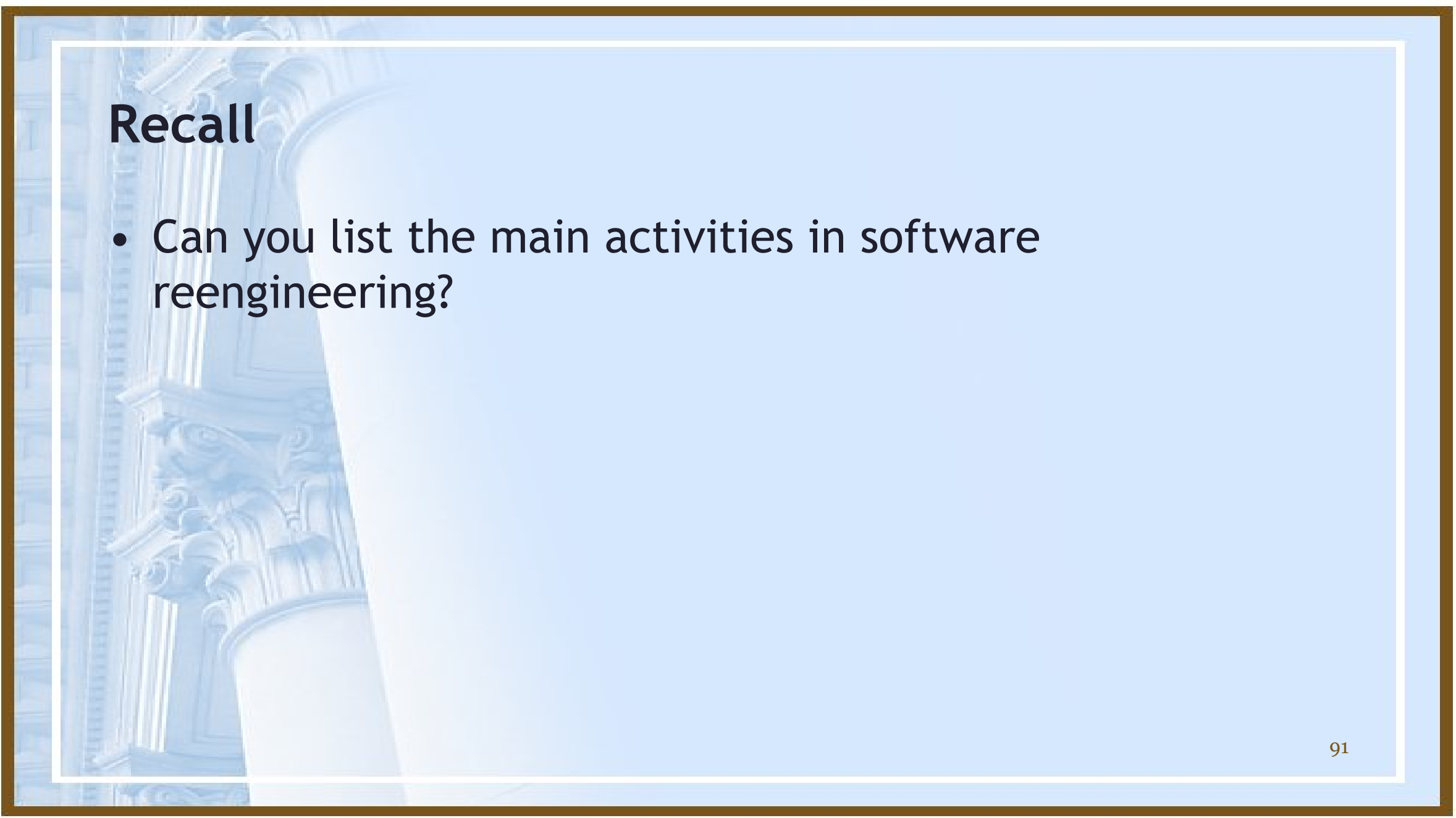
Re-engineer

Replace all/part

Legacy System Management

Legacy system assessment



The background of the slide features a low-angle, perspective view of several tall, white classical columns with Corinthian capitals. The columns are set against a clear, bright blue sky. The image is slightly faded and serves as a decorative backdrop for the text.

Recall

- Can you list the main activities in software reengineering?

Quiz

An online trading legacy system in organization ACE serves millions of customer since 1980s. The system maintenance cost is high due to incomplete documentation, poor structured coding and outdated programming language used. Suggest a strategy to the organization to evolve the system.

Hint: Scrap, Re-engineer, Maintain or Unchange